

\$\Huge Computational\$ \$\Huge Intelligence\$ \$\Huge Lab\$

\$\Huge Assignment\$ \$\Huge 2\$

Jakub Jabłoński group 1 Monday 11:30-13:00 GSI

2.1 MLP with multiple outputs

We will use [Fashion-MNIST dataset](#) from Zalando. The dataset contains labelled grayscale 28x28 images of clothing articles (10 classes).

```
import numpy as np
from tensorflow.keras.datasets import fashion_mnist

(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()
X=np.concatenate((np.array(X_train),np.array(X_test)),axis=0);
y=np.concatenate((np.array(y_train),np.array(y_test)),axis=0);
# X=X.reshape(X.shape[0],X.shape[1]*X.shape[2])
X=X/255

Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 _____ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 _____ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 _____ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 _____ 0s 0us/step

import matplotlib.pyplot as plt
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'
col2 = 15
row2 = 5
fig = plt.figure(figsize=(col2, row2))
for index2 in range(0, col2*row2):
    fig.add_subplot(row2, col2, index2 + 1)
    plt.axis('off')
    plt.imshow(X[index2]) # index of the sample image
plt.show()
```



How many classes?

```
print(max(y)+1)
# or

from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
le.fit(y)
print(f"{le.classes_} {len(le.classes_)}")

10
[0 1 2 3 4 5 6 7 8 9] 10
```

We create a neural net with 10 neurons in the output layer and *softmax* activation function.

As the loss, *sparse_categorical_crossentropy* is used, $-\sum_{i=1}^m \sum_{j=1}^k 1(y_i=j) \cdot \ln(p_j)$ where

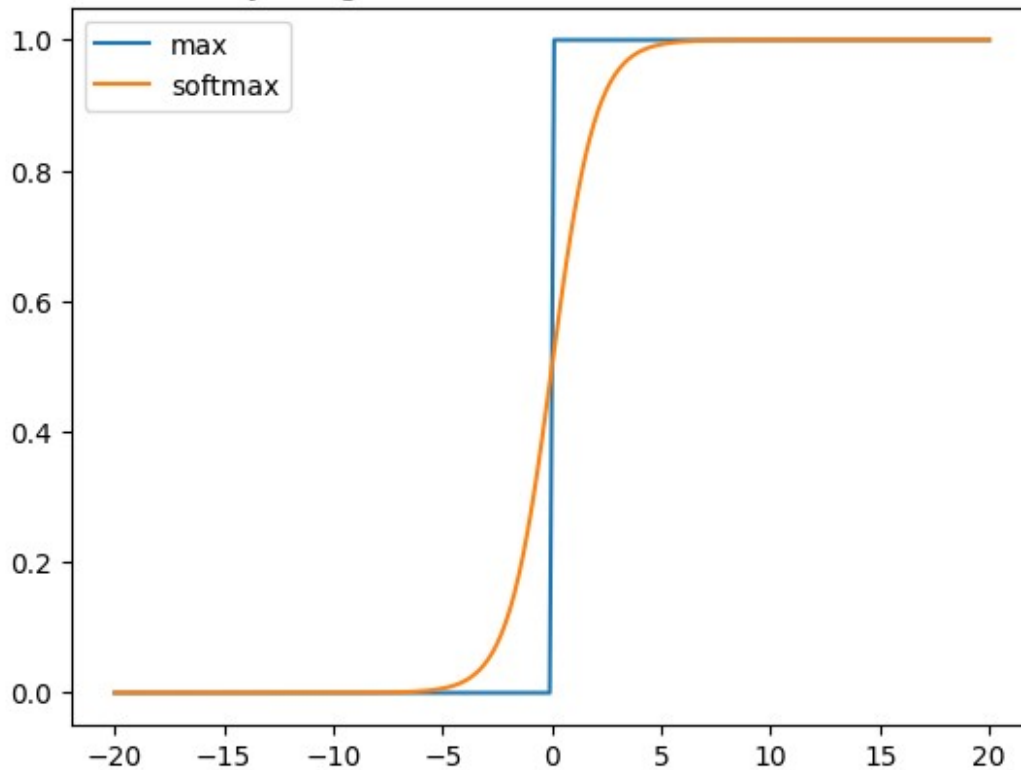
- m - number of observations
- k - number of labels
- $1(y_i=j)$ - indicator function, returns 1 if condition is true, i.e. label y_i is equal to j , otherwise 0

softmax selects a neuron with the highest probability value but it is smooth and differentiable

```
x=np.linspace(-20,20,200)
zeros = x*0
X_all=np.stack((zeros,x),axis=-1)

plt.title("The first neuron has the (constant) value 0, the second x.\nProbability assigned to the second neuron for various x")
plt.plot(x,np.argmax(X_all,axis=1),label='max')
plt.plot(x,np.exp(X_all[:,1])/((np.exp(X_all[:,0])
+np.exp(X_all[:,1]))),label='softmax')
plt.legend()
plt.show()
```

The first neuron has the (constant) value 0, the second x.
 Probability assigned to the second neuron for various x



2.1.1 Tensorflow based implementation

```
from keras import models
from keras import layers

num_classes = y_train.max()+1

model = models.Sequential()
model.add(layers.Input(shape=(28,28,)))
model.add(layers.Flatten())
model.add(layers.Dense(256, activation='relu' ))
model.add(layers.Dense(40, activation='relu'))
model.add(layers.Dense(num_classes, activation='softmax'))
model.compile(optimizer='rmsprop',loss='sparse_categorical_crossentropy',metrics=['accuracy'])
model.summary()
```

Model: "sequential_4"

Layer (type)	Output Shape
Param #	

0	flatten_4 (Flatten)	(None, 784)
200,960	dense_12 (Dense)	(None, 256)
10,280	dense_13 (Dense)	(None, 40)
410	dense_14 (Dense)	(None, 10)

Total params: 211,650 (826.76 KB)

Trainable params: 211,650 (826.76 KB)

Non-trainable params: 0 (0.00 B)

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.33, random_state=42)
```

y.shape

(70000,)

**** TODO **** Select number of epochs and batch size

Section 2.1

- epochs = 75, batch_size = 256

```
hist = model.fit(X_train,y_train,epochs=60,batch_size=256, verbose=1)
```

Epoch 1/60

184/184 ————— 3s 7ms/step - accuracy: 0.6637 - loss: 0.9739

Epoch 2/60

184/184 ————— 1s 3ms/step - accuracy: 0.8236 - loss: 0.4889

Epoch 3/60

184/184 ————— 0s 2ms/step - accuracy: 0.8518 - loss: 0.4162

Epoch 4/60

184/184 ————— 0s 3ms/step - accuracy: 0.8623 - loss:

```
0.3830
Epoch 5/60
184/184 ————— 0s 2ms/step - accuracy: 0.8709 - loss:
0.3487
Epoch 6/60
184/184 ————— 0s 3ms/step - accuracy: 0.8795 - loss:
0.3292
Epoch 7/60
184/184 ————— 0s 2ms/step - accuracy: 0.8836 - loss:
0.3168
Epoch 8/60
184/184 ————— 0s 3ms/step - accuracy: 0.8909 - loss:
0.2943
Epoch 9/60
184/184 ————— 0s 3ms/step - accuracy: 0.8922 - loss:
0.2908
Epoch 10/60
184/184 ————— 1s 2ms/step - accuracy: 0.8954 - loss:
0.2843
Epoch 11/60
184/184 ————— 0s 2ms/step - accuracy: 0.9018 - loss:
0.2642
Epoch 12/60
184/184 ————— 0s 2ms/step - accuracy: 0.9062 - loss:
0.2542
Epoch 13/60
184/184 ————— 0s 2ms/step - accuracy: 0.9042 - loss:
0.2558
Epoch 14/60
184/184 ————— 0s 2ms/step - accuracy: 0.9113 - loss:
0.2424
Epoch 15/60
184/184 ————— 0s 2ms/step - accuracy: 0.9121 - loss:
0.2394
Epoch 16/60
184/184 ————— 0s 2ms/step - accuracy: 0.9168 - loss:
0.2253
Epoch 17/60
184/184 ————— 0s 2ms/step - accuracy: 0.9182 - loss:
0.2225
Epoch 18/60
184/184 ————— 0s 2ms/step - accuracy: 0.9175 - loss:
0.2149
Epoch 19/60
184/184 ————— 0s 2ms/step - accuracy: 0.9229 - loss:
0.2083
Epoch 20/60
184/184 ————— 0s 2ms/step - accuracy: 0.9263 - loss:
0.1962
```

```
Epoch 21/60
184/184 ————— 1s 2ms/step - accuracy: 0.9279 - loss:
0.1935
Epoch 22/60
184/184 ————— 0s 2ms/step - accuracy: 0.9297 - loss:
0.1834
Epoch 23/60
184/184 ————— 1s 3ms/step - accuracy: 0.9290 - loss:
0.1869
Epoch 24/60
184/184 ————— 1s 4ms/step - accuracy: 0.9317 - loss:
0.1807
Epoch 25/60
184/184 ————— 1s 4ms/step - accuracy: 0.9351 - loss:
0.1726
Epoch 26/60
184/184 ————— 1s 3ms/step - accuracy: 0.9368 - loss:
0.1703
Epoch 27/60
184/184 ————— 0s 2ms/step - accuracy: 0.9367 - loss:
0.1707
Epoch 28/60
184/184 ————— 0s 2ms/step - accuracy: 0.9401 - loss:
0.1627
Epoch 29/60
184/184 ————— 0s 2ms/step - accuracy: 0.9406 - loss:
0.1588
Epoch 30/60
184/184 ————— 0s 3ms/step - accuracy: 0.9428 - loss:
0.1522
Epoch 31/60
184/184 ————— 0s 2ms/step - accuracy: 0.9442 - loss:
0.1517
Epoch 32/60
184/184 ————— 0s 2ms/step - accuracy: 0.9470 - loss:
0.1443
Epoch 33/60
184/184 ————— 0s 2ms/step - accuracy: 0.9467 - loss:
0.1431
Epoch 34/60
184/184 ————— 0s 2ms/step - accuracy: 0.9464 - loss:
0.1410
Epoch 35/60
184/184 ————— 0s 2ms/step - accuracy: 0.9480 - loss:
0.1398
Epoch 36/60
184/184 ————— 0s 3ms/step - accuracy: 0.9515 - loss:
0.1276
Epoch 37/60
```

```
184/184 ————— 0s 2ms/step - accuracy: 0.9516 - loss: 0.1291
Epoch 38/60
184/184 ————— 1s 3ms/step - accuracy: 0.9538 - loss: 0.1234
Epoch 39/60
184/184 ————— 0s 2ms/step - accuracy: 0.9531 - loss: 0.1255
Epoch 40/60
184/184 ————— 0s 2ms/step - accuracy: 0.9555 - loss: 0.1204
Epoch 41/60
184/184 ————— 0s 3ms/step - accuracy: 0.9541 - loss: 0.1225
Epoch 42/60
184/184 ————— 0s 2ms/step - accuracy: 0.9567 - loss: 0.1130
Epoch 43/60
184/184 ————— 0s 3ms/step - accuracy: 0.9563 - loss: 0.1161
Epoch 44/60
184/184 ————— 0s 2ms/step - accuracy: 0.9582 - loss: 0.1119
Epoch 45/60
184/184 ————— 0s 2ms/step - accuracy: 0.9622 - loss: 0.1029
Epoch 46/60
184/184 ————— 0s 2ms/step - accuracy: 0.9590 - loss: 0.1076
Epoch 47/60
184/184 ————— 1s 3ms/step - accuracy: 0.9613 - loss: 0.1041
Epoch 48/60
184/184 ————— 1s 3ms/step - accuracy: 0.9608 - loss: 0.1016
Epoch 49/60
184/184 ————— 1s 4ms/step - accuracy: 0.9611 - loss: 0.1016
Epoch 50/60
184/184 ————— 1s 3ms/step - accuracy: 0.9618 - loss: 0.1023
Epoch 51/60
184/184 ————— 0s 2ms/step - accuracy: 0.9645 - loss: 0.0936
Epoch 52/60
184/184 ————— 0s 2ms/step - accuracy: 0.9655 - loss: 0.0945
Epoch 53/60
184/184 ————— 0s 2ms/step - accuracy: 0.9664 - loss:
```

```

0.0899
Epoch 54/60
184/184 _____ 0s 2ms/step - accuracy: 0.9678 - loss:
0.0855
Epoch 55/60
184/184 _____ 0s 3ms/step - accuracy: 0.9658 - loss:
0.0898
Epoch 56/60
184/184 _____ 1s 3ms/step - accuracy: 0.9660 - loss:
0.0887
Epoch 57/60
184/184 _____ 0s 2ms/step - accuracy: 0.9682 - loss:
0.0866
Epoch 58/60
184/184 _____ 0s 2ms/step - accuracy: 0.9693 - loss:
0.0817
Epoch 59/60
184/184 _____ 0s 2ms/step - accuracy: 0.9689 - loss:
0.0844
Epoch 60/60
184/184 _____ 0s 2ms/step - accuracy: 0.9705 - loss:
0.0795

```

```
test_proba = model.predict(X_test)
```

```
722/722 _____ 2s 2ms/step
```

test_proba contains probabilities assigned to output neurons.

```

import pandas as pd

df = pd.DataFrame(test_proba)
df.head()

{"summary":{"\n  \"name\": \"df\",\n  \"rows\": 23100,\n  \"fields\": [\n    {\n      \"column\": 0,\n      \"properties\": {\n        \"dtype\": \"float32\",\n        \"num_unique_values\": 22984,\n        \"samples\": [\n          1.4002216630615294e-06,\n          2.0707157091237605e-06,\n          3.248155731365311e-26\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": 1,\n      \"properties\": {\n        \"dtype\": \"float32\",\n        \"num_unique_values\": 20967,\n        \"samples\": [\n          1.2065031196328846e-09,\n          5.257307176403936e-13,\n          1.3373196994237446e-09\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": 2,\n      \"properties\": {\n        \"dtype\": \"float32\",\n        \"num_unique_values\": 23083,\n        \"samples\": [\n          2.9445850190665604e-14,\n          2.0564702651881817e-07,\n          0.5373303294181824\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ]\n}}

```

```

n    },\n    {\n        \"column\": 3,\n        \"properties\": {\n            \"dtype\": \"float32\",\n            \"num_unique_values\": 22398,\n            \"samples\": [\n                2.1804762582178228e-06,\n                4.8687245346457075e-09,\n                4.735945565977816e-19\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\"\n        }\n    },\n    {\n        \"column\": 4,\n        \"properties\": {\n            \"dtype\": \"float32\",\n            \"num_unique_values\": 23013,\n            \"samples\": [\n                1.106247524140791e-10,\n                1.5144753220537838e-16,\n                0.9813165664672852\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\"\n        }\n    },\n    {\n        \"column\": 5,\n        \"properties\": {\n            \"dtype\": \"float32\",\n            \"num_unique_values\": 21245,\n            \"samples\": [\n                1.2674432170456384e-08,\n                4.984749835124092e-10,\n                9.528850148168844e-13\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\"\n        }\n    },\n    {\n        \"column\": 6,\n        \"properties\": {\n            \"dtype\": \"float32\",\n            \"num_unique_values\": 22759,\n            \"samples\": [\n                1.9874696590704843e-06,\n                1.8096252229021075e-24,\n                2.0140027148966327e-20\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\"\n        }\n    },\n    {\n        \"column\": 7,\n        \"properties\": {\n            \"dtype\": \"float32\",\n            \"num_unique_values\": 21612,\n            \"samples\": [\n                7.281177885013618e-14,\n                8.890245226211846e-05,\n                2.1474120259767915e-08\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\"\n        }\n    },\n    {\n        \"column\": 8,\n        \"properties\": {\n            \"dtype\": \"float32\",\n            \"num_unique_values\": 21254,\n            \"samples\": [\n                1.1370585107018305e-15,\n                3.1923496771479554e-26,\n                1.749690170527174e-09\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\"\n        }\n    },\n    {\n        \"column\": 9,\n        \"properties\": {\n            \"dtype\": \"float32\",\n            \"num_unique_values\": 21548,\n            \"samples\": [\n                1.6701449944163035e-19,\n                1.1591770032737259e-07,\n                1.419399583852724e-32\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\"\n        }\n    }\n]\n\"type\": \"dataframe\", \"variable_name\": \"df\"}

```

```
y_test_pred = np.argmax(test_proba,axis=1)
```

Confusion matrix

```

import seaborn as sns
from sklearn import metrics

def show_confusion_matrix(matrix, labels=None):
    num_classes = matrix.shape[0]
    if labels is None:
        labels = [str(i) for i in range(num_classes)]
    plt.figure(figsize=(num_classes, num_classes))

```

```

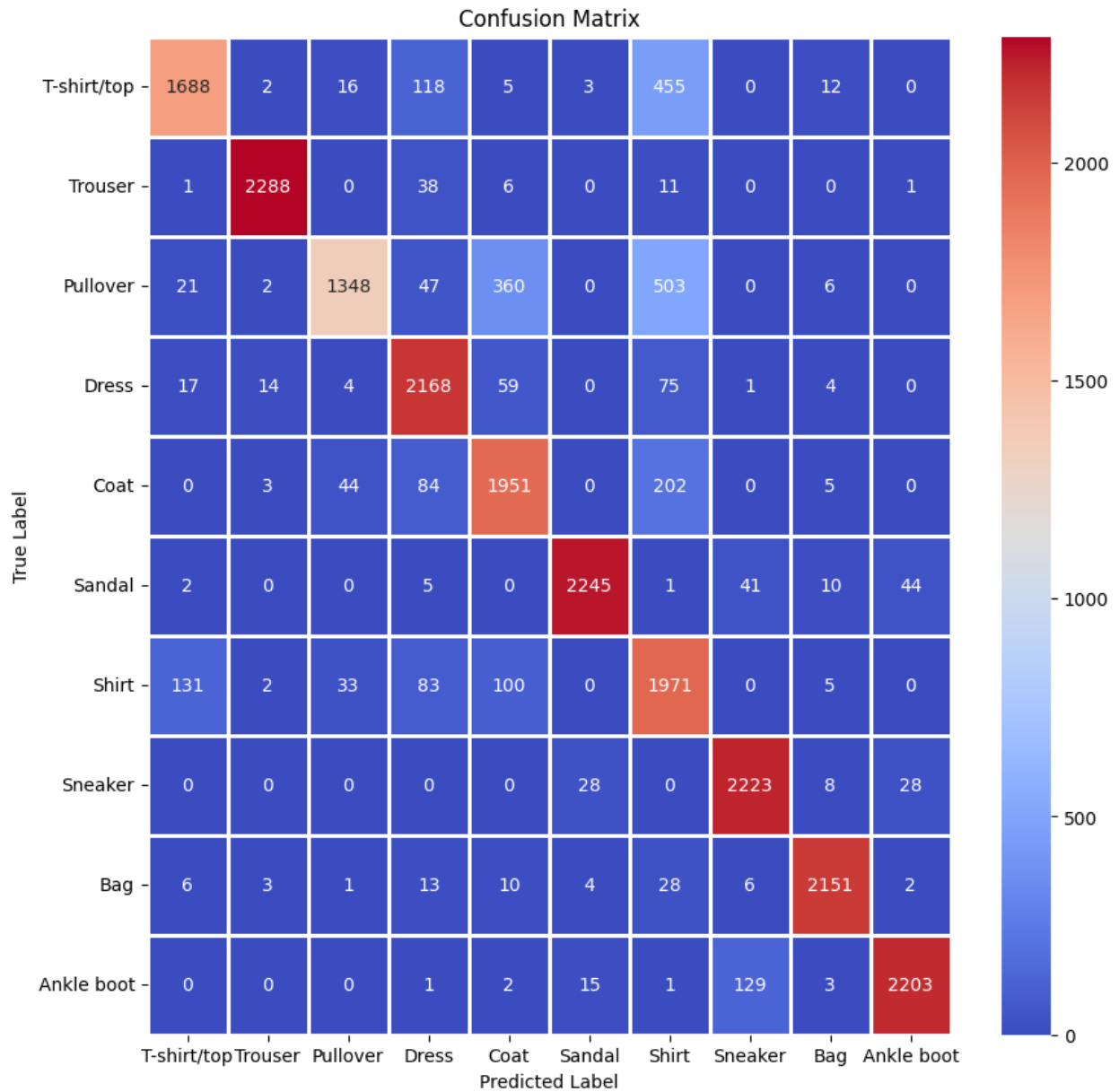
    hm = sns.heatmap(matrix,
                      cmap='coolwarm',
                      linecolor='white',
                      linewidths=1,
                      xticklabels=labels[0:num_classes],
                      yticklabels=labels[0:num_classes],
                      annot=True,
                      fmt='d')
    plt.yticks(rotation = 0) # Don't rotate (vertically) the y-axis labels
    # hm.set_ylim(0, len(matrix))
    plt.title('Confusion Matrix')
    plt.ylabel('True Label')
    plt.xlabel('Predicted Label')
    plt.show()

from sklearn.metrics import confusion_matrix

LABELS= ["T-shirt/top",
          "Trouser",
          "Pullover",
          "Dress",
          "Coat",
          "Sandal",
          "Shirt",
          "Sneaker",
          "Bag",
          "Ankle boot"]

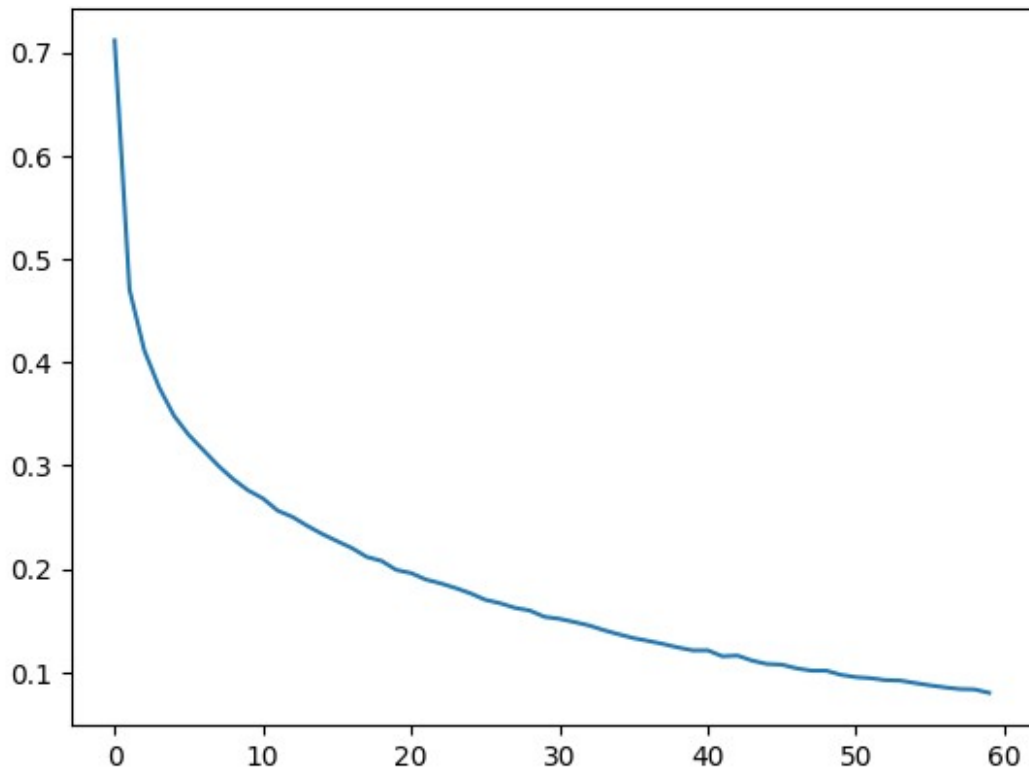
show_confusion_matrix(confusion_matrix(y_test,y_test_pred),LABELS)

```



TODO Analyze cases of misclassification

```
plt.plot(hist.history['loss'])  
plt.show()
```



2.1.2 PyTorch based implementation

```
import torch
import torch.nn as nn
import torch.optim as optim

# --- parametry ---
input_shape = (X.shape[0], X.shape[1])
num_classes = int(y_train.max()) + 1

# --- model ---
class SimpleNet(nn.Module):
    def __init__(self, num_classes):
        super(SimpleNet, self).__init__()
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(28*28, 256)
        self.fc2 = nn.Linear(256, 40)
        self.fc3 = nn.Linear(40, num_classes)
        self.relu = nn.ReLU()
        self.softmax = nn.Softmax(dim=1)

    def forward(self, x):
        x = self.flatten(x)
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        x = self.fc3(x)
```

```

        x = self.softmax(x)
        return x

model = SimpleNet(num_classes)
print(model)

SimpleNet(
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (fc1): Linear(in_features=784, out_features=256, bias=True)
  (fc2): Linear(in_features=256, out_features=40, bias=True)
  (fc3): Linear(in_features=40, out_features=10, bias=True)
  (relu): ReLU()
  (softmax): Softmax(dim=1)
)

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.33, random_state=42)

```

- `torch.TensorDataset` stores X and y tensors in memory. It is much simpler than equivalent `tf.Dataset`, which can provide access to any data regardless of their location.
- `torch.DataLoader` provides functionality analogous to `tf.Dataset`: forming batches and buffer shuffling.

```

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader

# conversion from numpy to PyTorch tensors
X_train_t = torch.tensor(X_train, dtype=torch.float32)
y_train_t = torch.tensor(y_train, dtype=torch.long)
X_test_t = torch.tensor(X_test, dtype=torch.float32)
y_test_t = torch.tensor(y_test, dtype=torch.long)

# Dataset
train_dataset = TensorDataset(X_train_t, y_train_t)
test_dataset = TensorDataset(X_test_t, y_test_t)

# DataLoader
train_loader = DataLoader(train_dataset, batch_size=256)
test_loader = DataLoader(test_dataset, batch_size=len(y_test),
num_workers=2)

num_classes = int(y_train.max()) + 1
model = SimpleNet(num_classes)

```

```

criterion = nn.CrossEntropyLoss() # sparse_categorical_crossentropy
optimizer = optim.RMSprop(model.parameters(), lr=0.001)

# if you use torch 2.0+ you may compile the model to create
# computation graph as in TF
# model = torch.compile(model)

n_epochs = 60

for epoch in range(n_epochs):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for X_batch, y_batch in train_loader:
        optimizer.zero_grad()
        outputs = model(X_batch)
        loss = criterion(outputs, y_batch)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        _, predicted = torch.max(outputs, 1)
        total += y_batch.size(0)
        correct += (predicted == y_batch).sum().item()

    acc = 100 * correct / total
    print(f"Epoch {epoch+1}/{n_epochs}, Loss:
{running_loss/len(train_loader):.4f}, Accuracy: {acc:.2f}%")

```

```

Epoch 1/60, Loss: 1.7632, Accuracy: 71.29%
Epoch 2/60, Loss: 1.6590, Accuracy: 81.13%
Epoch 3/60, Loss: 1.6339, Accuracy: 83.28%
Epoch 4/60, Loss: 1.6221, Accuracy: 84.34%
Epoch 5/60, Loss: 1.6142, Accuracy: 85.05%
Epoch 6/60, Loss: 1.6089, Accuracy: 85.56%
Epoch 7/60, Loss: 1.6029, Accuracy: 86.12%
Epoch 8/60, Loss: 1.5990, Accuracy: 86.48%
Epoch 9/60, Loss: 1.5964, Accuracy: 86.62%
Epoch 10/60, Loss: 1.5931, Accuracy: 87.00%
Epoch 11/60, Loss: 1.5904, Accuracy: 87.24%
Epoch 12/60, Loss: 1.5883, Accuracy: 87.47%
Epoch 13/60, Loss: 1.5867, Accuracy: 87.59%
Epoch 14/60, Loss: 1.5830, Accuracy: 87.97%
Epoch 15/60, Loss: 1.5824, Accuracy: 88.01%
Epoch 16/60, Loss: 1.5795, Accuracy: 88.34%
Epoch 17/60, Loss: 1.5777, Accuracy: 88.52%
Epoch 18/60, Loss: 1.5759, Accuracy: 88.64%
Epoch 19/60, Loss: 1.5745, Accuracy: 88.77%

```

```
Epoch 20/60, Loss: 1.5737, Accuracy: 88.86%
Epoch 21/60, Loss: 1.5714, Accuracy: 89.08%
Epoch 22/60, Loss: 1.5696, Accuracy: 89.27%
Epoch 23/60, Loss: 1.5687, Accuracy: 89.38%
Epoch 24/60, Loss: 1.5669, Accuracy: 89.58%
Epoch 25/60, Loss: 1.5651, Accuracy: 89.68%
Epoch 26/60, Loss: 1.5648, Accuracy: 89.77%
Epoch 27/60, Loss: 1.5629, Accuracy: 89.96%
Epoch 28/60, Loss: 1.5631, Accuracy: 89.91%
Epoch 29/60, Loss: 1.5612, Accuracy: 90.11%
Epoch 30/60, Loss: 1.5598, Accuracy: 90.23%
Epoch 31/60, Loss: 1.5590, Accuracy: 90.28%
Epoch 32/60, Loss: 1.5587, Accuracy: 90.34%
Epoch 33/60, Loss: 1.5576, Accuracy: 90.43%
Epoch 34/60, Loss: 1.5576, Accuracy: 90.41%
Epoch 35/60, Loss: 1.5560, Accuracy: 90.59%
Epoch 36/60, Loss: 1.5556, Accuracy: 90.62%
Epoch 37/60, Loss: 1.5540, Accuracy: 90.80%
Epoch 38/60, Loss: 1.5531, Accuracy: 90.88%
Epoch 39/60, Loss: 1.5526, Accuracy: 90.91%
Epoch 40/60, Loss: 1.5515, Accuracy: 91.04%
Epoch 41/60, Loss: 1.5519, Accuracy: 90.99%
Epoch 42/60, Loss: 1.5512, Accuracy: 91.07%
Epoch 43/60, Loss: 1.5497, Accuracy: 91.17%
Epoch 44/60, Loss: 1.5491, Accuracy: 91.31%
Epoch 45/60, Loss: 1.5486, Accuracy: 91.30%
Epoch 46/60, Loss: 1.5475, Accuracy: 91.45%
Epoch 47/60, Loss: 1.5472, Accuracy: 91.46%
Epoch 48/60, Loss: 1.5474, Accuracy: 91.42%
Epoch 49/60, Loss: 1.5457, Accuracy: 91.56%
Epoch 50/60, Loss: 1.5454, Accuracy: 91.61%
Epoch 51/60, Loss: 1.5455, Accuracy: 91.63%
Epoch 52/60, Loss: 1.5449, Accuracy: 91.67%
Epoch 53/60, Loss: 1.5455, Accuracy: 91.64%
Epoch 54/60, Loss: 1.5437, Accuracy: 91.78%
Epoch 55/60, Loss: 1.5436, Accuracy: 91.80%
Epoch 56/60, Loss: 1.5433, Accuracy: 91.78%
Epoch 57/60, Loss: 1.5420, Accuracy: 91.94%
Epoch 58/60, Loss: 1.5417, Accuracy: 91.98%
Epoch 59/60, Loss: 1.5414, Accuracy: 92.01%
Epoch 60/60, Loss: 1.5406, Accuracy: 92.12%
```

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt
```

```
model.eval()
correct = 0
total = 0
test_loss = 0.0
```

```

y_true = []
y_pred = []

with torch.no_grad():
    for X_batch, y_batch in test_loader:
        outputs = model(X_batch)
        loss = criterion(outputs, y_batch)
        test_loss += loss.item()

        predicted = outputs.argmax(dim=1)
        total += y_batch.size(0)
        correct += (predicted == y_batch).sum().item()

        # Zbieramy dane do analizy
        y_true.extend(y_batch.cpu().numpy())
        y_pred.extend(predicted.cpu().numpy())

# --- metrics ---
print(f"Test Loss: {test_loss/len(test_loader):.4f}, Test Accuracy:
{100 * correct / total:.2f}%")

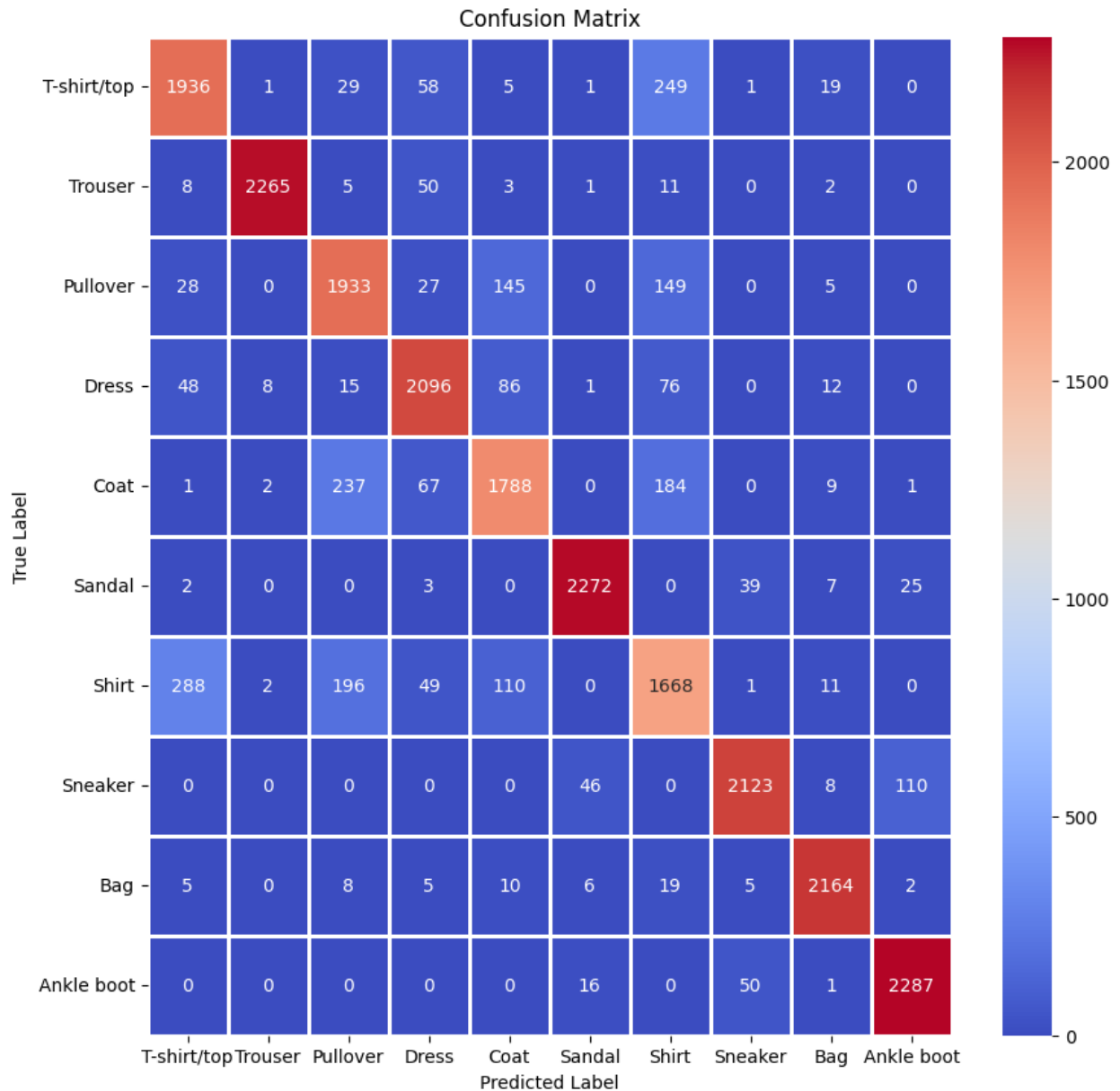
# --- confusion matrix ---
cm = confusion_matrix(y_true, y_pred)

LABELS= ["T-shirt/top",
"Trouser",
"Pullover",
"Dress",
"Coat",
"Sandal",
"Shirt",
"Sneaker",
"Bag",
"Ankle boot"]

show_confusion_matrix(cm, labels=LABELS)

Test Loss: 1.5724, Test Accuracy: 88.88%

```



TODO Experiment with various numbers of epochs and batch sizes

2.2 Convolutional Neural Networks – introduction

```
#Load an image
from PIL import Image
import requests
from io import BytesIO
```

```
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize'] = (5, 5)

response =
requests.get('https://cdn.pixabay.com/photo/2016/05/11/18/57/ella-
fitzgerald-1386214_1280.jpg')
img = Image.open(BytesIO(response.content))

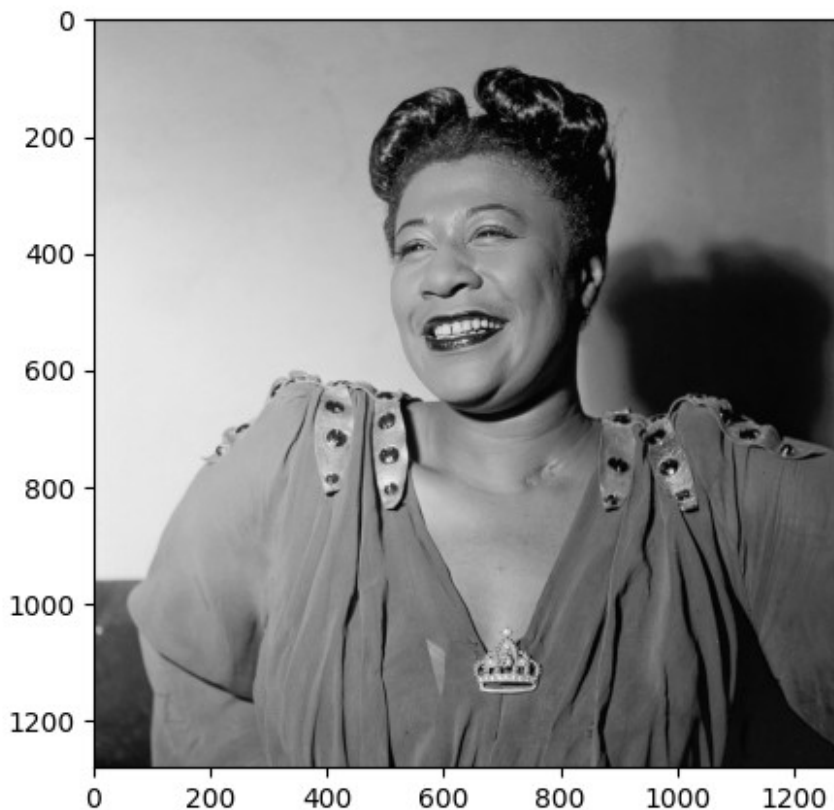
plt.imshow(img)
<matplotlib.image.AxesImage at 0x7dd3bac7a840>
```



```
import numpy as np
X_img = np.array(img)
print(f'X_img.shape={X_img.shape}')

X=X_img[:, :, 0]
print(f'X.shape={X.shape}')
plt.rcParams['image.cmap'] = 'gray'
plt.imshow(X)
plt.show()
```

```
X_img.shape=(1280, 1280, 3)
X.shape=(1280, 1280)
```



2.2.1 Convolution 2D

During the convolution a small filter F slides along an image X . Then the filter elements are multiplied by corresponding pixel values and summed. Convolution can be used to blurr an image, but also detect edges or extract gradients of pixel intensity.

Diagram

```
def conv2D(X,F,bias=0):
    w = F.shape[1]//2
    h = F.shape[0]//2
    Y=[ [ np.sum(X[i-h:i+h+1,j-w:j+w+1]*F) + bias for j in
np.arange(w,X.shape[1]-w)] for i in np.arange(h,X.shape[0]-h)]
    return np.array(Y)

def apply_two_convolutions(X):
    F_sobel_y=np.array([[ -1,-2,-1],[0,0,0],[1,2,1]])
    F_sobel_x=np.array([[ -1,0,1],[-2,0,2],[-1,0,1]])

    XX = conv2D(X,F_sobel_x,1)
    XY = conv2D(X,F_sobel_y,1)
```

```

    return XX,XY

AX,AY = apply_two_convolutions(X)

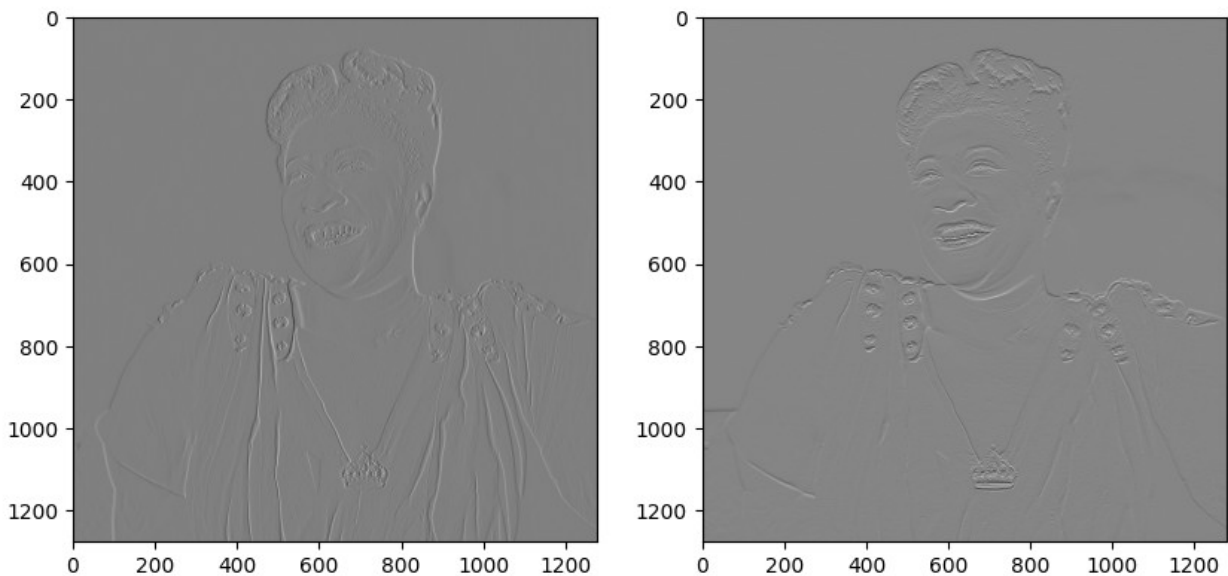
print(f'X.shape={X.shape} AX.shape={AX.shape} AY.shape={AY.shape}')

X.shape=(1280, 1280) AX.shape=(1278, 1278) AY.shape=(1278, 1278)

def plot(X,Y,cmap='gray'):
    fig = plt.figure(figsize=(10,10))
    plt.set_cmap(cmap)
    fig.add_subplot(1,2,1)
    # plt.axis('off')
    plt.imshow(X)
    fig.add_subplot(1,2,2)
    plt.imshow(Y)

plot(AX,AY)

```



During max pooling a window slides through an image (the strides are equal to the window sizes). At each position the maximal value of the pixel within the window is selected. Max pooling shrinks the image size.

```

def maxPool2D(X,shape):
    w = shape[1]
    h = shape[0]
    # print(f'h={h} w={w}')
    Y=[ [ np.max(X[i:i+h+1,j:j+w+1]) for j in
np.arange(0,X.shape[1],shape[1])] for i in
np.arange(0,X.shape[0],shape[0])]
    return np.array(Y)

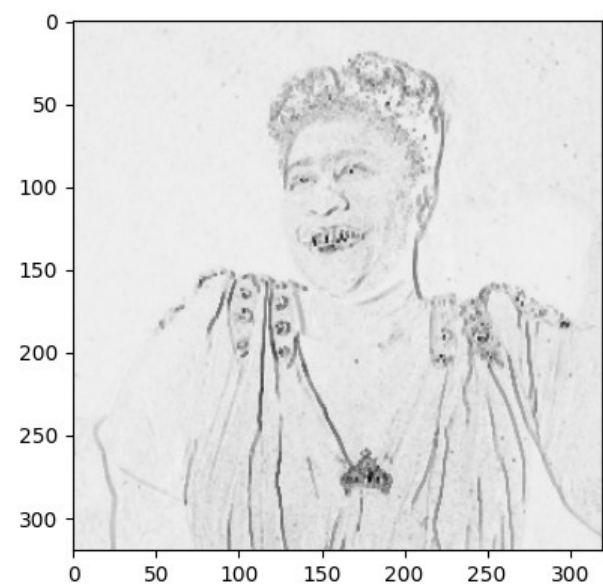
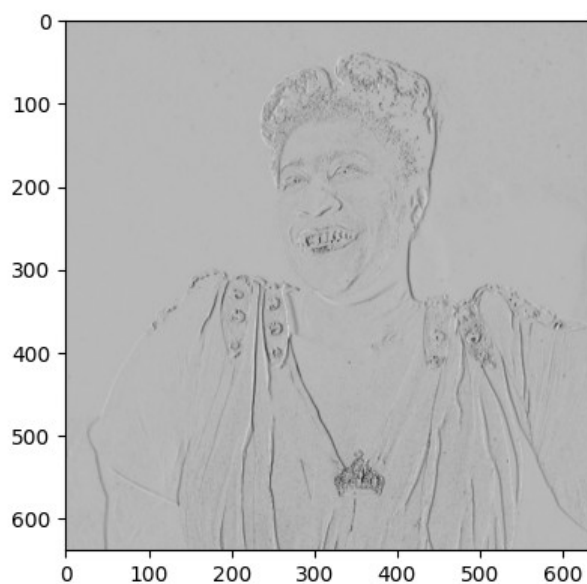
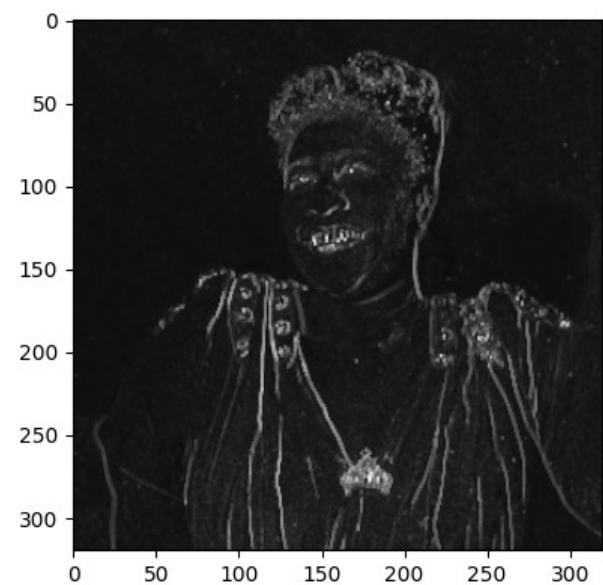
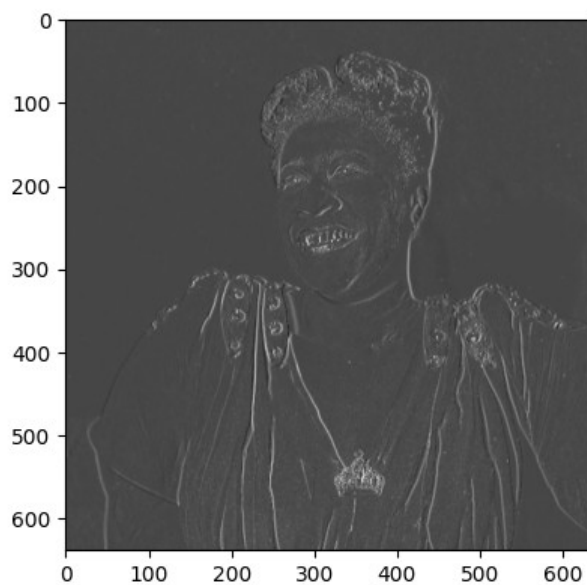
```

```

AX=maxPool2D(AX,(2,2))
AY=maxPool2D(AX,(2,2))
plot(AX,AY)
plot(AX,AY,cmap='gray_r')
print(f'image size:{AX.shape}')

```

image size:(639, 639)



We will repeat the convolution and max-pooling steps several times...

```

AX,AY = apply_two_convolutions(AX)
AX=maxPool2D(AX,(2,2))

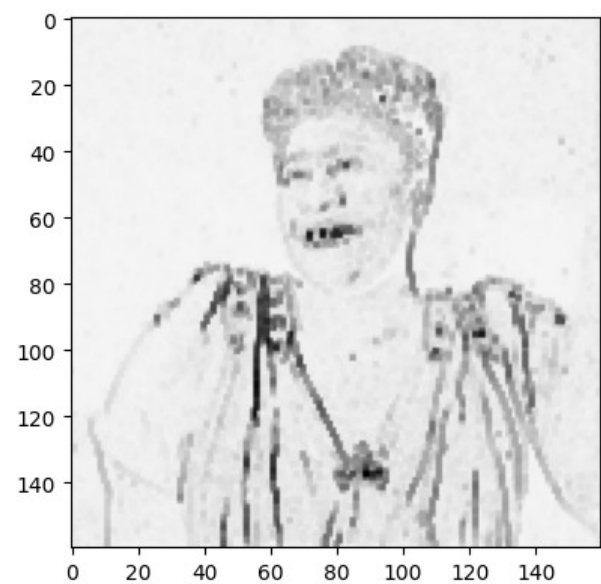
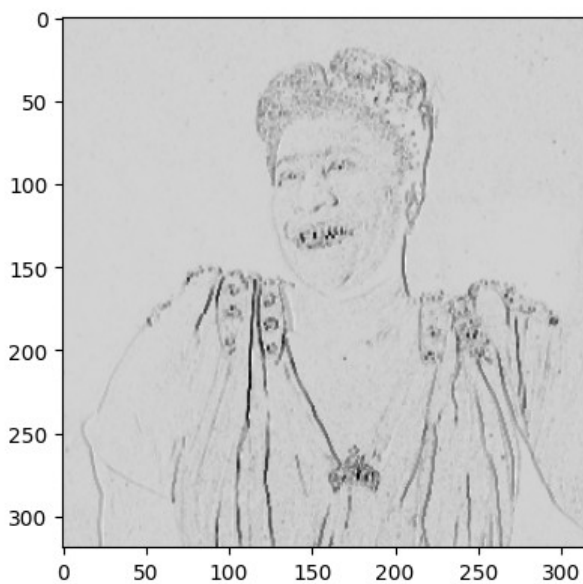
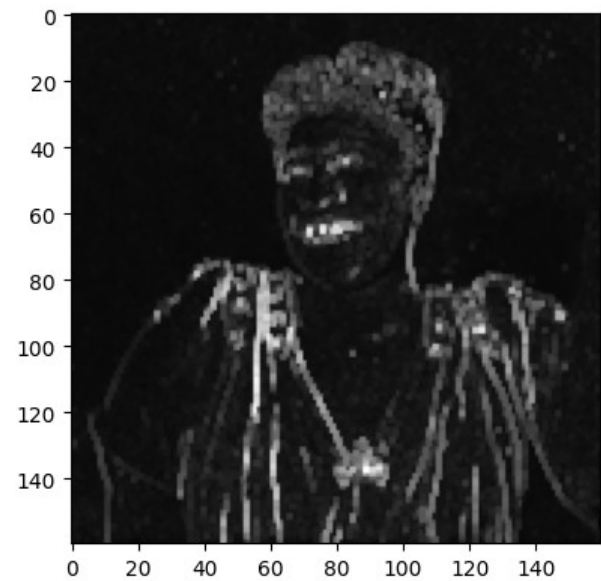
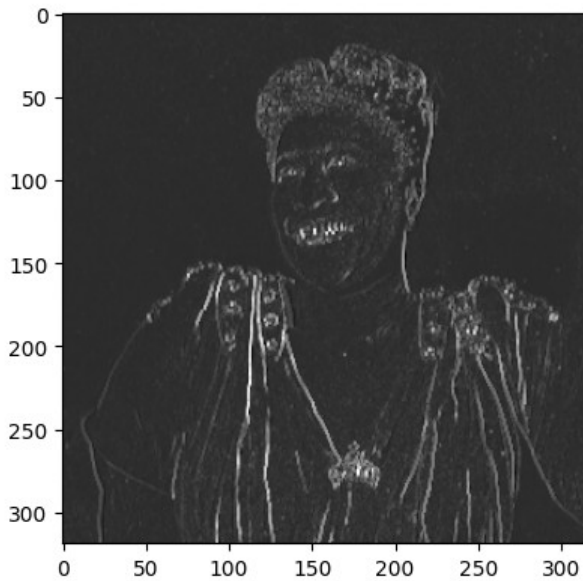
```

```

AY=maxPool2D(AX,(2,2))
plot(AX,AY)
plot(AX,AY,cmap='gray_r')
print(f'image size:{AX.shape}')

```

image size:(319, 319)



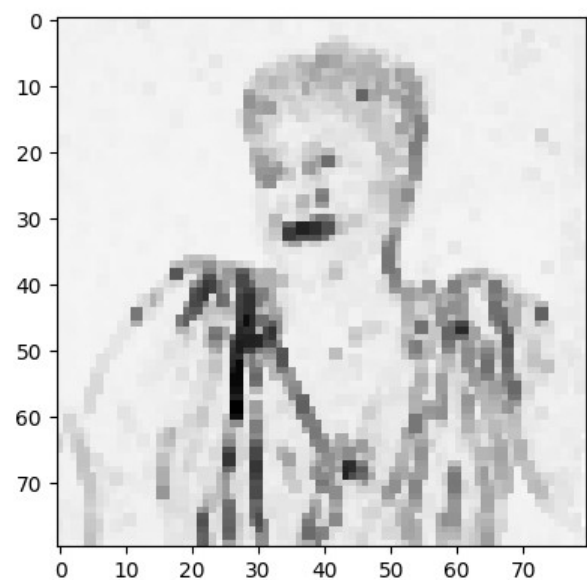
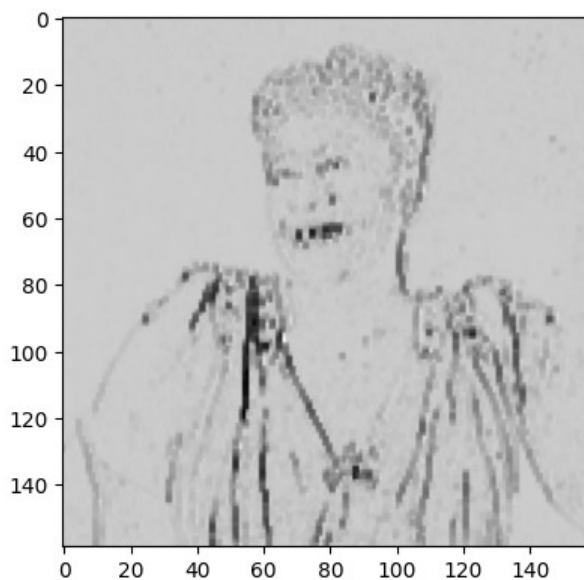
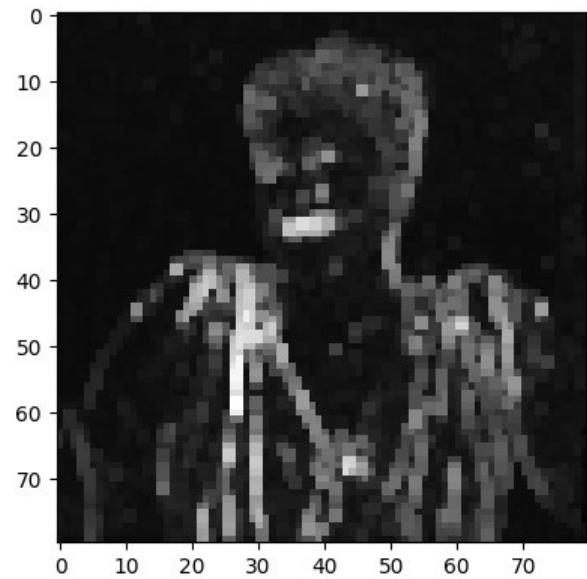
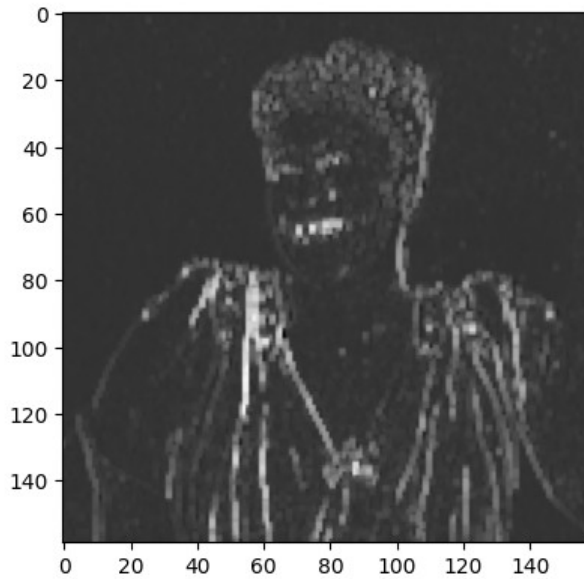
```

AX,AY = apply_two_convolutions(AX)
AX=maxPool2D(AX,(2,2))
AY=maxPool2D(AX,(2,2))
plot(AX,AY)

```

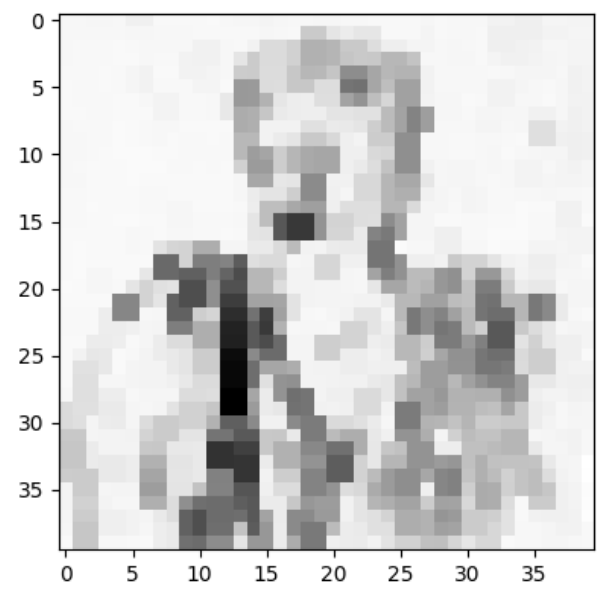
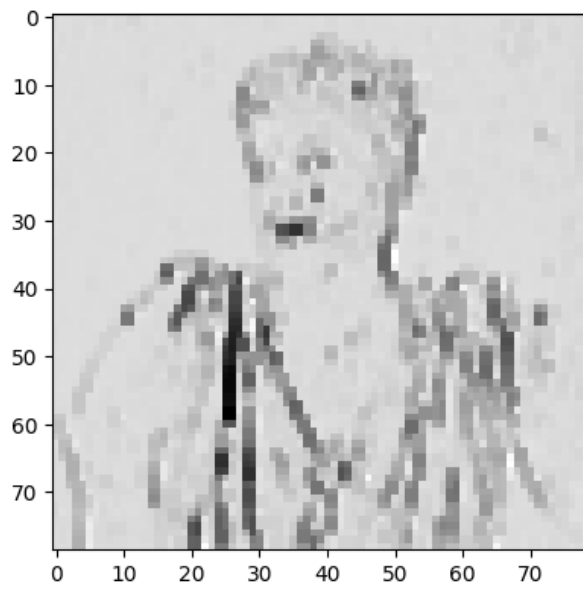
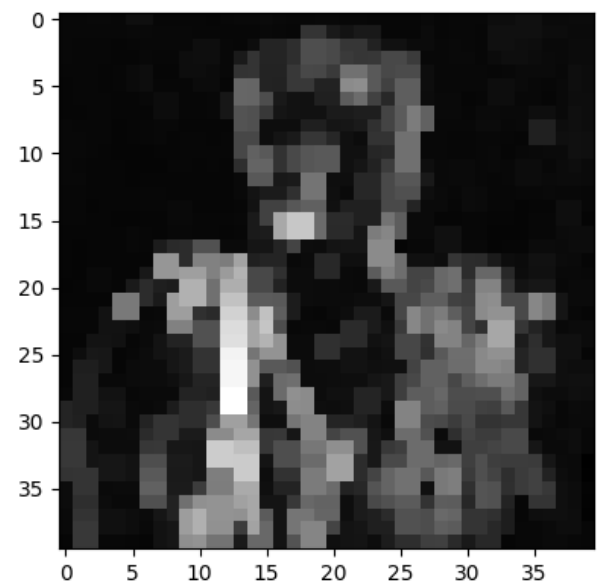
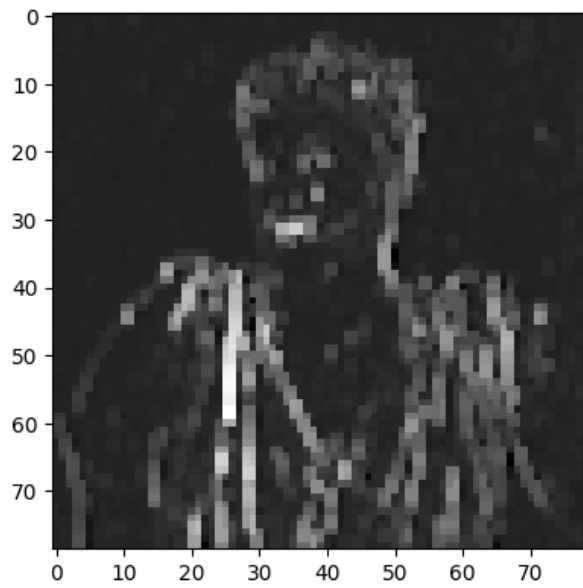
```
plot(AX,AY,cmap='gray_r')
print(f'image size:{AX.shape}')
```

image size:(159, 159)

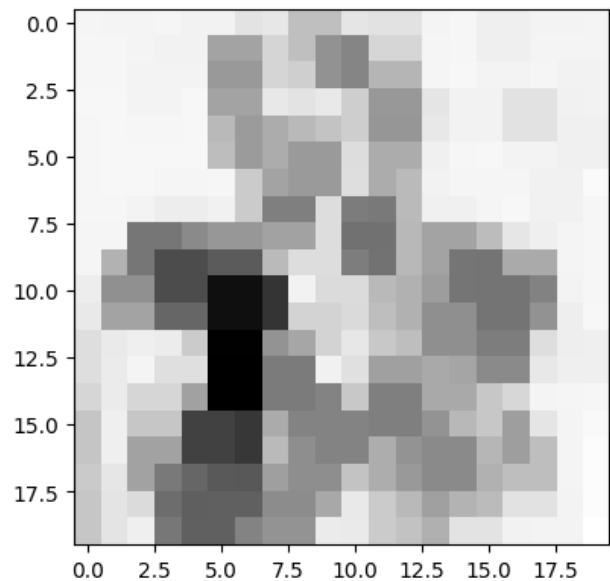
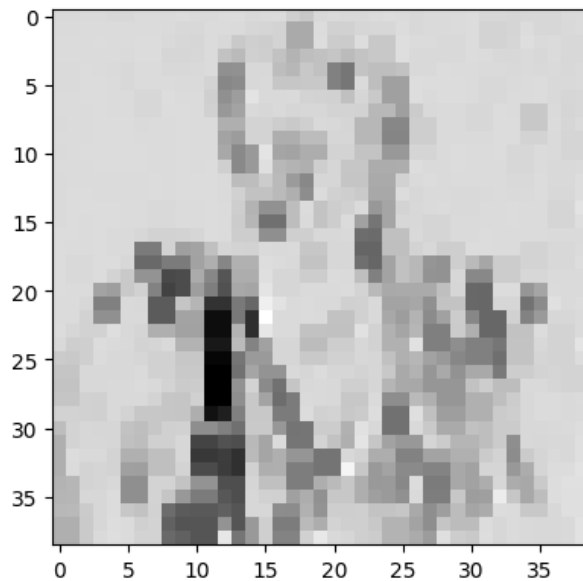
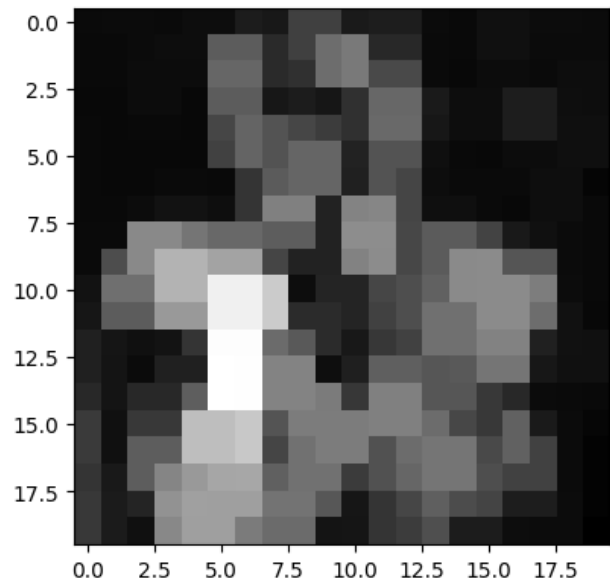
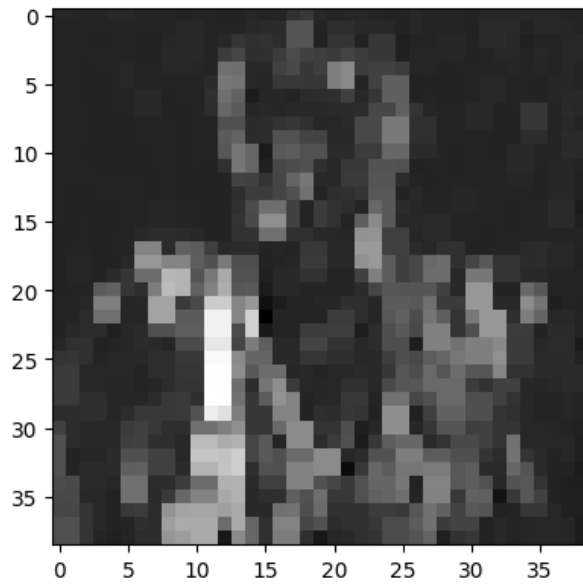


```
AX,AY = apply_two_convolution(AX)
AX=maxPool2D(AX,(2,2))
AY=maxPool2D(AX,(2,2))
plot(AX,AY)
plot(AX,AY,cmap='gray_r')
print(f'image size:{AX.shape}')
```

image size:(79, 79)



```
AX,AY = apply_two_convolutions(AX)
AX=maxPool2D(AX,(2,2))
AY=maxPool2D(AX,(2,2))
plot(AX,AY)
plot(AX,AY,cmap='gray_r')
print(f'image size:{AX.shape}')
image size:(39, 39)
```



Probably the last convolution went too far...

Then the image is flattened (converted to a vector) and submitted as input to next processing steps (e.g. classification).

```
AX_flattened = AX.flatten()
print(f'After flattening: {AX_flattened.shape}')
```

After flattening: (1521,)

2.2.2. CNN

Typical process in image recognition (old approach):

Feature extraction

1. Preprocess an image (eg. apply filters, convert to grayscale)
2. Select a convolution filter
3. Apply convolution in order to extract features, use a functions like [scipy.signal.convolve2d](#), probably written in C
4. Optionally: apply max pooling
5. Optionally: repeat steps 2, 3, 4
6. Select a classifier, e.g. SVM
7. Train the classifier using vectors of extracted features as observations
8. Test its performance
9. If not satisfied return to 1

Basic ideas of CNNs:

- Feature extraction is included into the training process: convolutional layers are parts of the architecture
- No savvy filter selection is required: filters are learned (based on the loss function that expresses classification or regression goals)
- At a given steps a number of filters can be applied simultaneously adding the corresponding number of channels at output
- Architecture may contain multiple sequences of convolution and pooling steps. It is believed that this corresponds to extracting more and more abstract features, eg:
 - First sequence: extract contours
 - Second layer: extract such objects as eyes, noses
 - Third layer: extract faces, etc

TODO Calculate the number of trainable parameters.

Provide the formula that gives the number of trainable parameters based on the values of defined variables.

$P = ???$

Fill in the table below

(img_cols, img_rows)	channels	filters	kernel size	Tensor shape after convolution	Tensor shape after pooling	number of parameters
(10,10)	1	1	(3,3)	???	???	???
(10,10)	1	5	(3,3)			
(10,10)	3	1	(5,5)			
(10,10)	3	5	(3,3)			
(16,16)	3	64	(3,3)			

Observe the following:

- None in an output shape is the dimension for multiple observations (here: images) in an input batch

- The same filter is applied to all image channels, then the bias term added.
- All filters have separate parameters
- Number of parameters for a dense layer is equal to the number of connections + one for the bias term

```
from keras import layers
from keras import models
from keras.models import Sequential
from keras.layers import Dense, Dropout, Flatten
from keras.layers import Conv2D, MaxPooling2D

#variables
img_rows=10
img_cols=10
channels=1
filters=5
kernel_size=(3,3)

model = models.Sequential()
model.add(layers.Conv2D(filters, kernel_size=kernel_size,
activation='relu', input_shape=(img_rows, img_cols,channels,)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Flatten())
model.add(layers.Dense(1))
model.summary()
```

Model: "sequential_11"

Layer (type)	Output Shape
Param #	
conv2d_11 (Conv2D)	(None, 8, 8, 5)
50	
max_pooling2d_11 (MaxPooling2D)	(None, 4, 4, 5)
0	
flatten_11 (Flatten)	(None, 80)
0	
dense_12 (Dense)	(None, 1)
81	

Total params: 131 (524.00 B)

Trainable params: 131 (524.00 B)

Non-trainable params: 0 (0.00 B)

2.2.3 Applying CNN to Fashion dataset

Create a CNN model

```
from keras import layers
from keras import models
from keras.models import Sequential
from keras.layers import Dense, Dropout, Flatten
from keras.layers import Conv2D, MaxPooling2D
import tensorflow as tf

tf.random.set_seed(42)
num_classes = int(y_train.max() + 1)

model = models.Sequential()
model.add(layers.Input(shape=(X_train.shape[1], X_train.shape[2], 1)))

model.add(layers.Conv2D(8, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
# Flatten the output to input data to the Dense layer
model.add(layers.Flatten())
# Dense layers - similar to the previous model, but using the smaller number of units
model.add(layers.Dense(32, activation='relu'))
model.add(layers.Dense(num_classes, activation='softmax'))

model.summary()

Model: "sequential_12"
```

Layer (type)	Output Shape
Param #	
conv2d_12 (Conv2D)	(None, 26, 26, 8)
80	
max_pooling2d_12 (MaxPooling2D)	(None, 13, 13, 8)
0	

0	flatten_12 (Flatten)	(None, 1352)	
43,296	dense_13 (Dense)	(None, 32)	
330	dense_14 (Dense)	(None, 10)	

Total params: 43,706 (170.73 KB)

Trainable params: 43,706 (170.73 KB)

Non-trainable params: 0 (0.00 B)

```
model.compile(optimizer=tf.keras.optimizers.RMSprop(0.02), loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

epochs = 30

batch_size = 2048

```
hist = model.fit(X_train, y_train, epochs = epochs, batch_size = batch_size, validation_data=(X_test, y_test))
```

Epoch 1/30

30/30 ————— 4s 83ms/step - accuracy: 0.3890 - loss: 465.7509 - val_accuracy: 0.6934 - val_loss: 4.8937

Epoch 2/30

30/30 ————— 2s 12ms/step - accuracy: 0.7498 - loss: 3.1780 - val_accuracy: 0.7878 - val_loss: 1.1910

Epoch 3/30

30/30 ————— 0s 10ms/step - accuracy: 0.7931 - loss: 0.9369 - val_accuracy: 0.8115 - val_loss: 0.5979

Epoch 4/30

30/30 ————— 0s 11ms/step - accuracy: 0.8052 - loss: 0.5883 - val_accuracy: 0.8190 - val_loss: 0.5517

Epoch 5/30

30/30 ————— 1s 12ms/step - accuracy: 0.8073 - loss: 0.5592 - val_accuracy: 0.7182 - val_loss: 0.8303

Epoch 6/30

30/30 ————— 1s 13ms/step - accuracy: 0.8038 - loss: 0.5499 - val_accuracy: 0.8108 - val_loss: 0.5772

Epoch 7/30

30/30 ————— 0s 12ms/step - accuracy: 0.8339 - loss: 0.4635 - val_accuracy: 0.8261 - val_loss: 0.5152

Epoch 8/30

30/30 ————— 0s 10ms/step - accuracy: 0.8386 - loss: 0.4298 - val_accuracy: 0.8361 - val_loss: 0.5326

Epoch 9/30
30/30 _____ 0s 10ms/step - accuracy: 0.8366 - loss: 0.4559 - val_accuracy: 0.8382 - val_loss: 0.5235

Epoch 10/30
30/30 _____ 0s 10ms/step - accuracy: 0.8517 - loss: 0.4031 - val_accuracy: 0.8423 - val_loss: 0.5043

Epoch 11/30
30/30 _____ 0s 10ms/step - accuracy: 0.8489 - loss: 0.4322 - val_accuracy: 0.8421 - val_loss: 0.5134

Epoch 12/30
30/30 _____ 0s 10ms/step - accuracy: 0.8599 - loss: 0.3731 - val_accuracy: 0.7713 - val_loss: 1.1520

Epoch 13/30
30/30 _____ 0s 10ms/step - accuracy: 0.8402 - loss: 0.4793 - val_accuracy: 0.8418 - val_loss: 0.5239

Epoch 14/30
30/30 _____ 0s 10ms/step - accuracy: 0.8683 - loss: 0.3518 - val_accuracy: 0.8300 - val_loss: 0.5957

Epoch 15/30
30/30 _____ 0s 10ms/step - accuracy: 0.8622 - loss: 0.3948 - val_accuracy: 0.8485 - val_loss: 0.5201

Epoch 16/30
30/30 _____ 0s 10ms/step - accuracy: 0.8762 - loss: 0.3325 - val_accuracy: 0.8155 - val_loss: 0.7245

Epoch 17/30
30/30 _____ 0s 10ms/step - accuracy: 0.8658 - loss: 0.3692 - val_accuracy: 0.8402 - val_loss: 0.5411

Epoch 18/30
30/30 _____ 0s 10ms/step - accuracy: 0.8767 - loss: 0.3312 - val_accuracy: 0.8474 - val_loss: 0.5690

Epoch 19/30
30/30 _____ 0s 11ms/step - accuracy: 0.8786 - loss: 0.3217 - val_accuracy: 0.8505 - val_loss: 0.5830

Epoch 20/30
30/30 _____ 0s 10ms/step - accuracy: 0.8816 - loss: 0.3182 - val_accuracy: 0.8210 - val_loss: 0.7148

Epoch 21/30
30/30 _____ 0s 10ms/step - accuracy: 0.8689 - loss: 0.3636 - val_accuracy: 0.8310 - val_loss: 0.6939

Epoch 22/30
30/30 _____ 0s 10ms/step - accuracy: 0.8826 - loss: 0.3182 - val_accuracy: 0.8438 - val_loss: 0.6284

Epoch 23/30
30/30 _____ 0s 10ms/step - accuracy: 0.8819 - loss: 0.3161 - val_accuracy: 0.8343 - val_loss: 0.7335

Epoch 24/30
30/30 _____ 0s 10ms/step - accuracy: 0.8815 - loss: 0.3395 - val_accuracy: 0.8223 - val_loss: 0.7205

Epoch 25/30

```

30/30 _____ 0s 10ms/step - accuracy: 0.8842 - loss:
0.3182 - val_accuracy: 0.8183 - val_loss: 0.8015
Epoch 26/30
30/30 _____ 0s 10ms/step - accuracy: 0.8805 - loss:
0.3250 - val_accuracy: 0.8258 - val_loss: 0.7445
Epoch 27/30
30/30 _____ 0s 10ms/step - accuracy: 0.8878 - loss:
0.2998 - val_accuracy: 0.8497 - val_loss: 0.7106
Epoch 28/30
30/30 _____ 0s 10ms/step - accuracy: 0.8880 - loss:
0.3017 - val_accuracy: 0.8357 - val_loss: 0.7430
Epoch 29/30
30/30 _____ 0s 10ms/step - accuracy: 0.8893 - loss:
0.2983 - val_accuracy: 0.8244 - val_loss: 0.7476
Epoch 30/30
30/30 _____ 1s 10ms/step - accuracy: 0.8854 - loss:
0.3252 - val_accuracy: 0.8427 - val_loss: 0.6606

```

Compute scores on the validation set

```

test_proba = model.predict(X_test)
y_test_pred = np.argmax(test_proba,axis=1)
313/313 _____ 1s 3ms/step

```

Display the confusion matrix

The function from the previous classes was split into two functions...

```

show_confusion_matrix(confusion_matrix(y_test,y_test_pred),LABELS)
-----
-----
NameError                                Traceback (most recent call
last)
/tmp/ipython-input-3728924533.py in <cell line: 0>()
----> 1
show_confusion_matrix(confusion_matrix(y_test,y_test_pred),LABELS)

NameError: name 'show_confusion_matrix' is not defined

probs = model.predict(X_test)
y_pred = np.argmax(probs,axis=1)
print(f'y_pred.shape={y_pred.shape}')

722/722 _____ 2s 3ms/step
y_pred.shape=(23100,)

from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))

```

	precision	recall	f1-score	support
0	0.87	0.77	0.82	2299
1	0.98	0.98	0.98	2345
2	0.86	0.77	0.82	2287
3	0.90	0.89	0.90	2342
4	0.78	0.86	0.82	2289
5	0.97	0.98	0.97	2348
6	0.67	0.77	0.72	2325
7	0.97	0.90	0.93	2287
8	0.97	0.97	0.97	2224
9	0.92	0.98	0.95	2354
accuracy			0.89	23100
macro avg	0.89	0.89	0.89	23100
weighted avg	0.89	0.89	0.89	23100

2.3 Wrap it as a function

Digression related to Python technicalities:

- it is possible to define a function taking as parameter a dictionary mapping keywords to values
- and call it specifying keywords or passing a dictionary

```
def foo(**kwargs):
    print('foo')
    for k in kwargs:
        print(k, kwargs[k])

foo(ala=0, ma=1, kota=2)

foo
ala 0
ma 1
kota 2

def boo(**kwargs):
    ala = kwargs.get('ala', 0)
    ma = kwargs.get('ma', 111)
    kota = kwargs.get('kota', 222)
    print('boo', ala, ma, kota)

boo(ala=23)

boo(**{'ala': 123, 'ma': 12})
```

```
boo 23 111 222
boo 123 12 222
```

It is up to you what to do with unknown keywords

```
def woo(**kwargs):
    keywords = ['ala', 'ma', 'kota']
    for k in kwargs:
        if k not in keywords:
            raise ValueError(f'Unknown keyword {k}')
    ala = kwargs.get('ala', 0)
    ma = kwargs.get('ma', 1)
    kota = kwargs.get('kota', 2)
    print('woo', ala, ma, kota)

woo(**{'ala': -1})
try:
    woo(**{'ala': -1, 'ola': -2})
except Exception as e:
    print(e)

woo -1 1 2
Unknown keyword ola
```

Define a function building a CNN model

TODO Define at adding **at least** two new MLP and two new CNN models

```
from keras import layers
from keras import models
from keras.models import Sequential
from keras.layers import Dense, Dropout, Flatten, BatchNormalization
from keras.layers import Conv2D, MaxPooling2D
import tensorflow as tf

def build_model(model_name, input_shape, num_classes, random_state=42):
    tf.random.set_seed(random_state)
    if model_name == 'MLP_0':
        model = models.Sequential()
        model.add(layers.Input(shape=(28, 28,)))
        model.add(layers.Flatten())
        model.add(layers.Dense(256, activation='relu'))
        model.add(layers.Dense(40, activation='relu'))
        model.add(layers.Dense(num_classes, activation='softmax'))
        return model
    elif model_name == 'MLP_1':
        model = models.Sequential()
        model.add(layers.Input(shape=input_shape))
        model.add(layers.Flatten())
```

```

        model.add(layers.Dense(512, activation='relu' ))
        model.add(layers.Dense(128, activation='relu'))
        model.add(layers.Dense(num_classes, activation='softmax'))
        return model
    elif model_name == 'MLP_2':
        model = models.Sequential()
        model.add(layers.Input(shape=input_shape))
        model.add(layers.Flatten())
        model.add(layers.Dense(256, activation='relu' ))
        model.add(layers.Dense(128, activation='relu'))
        model.add(layers.Dense(num_classes, activation='softmax'))
        return model
    elif model_name == 'CNN_0':
        model = models.Sequential()
        model.add(layers.Input(shape=input_shape))
        model.add(layers.Conv2D(8, (3, 3), activation='relu'))
        model.add(layers.MaxPooling2D((2, 2)))
        model.add(layers.Flatten())
        model.add(layers.Dense(32, activation='relu'))
        model.add(layers.Dense(num_classes, activation='softmax'))
        return model
    elif model_name == 'CNN_1':
        model = models.Sequential()
        model.add(layers.Input(shape=input_shape))
        model.add(layers.Conv2D(16, (3, 3), activation='relu'))
        model.add(layers.MaxPooling2D((2, 2)))
        model.add(layers.Flatten())
        model.add(layers.Dense(64, activation='relu'))
        model.add(layers.Dense(num_classes, activation='softmax'))
        return model
    elif model_name == 'CNN_2':
        model = models.Sequential()
        model.add(layers.Input(shape=input_shape))
        model.add(layers.Conv2D(32, (5, 5), activation='relu'))
        model.add(layers.MaxPooling2D((2, 2)))
        model.add(layers.Flatten())
        model.add(layers.Dense(128, activation='relu'))
        model.add(layers.Dense(num_classes, activation='softmax'))
        return model
    else:
        return None

```

The function that trains a model, tests it and return scores

```

from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score, confusion_matrix

def train_and_test(X_train,y_train,X_test,y_test,**kwargs):
    model_name = kwargs['model_name']

```

```

epochs = kwargs.get('epochs',20)
learning_rate = kwargs.get('learning_rate',0.01)
batch_size = kwargs.get('batch_size',1024)
random_seed = kwargs.get('random_seed',42)
returnModel = kwargs.get('returnModel',True)

input_shape = list(X_train.shape[1:])
input_shape.append(1)
input_shape=tuple(input_shape)

num_classes = np.max(y_train)+1
model =
build_model(model_name,input_shape=input_shape,num_classes=num_classes
,random_state=random_seed)
model.compile(optimizer=tf.keras.optimizers.RMSprop(learning_rate),
loss='sparse_categorical_crossentropy',metrics=['accuracy'])
model.fit(X_train,y_train,epochs=epochs,batch_size=batch_size)
preds = model.predict(X_test)
y_pred = np.argmax(preds,axis=1)
print(f'y_pred {y_pred.shape} {y_test.shape}')
results = {}
results['confusion_matrix'] = confusion_matrix(y_test,y_pred)
results['accuracy'] = accuracy_score(y_pred,y_test)
results['precision'] =
precision_score(y_pred,y_test,average='macro')
results['recall'] = recall_score(y_pred,y_test,average='macro')
results['f1'] = f1_score(y_pred,y_test,average='macro')
if returnModel:
    results['model'] = model
return results

```

Configurations

We will define a number of configurations to be tested. Each configuration defines the model name and a number of hyperparameters.

```

configurations={
    'MLP_0':
    {'model_name':'MLP_0','epochs':20,'learning_rate':0.01,'random_seed':42},
    'MLP_1':
    {'model_name':'MLP_1','epochs':20,'learning_rate':0.01,'random_seed':42},
    'MLP_1_ver2':
    {'model_name':'MLP_1','epochs':40,'learning_rate':0.01,'random_seed':42},
    'MLP_2':
    {'model_name':'MLP_2','epochs':20,'learning_rate':0.01,'random_seed':42}
}

```

```

2},
    'MLP_2_ver2':
    {'model_name': 'MLP_2', 'epochs': 40, 'learning_rate': 0.005, 'random_seed':
42},
    'CNN_0':
    {'model_name': 'CNN_0', 'epochs': 20, 'learning_rate': 0.01, 'random_seed': 4
2},
    'CNN_1':
    {'model_name': 'CNN_1', 'epochs': 20, 'learning_rate': 0.01, 'random_seed': 4
2},
    'CNN_1_ver2':
    {'model_name': 'CNN_1', 'epochs': 40, 'learning_rate': 0.005, 'random_seed':
42},
    'CNN_2':
    {'model_name': 'CNN_2', 'epochs': 20, 'learning_rate': 0.01, 'random_seed': 4
2},
    'CNN_2_ver2':
    {'model_name': 'CNN_2', 'epochs': 40, 'learning_rate': 0.01, 'random_seed': 4
2},
    'CNN_2_ver3':
    {'model_name': 'CNN_2', 'epochs': 40, 'learning_rate': 0.005, 'random_seed':
42},
}

import pandas as pd
summary = []
for cname in configurations:
    results =
train_and_test(X_train, y_train, X_test, y_test, **configurations[cname])
    print(f'--- {cname} ---')
    # show results
    show_confusion_matrix(results['confusion_matrix'])
    print(f'Accuracy:\t{results["accuracy"]:.3f}')
    print(f'Precision:\t{results["precision"]:.3f}')
    print(f'Recall:\t{results["recall"]:.3f}')
    print(f'F1:\t{results["f1"]:.3f}')

    summary.append({
        'Model': cname,
        'Epochs': configurations[cname]['epochs'],
        'Learning_rate': configurations[cname]['learning_rate'],
        'Random_seed': configurations[cname]['random_seed'],
        'Accuracy': results['accuracy'],
        'Precision': results['precision'],
        'Recall': results['recall'],
        'F1': results['f1']
    })
summary_df = pd.DataFrame(summary)
print(summary_df)

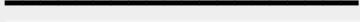
```

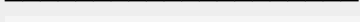
Epoch 1/20
59/59  5s 12ms/step - accuracy: 0.1048 - loss: 459.7908

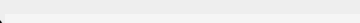
Epoch 2/20
59/59  0s 3ms/step - accuracy: 0.0957 - loss: 2.3034

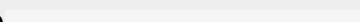
Epoch 3/20
59/59  0s 3ms/step - accuracy: 0.0991 - loss: 2.4544

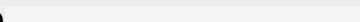
Epoch 4/20
59/59  0s 3ms/step - accuracy: 0.0977 - loss: 2.3030

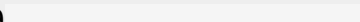
Epoch 5/20
59/59  0s 3ms/step - accuracy: 0.0992 - loss: 2.3029

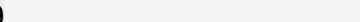
Epoch 6/20
59/59  0s 3ms/step - accuracy: 0.0992 - loss: 2.3029

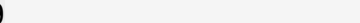
Epoch 7/20
59/59  0s 3ms/step - accuracy: 0.0992 - loss: 2.3029

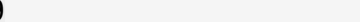
Epoch 8/20
59/59  0s 3ms/step - accuracy: 0.0992 - loss: 2.3029

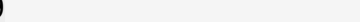
Epoch 9/20
59/59  0s 3ms/step - accuracy: 0.0992 - loss: 2.3029

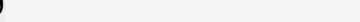
Epoch 10/20
59/59  0s 3ms/step - accuracy: 0.0992 - loss: 2.3029

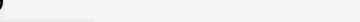
Epoch 11/20
59/59  0s 3ms/step - accuracy: 0.0992 - loss: 2.3029

Epoch 12/20
59/59  0s 3ms/step - accuracy: 0.0992 - loss: 2.3029

Epoch 13/20
59/59  0s 3ms/step - accuracy: 0.0992 - loss: 2.3029

Epoch 14/20
59/59  0s 3ms/step - accuracy: 0.0992 - loss: 2.3029

Epoch 15/20
59/59  0s 3ms/step - accuracy: 0.0992 - loss: 2.3029

Epoch 16/20
59/59  0s 3ms/step - accuracy: 0.0992 - loss: 2.3029

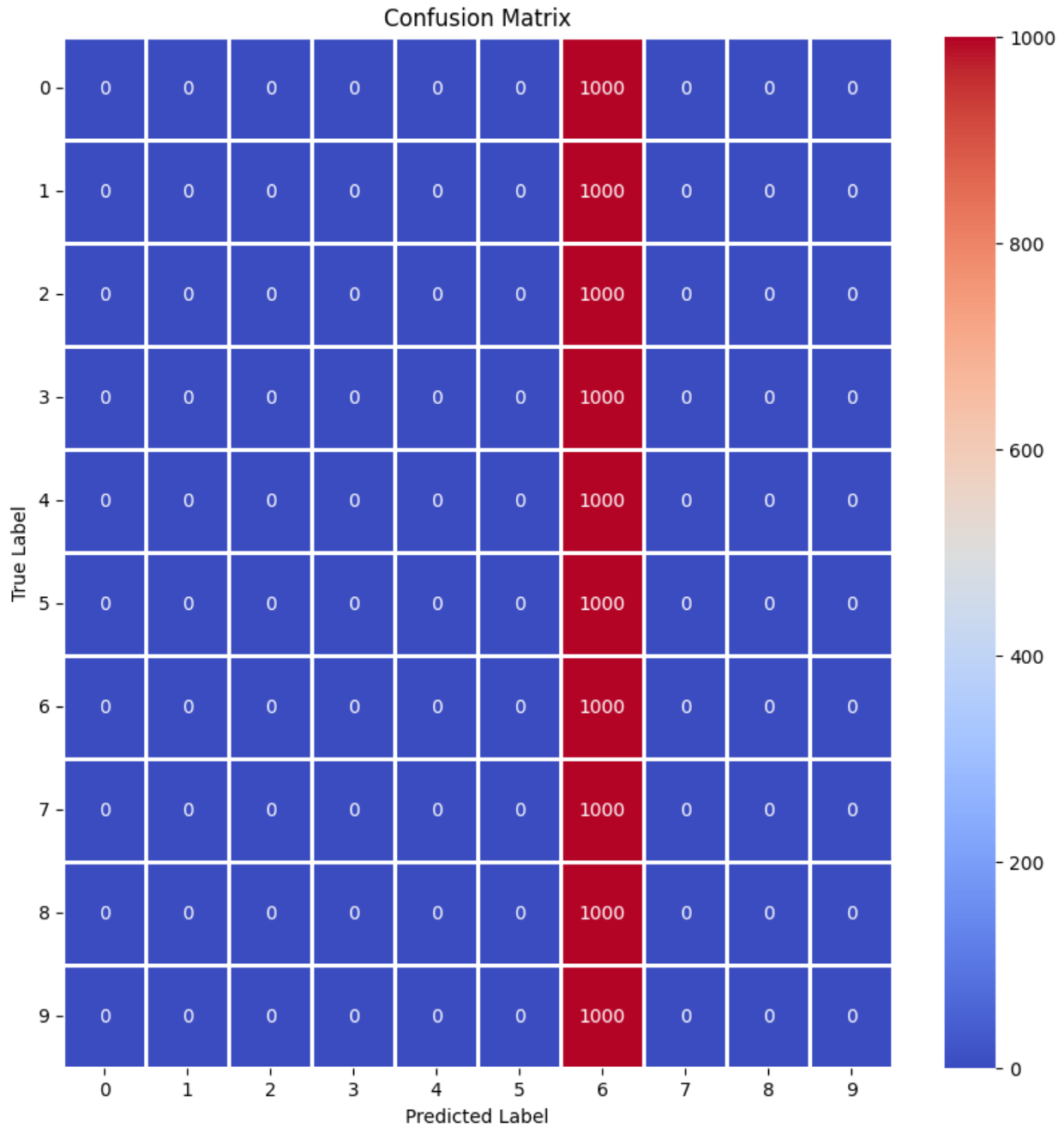
Epoch 17/20
59/59  0s 3ms/step - accuracy: 0.0992 - loss:

```

2.3029
Epoch 18/20
59/59 _____ 0s 3ms/step - accuracy: 0.0992 - loss:
2.3029
Epoch 19/20
59/59 _____ 0s 3ms/step - accuracy: 0.0992 - loss:
2.3029
Epoch 20/20
59/59 _____ 0s 3ms/step - accuracy: 0.0992 - loss:
2.3029
313/313 _____ 1s 2ms/step
y_pred (10000,) (10000,)
--- MLP_0 ---

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/
_classification.py:1565: UndefinedMetricWarning: Recall is ill-defined
and being set to 0.0 in labels with no true samples. Use
`zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))

```



Accuracy: 0.1
Precision: 0.1
Recall: 0.01
F1: 0.0182

Epoch 1/20

59/59 ————— 2s 13ms/step - accuracy: 0.4263 - loss: 2184.5347

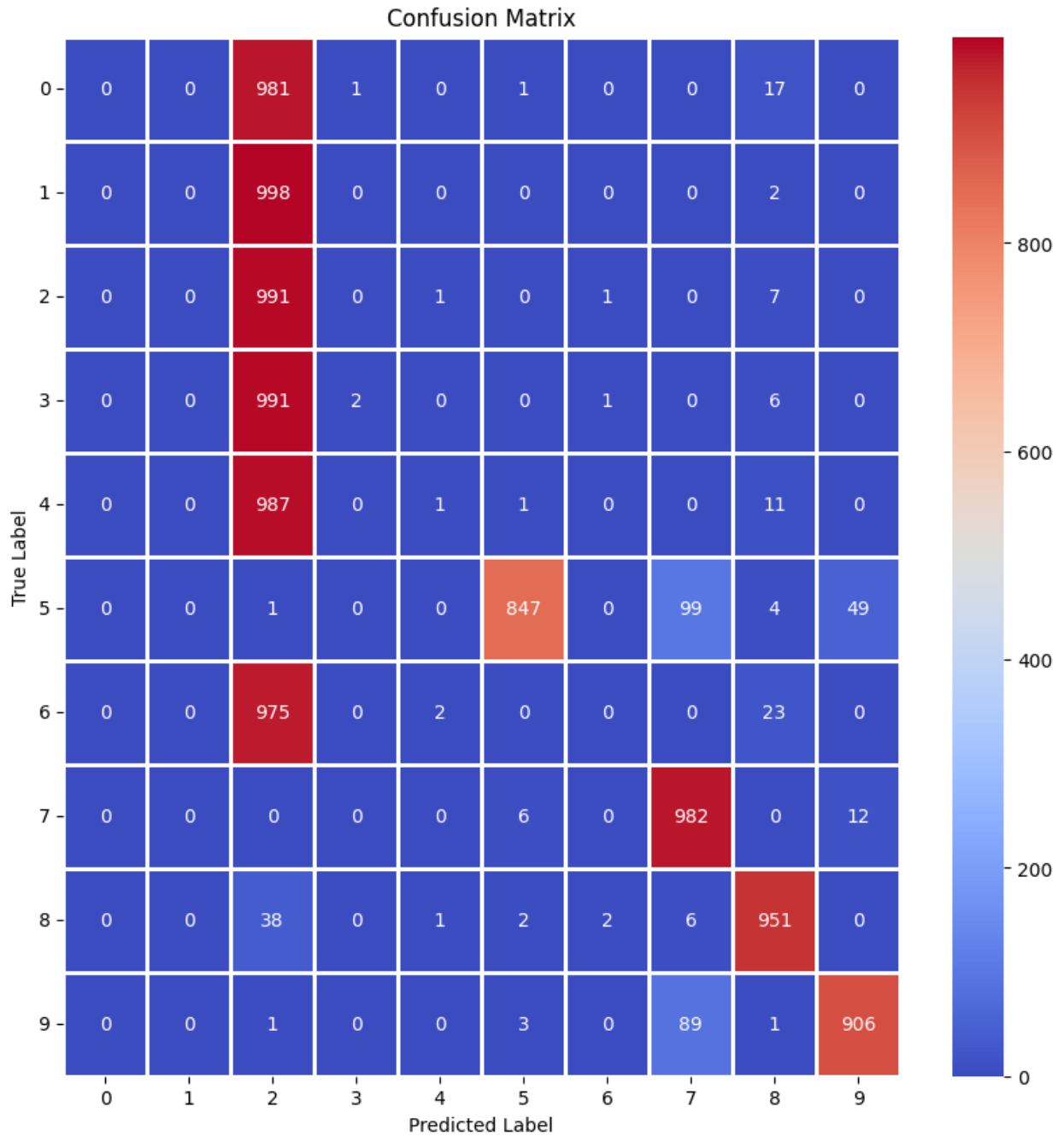
Epoch 2/20

59/59 ————— 0s 3ms/step - accuracy: 0.4016 - loss: 16.1122

Epoch 3/20
59/59 _____ 0s 4ms/step - accuracy: 0.3141 - loss: 4.3882
Epoch 4/20
59/59 _____ 0s 5ms/step - accuracy: 0.3631 - loss: 2.6972
Epoch 5/20
59/59 _____ 1s 5ms/step - accuracy: 0.3795 - loss: 4.1090
Epoch 6/20
59/59 _____ 0s 5ms/step - accuracy: 0.3947 - loss: 1.6689
Epoch 7/20
59/59 _____ 0s 5ms/step - accuracy: 0.4143 - loss: 1.6729
Epoch 8/20
59/59 _____ 0s 5ms/step - accuracy: 0.4523 - loss: 1.2958
Epoch 9/20
59/59 _____ 0s 4ms/step - accuracy: 0.4551 - loss: 1.2867
Epoch 10/20
59/59 _____ 0s 3ms/step - accuracy: 0.4588 - loss: 1.2619
Epoch 11/20
59/59 _____ 0s 3ms/step - accuracy: 0.4585 - loss: 1.2414
Epoch 12/20
59/59 _____ 0s 4ms/step - accuracy: 0.4576 - loss: 1.2771
Epoch 13/20
59/59 _____ 0s 4ms/step - accuracy: 0.4557 - loss: 1.3357
Epoch 14/20
59/59 _____ 0s 3ms/step - accuracy: 0.4130 - loss: 1.5808
Epoch 15/20
59/59 _____ 0s 4ms/step - accuracy: 0.4650 - loss: 1.2658
Epoch 16/20
59/59 _____ 0s 3ms/step - accuracy: 0.4636 - loss: 1.2870
Epoch 17/20
59/59 _____ 0s 4ms/step - accuracy: 0.4654 - loss: 1.2513
Epoch 18/20
59/59 _____ 0s 4ms/step - accuracy: 0.4579 - loss: 1.2535
Epoch 19/20

```
59/59 _____ 0s 4ms/step - accuracy: 0.4634 - loss:
1.2421
Epoch 20/20
59/59 _____ 0s 3ms/step - accuracy: 0.4716 - loss:
1.2175
313/313 _____ 1s 2ms/step
y_pred (10000,) (10000,)
--- MLP_1 ---

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/
_classification.py:1565: UndefinedMetricWarning: Recall is ill-defined
and being set to 0.0 in labels with no true samples. Use
`zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))
```



Accuracy: 0.468

Precision: 0.468

Recall: 0.472

F1: 0.397

Epoch 1/40

59/59 ————— 2s 13ms/step - accuracy: 0.3565 - loss:

2101.2820

Epoch 2/40

59/59 ————— 0s 4ms/step - accuracy: 0.2938 - loss:

```
23.8428
Epoch 3/40
59/59 _____ 0s 4ms/step - accuracy: 0.3014 - loss:
11.1686
Epoch 4/40
59/59 _____ 0s 4ms/step - accuracy: 0.3239 - loss:
2.2740
Epoch 5/40
59/59 _____ 0s 4ms/step - accuracy: 0.3112 - loss:
2.3838
Epoch 6/40
59/59 _____ 0s 4ms/step - accuracy: 0.3172 - loss:
1.9355
Epoch 7/40
59/59 _____ 0s 4ms/step - accuracy: 0.2611 - loss:
2.3671
Epoch 8/40
59/59 _____ 0s 4ms/step - accuracy: 0.2626 - loss:
2.3836
Epoch 9/40
59/59 _____ 0s 3ms/step - accuracy: 0.3107 - loss:
1.8528
Epoch 10/40
59/59 _____ 0s 4ms/step - accuracy: 0.2843 - loss:
2.0063
Epoch 11/40
59/59 _____ 0s 4ms/step - accuracy: 0.3107 - loss:
1.8473
Epoch 12/40
59/59 _____ 0s 4ms/step - accuracy: 0.3230 - loss:
1.8850
Epoch 13/40
59/59 _____ 0s 4ms/step - accuracy: 0.3561 - loss:
1.8256
Epoch 14/40
59/59 _____ 0s 4ms/step - accuracy: 0.4152 - loss:
1.5576
Epoch 15/40
59/59 _____ 0s 4ms/step - accuracy: 0.4582 - loss:
1.4435
Epoch 16/40
59/59 _____ 0s 4ms/step - accuracy: 0.4779 - loss:
1.3131
Epoch 17/40
59/59 _____ 0s 4ms/step - accuracy: 0.4943 - loss:
1.1924
Epoch 18/40
59/59 _____ 0s 5ms/step - accuracy: 0.5192 - loss:
1.2068
```

```
Epoch 19/40
59/59 _____ 1s 4ms/step - accuracy: 0.5445 - loss:
1.1327
Epoch 20/40
59/59 _____ 0s 5ms/step - accuracy: 0.5410 - loss:
1.1047
Epoch 21/40
59/59 _____ 0s 5ms/step - accuracy: 0.5426 - loss:
1.1167
Epoch 22/40
59/59 _____ 0s 5ms/step - accuracy: 0.5351 - loss:
1.2029
Epoch 23/40
59/59 _____ 0s 4ms/step - accuracy: 0.5614 - loss:
1.4419
Epoch 24/40
59/59 _____ 0s 4ms/step - accuracy: 0.5990 - loss:
1.0002
Epoch 25/40
59/59 _____ 0s 4ms/step - accuracy: 0.6222 - loss:
0.9060
Epoch 26/40
59/59 _____ 0s 4ms/step - accuracy: 0.6233 - loss:
0.9578
Epoch 27/40
59/59 _____ 0s 4ms/step - accuracy: 0.6142 - loss:
1.0089
Epoch 28/40
59/59 _____ 0s 4ms/step - accuracy: 0.6401 - loss:
0.9658
Epoch 29/40
59/59 _____ 0s 4ms/step - accuracy: 0.6439 - loss:
0.8952
Epoch 30/40
59/59 _____ 0s 4ms/step - accuracy: 0.6463 - loss:
0.8999
Epoch 31/40
59/59 _____ 0s 4ms/step - accuracy: 0.6582 - loss:
0.8899
Epoch 32/40
59/59 _____ 0s 4ms/step - accuracy: 0.6433 - loss:
0.9431
Epoch 33/40
59/59 _____ 0s 4ms/step - accuracy: 0.6360 - loss:
0.9341
Epoch 34/40
59/59 _____ 0s 4ms/step - accuracy: 0.6401 - loss:
0.8620
Epoch 35/40
```

```
59/59 _____ 0s 4ms/step - accuracy: 0.6398 - loss: 0.8997
```

```
Epoch 36/40
```

```
59/59 _____ 0s 3ms/step - accuracy: 0.6444 - loss: 0.8873
```

```
Epoch 37/40
```

```
59/59 _____ 0s 4ms/step - accuracy: 0.6544 - loss: 0.9772
```

```
Epoch 38/40
```

```
59/59 _____ 0s 4ms/step - accuracy: 0.6722 - loss: 0.8191
```

```
Epoch 39/40
```

```
59/59 _____ 0s 4ms/step - accuracy: 0.6624 - loss: 0.8428
```

```
Epoch 40/40
```

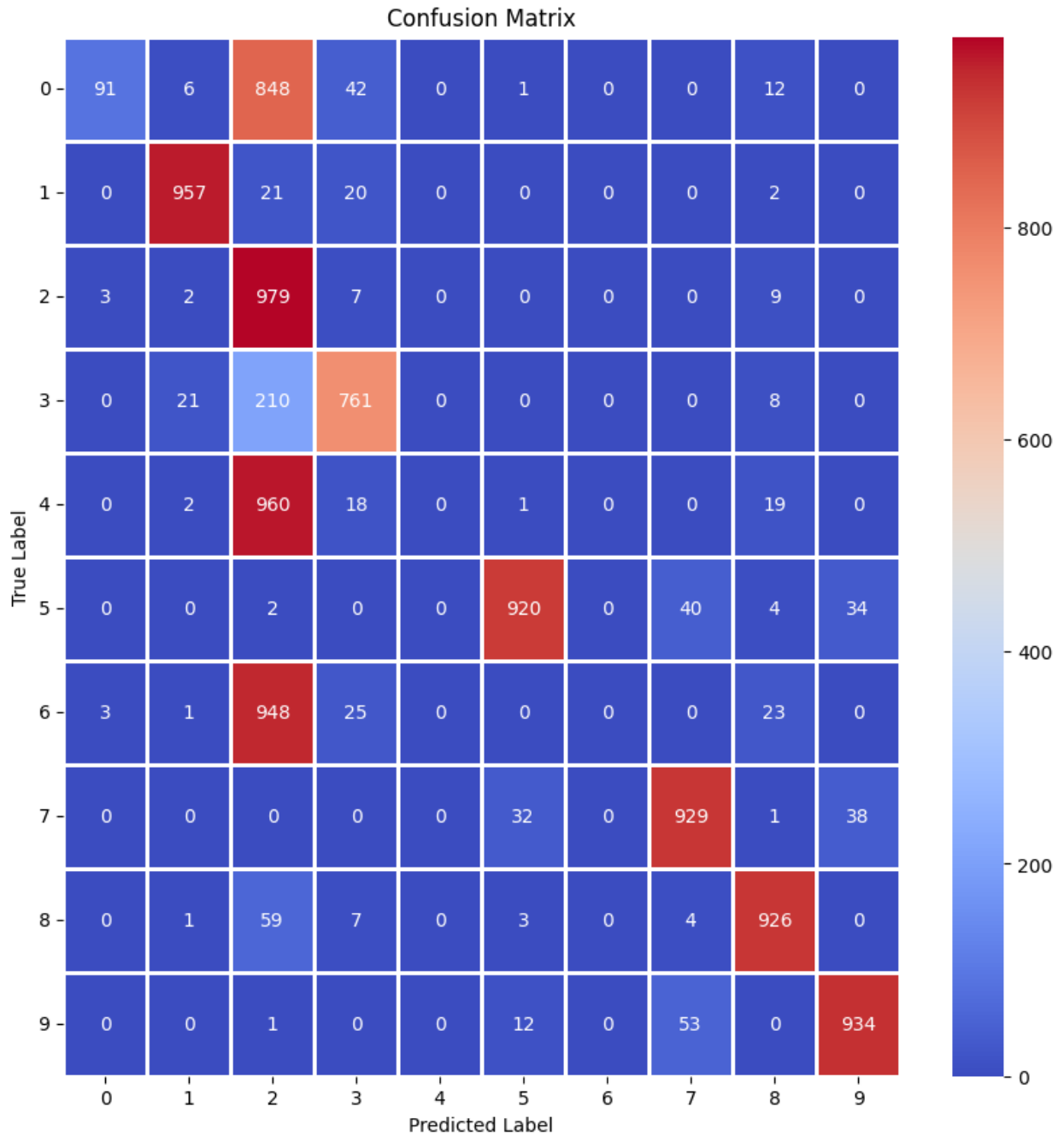
```
59/59 _____ 0s 3ms/step - accuracy: 0.6597 - loss: 0.8725
```

```
313/313 _____ 1s 2ms/step
```

```
y_pred (10000,) (10000,)
```

```
--- MLP_1_ver2 ---
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Recall is ill-defined and being set to 0.0 in labels with no true samples. Use `zero_division` parameter to control this behavior.  
  _warn_prf(average, modifier, f"{metric.capitalize()} is",  
len(result))
```



Accuracy: 0.65
Precision: 0.65
Recall: 0.672
F1: 0.603

Epoch 1/20

59/59 ————— 2s 12ms/step - accuracy: 0.3890 - loss: 1272.2474

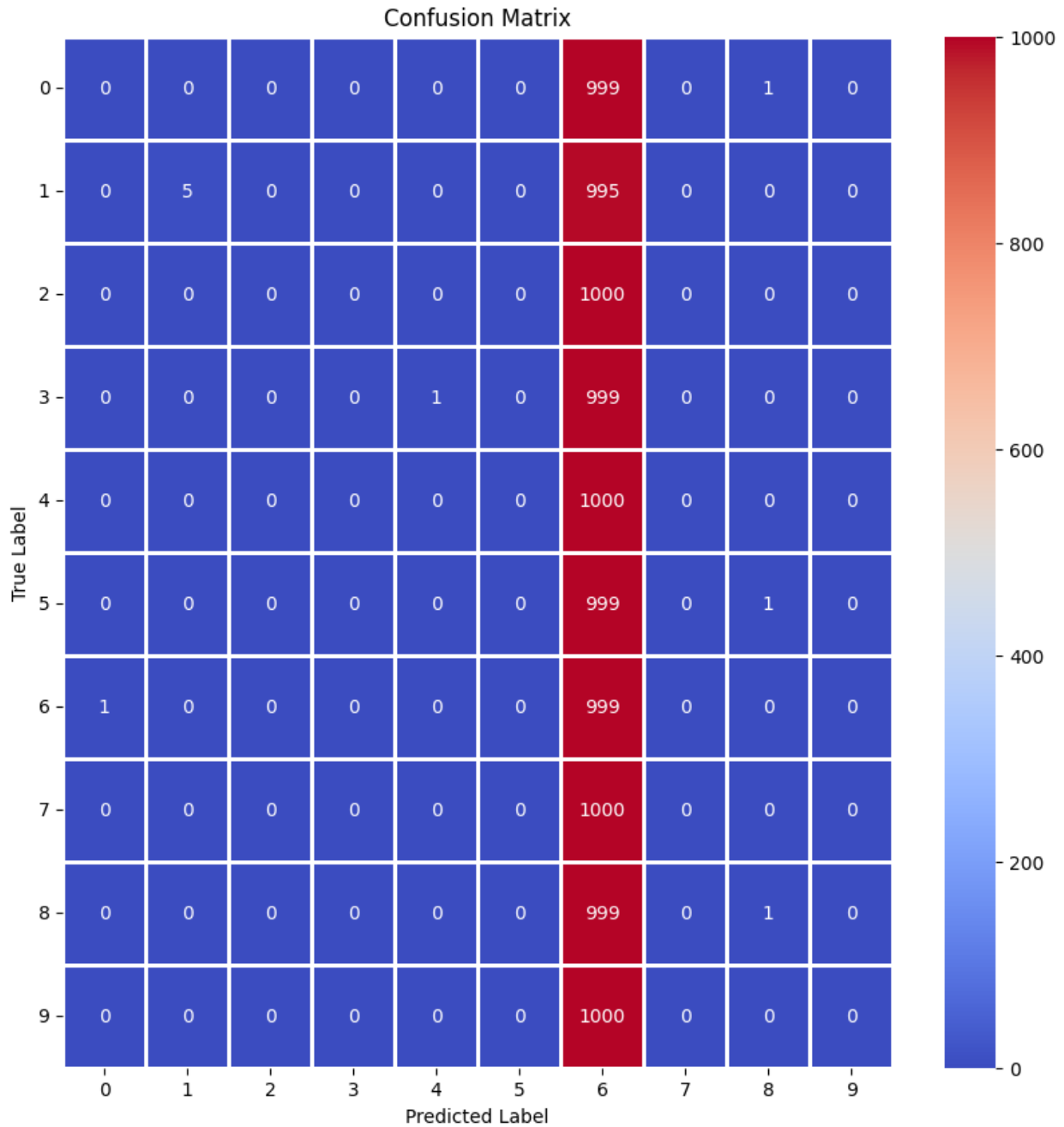
Epoch 2/20

59/59 ————— 0s 3ms/step - accuracy: 0.2129 - loss:

```
3.2109
Epoch 3/20
59/59 _____ 0s 3ms/step - accuracy: 0.1887 - loss:
3.1622
Epoch 4/20
59/59 _____ 0s 3ms/step - accuracy: 0.1810 - loss:
2.1408
Epoch 5/20
59/59 _____ 0s 3ms/step - accuracy: 0.1552 - loss:
2.1911
Epoch 6/20
59/59 _____ 0s 3ms/step - accuracy: 0.1026 - loss:
2.3574
Epoch 7/20
59/59 _____ 0s 3ms/step - accuracy: 0.1540 - loss:
2.2204
Epoch 8/20
59/59 _____ 0s 3ms/step - accuracy: 0.1716 - loss:
2.1216
Epoch 9/20
59/59 _____ 0s 3ms/step - accuracy: 0.1826 - loss:
2.0826
Epoch 10/20
59/59 _____ 0s 3ms/step - accuracy: 0.1849 - loss:
2.0855
Epoch 11/20
59/59 _____ 0s 3ms/step - accuracy: 0.1854 - loss:
2.0955
Epoch 12/20
59/59 _____ 0s 4ms/step - accuracy: 0.1898 - loss:
2.0356
Epoch 13/20
59/59 _____ 0s 4ms/step - accuracy: 0.1843 - loss:
2.0632
Epoch 14/20
59/59 _____ 0s 4ms/step - accuracy: 0.1902 - loss:
2.0939
Epoch 15/20
59/59 _____ 0s 4ms/step - accuracy: 0.1884 - loss:
2.0468
Epoch 16/20
59/59 _____ 0s 4ms/step - accuracy: 0.1876 - loss:
2.0535
Epoch 17/20
59/59 _____ 0s 5ms/step - accuracy: 0.1595 - loss:
2.1623
Epoch 18/20
59/59 _____ 0s 5ms/step - accuracy: 0.1885 - loss:
2.0713
```

```
Epoch 19/20
59/59 _____ 1s 3ms/step - accuracy: 0.1866 - loss:
2.1261
Epoch 20/20
59/59 _____ 0s 3ms/step - accuracy: 0.1912 - loss:
2.0315
313/313 _____ 1s 2ms/step
y_pred (10000,) (10000,)
--- MLP_2 ---

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/
_classification.py:1565: UndefinedMetricWarning: Recall is ill-defined
and being set to 0.0 in labels with no true samples. Use
`zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))
```



Accuracy: 0.101

Precision: 0.1

Recall: 0.143

F1: 0.0194

Epoch 1/40

59/59 ————— 2s 13ms/step - accuracy: 0.4269 - loss: 452.9422

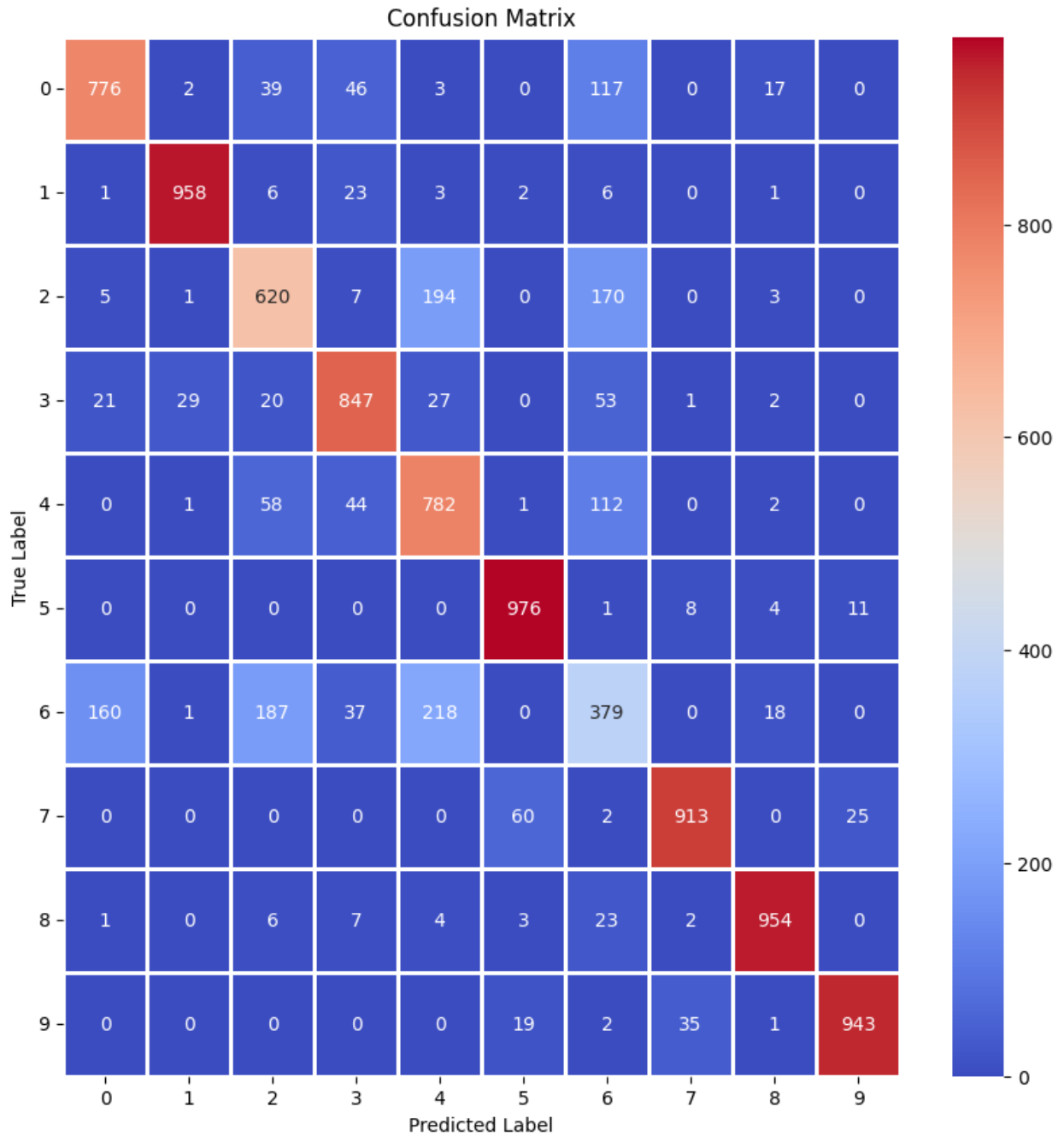
Epoch 2/40

59/59 ————— 0s 3ms/step - accuracy: 0.5290 - loss: 16.6965

```
Epoch 3/40
59/59 _____ 0s 3ms/step - accuracy: 0.5271 - loss:
12.5411
Epoch 4/40
59/59 _____ 0s 3ms/step - accuracy: 0.5404 - loss:
1.8792
Epoch 5/40
59/59 _____ 0s 3ms/step - accuracy: 0.5280 - loss:
1.4750
Epoch 6/40
59/59 _____ 0s 3ms/step - accuracy: 0.5885 - loss:
1.1613
Epoch 7/40
59/59 _____ 0s 3ms/step - accuracy: 0.5662 - loss:
1.1333
Epoch 8/40
59/59 _____ 0s 3ms/step - accuracy: 0.5680 - loss:
1.1878
Epoch 9/40
59/59 _____ 0s 3ms/step - accuracy: 0.5706 - loss:
1.0581
Epoch 10/40
59/59 _____ 0s 3ms/step - accuracy: 0.5637 - loss:
1.0822
Epoch 11/40
59/59 _____ 0s 3ms/step - accuracy: 0.5577 - loss:
1.1456
Epoch 12/40
59/59 _____ 0s 3ms/step - accuracy: 0.6147 - loss:
1.0083
Epoch 13/40
59/59 _____ 0s 3ms/step - accuracy: 0.6340 - loss:
1.0974
Epoch 14/40
59/59 _____ 0s 3ms/step - accuracy: 0.6553 - loss:
0.9406
Epoch 15/40
59/59 _____ 0s 3ms/step - accuracy: 0.6284 - loss:
0.9569
Epoch 16/40
59/59 _____ 0s 3ms/step - accuracy: 0.6662 - loss:
0.8706
Epoch 17/40
59/59 _____ 0s 3ms/step - accuracy: 0.6776 - loss:
0.8544
Epoch 18/40
59/59 _____ 0s 3ms/step - accuracy: 0.6854 - loss:
0.8316
Epoch 19/40
```

```
59/59 _____ 0s 3ms/step - accuracy: 0.6976 - loss:
0.7981
Epoch 20/40
59/59 _____ 0s 3ms/step - accuracy: 0.6912 - loss:
0.7917
Epoch 21/40
59/59 _____ 0s 3ms/step - accuracy: 0.6896 - loss:
0.7993
Epoch 22/40
59/59 _____ 0s 3ms/step - accuracy: 0.7083 - loss:
0.7723
Epoch 23/40
59/59 _____ 0s 3ms/step - accuracy: 0.7134 - loss:
0.7341
Epoch 24/40
59/59 _____ 0s 3ms/step - accuracy: 0.7213 - loss:
0.7026
Epoch 25/40
59/59 _____ 0s 3ms/step - accuracy: 0.7265 - loss:
0.6840
Epoch 26/40
59/59 _____ 0s 3ms/step - accuracy: 0.7268 - loss:
0.6894
Epoch 27/40
59/59 _____ 0s 3ms/step - accuracy: 0.7444 - loss:
0.6442
Epoch 28/40
59/59 _____ 0s 4ms/step - accuracy: 0.7458 - loss:
0.6280
Epoch 29/40
59/59 _____ 0s 5ms/step - accuracy: 0.7594 - loss:
0.5894
Epoch 30/40
59/59 _____ 0s 4ms/step - accuracy: 0.7609 - loss:
0.6049
Epoch 31/40
59/59 _____ 0s 4ms/step - accuracy: 0.7827 - loss:
0.5751
Epoch 32/40
59/59 _____ 0s 4ms/step - accuracy: 0.7896 - loss:
0.5572
Epoch 33/40
59/59 _____ 0s 4ms/step - accuracy: 0.7991 - loss:
0.5406
Epoch 34/40
59/59 _____ 0s 4ms/step - accuracy: 0.8117 - loss:
0.5115
Epoch 35/40
59/59 _____ 0s 5ms/step - accuracy: 0.8037 - loss:
```

```
0.5325
Epoch 36/40
59/59 ━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8087 - loss:
0.5150
Epoch 37/40
59/59 ━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.8022 - loss:
0.5135
Epoch 38/40
59/59 ━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.8021 - loss:
0.5143
Epoch 39/40
59/59 ━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.8073 - loss:
0.5119
Epoch 40/40
59/59 ━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.8073 - loss:
0.5055
313/313 ━━━━━━━━━ 1s 2ms/step
y_pred (10000,) (10000,)
--- MLP_2_ver2 ---
```



Accuracy: 0.815

Precision: 0.815

Recall: 0.813

F1: 0.813

Epoch 1/20

59/59 ————— 2s 19ms/step - accuracy: 0.4898 - loss:

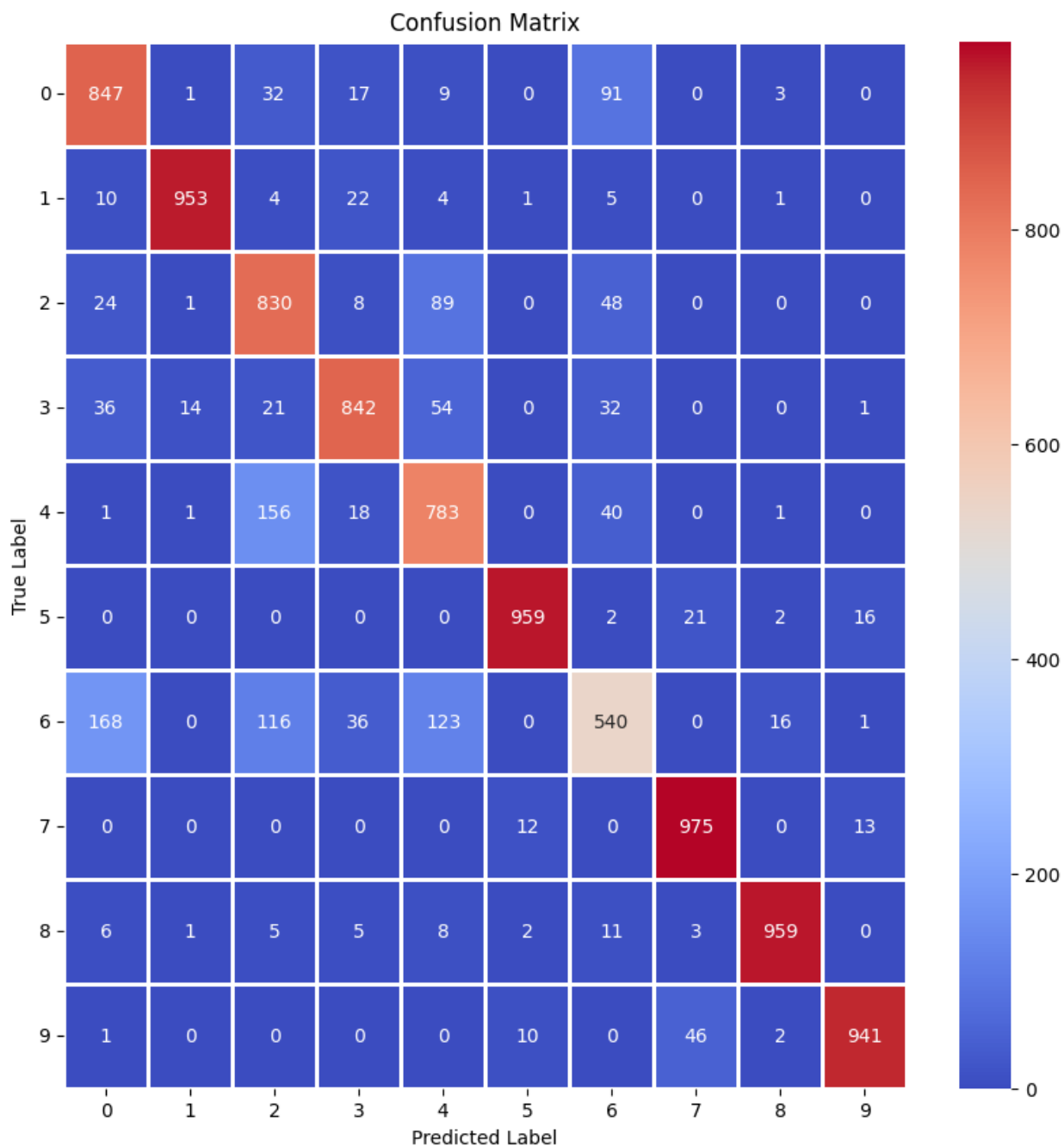
34.4710

Epoch 2/20

59/59 ————— 0s 5ms/step - accuracy: 0.8087 - loss:

```
0.5845
Epoch 3/20
59/59 _____ 0s 5ms/step - accuracy: 0.8372 - loss:
0.4538
Epoch 4/20
59/59 _____ 0s 5ms/step - accuracy: 0.8572 - loss:
0.3873
Epoch 5/20
59/59 _____ 0s 5ms/step - accuracy: 0.8633 - loss:
0.3677
Epoch 6/20
59/59 _____ 0s 5ms/step - accuracy: 0.8718 - loss:
0.3382
Epoch 7/20
59/59 _____ 0s 5ms/step - accuracy: 0.8746 - loss:
0.3351
Epoch 8/20
59/59 _____ 1s 5ms/step - accuracy: 0.8815 - loss:
0.3174
Epoch 9/20
59/59 _____ 1s 5ms/step - accuracy: 0.8834 - loss:
0.3082
Epoch 10/20
59/59 _____ 0s 5ms/step - accuracy: 0.8865 - loss:
0.2999
Epoch 11/20
59/59 _____ 0s 5ms/step - accuracy: 0.8897 - loss:
0.2988
Epoch 12/20
59/59 _____ 0s 5ms/step - accuracy: 0.8925 - loss:
0.2821
Epoch 13/20
59/59 _____ 0s 5ms/step - accuracy: 0.8939 - loss:
0.2795
Epoch 14/20
59/59 _____ 0s 6ms/step - accuracy: 0.8939 - loss:
0.2794
Epoch 15/20
59/59 _____ 0s 6ms/step - accuracy: 0.8973 - loss:
0.2701
Epoch 16/20
59/59 _____ 1s 6ms/step - accuracy: 0.9001 - loss:
0.2673
Epoch 17/20
59/59 _____ 0s 6ms/step - accuracy: 0.9013 - loss:
0.2615
Epoch 18/20
59/59 _____ 0s 6ms/step - accuracy: 0.9024 - loss:
0.2597
```

Epoch 19/20
59/59 ————— 0s 5ms/step - accuracy: 0.9038 - loss:
0.2531
Epoch 20/20
59/59 ————— 1s 5ms/step - accuracy: 0.9053 - loss:
0.2479
313/313 ————— 1s 2ms/step
y_pred (10000,) (10000,)
--- CNN_0 ---



Accuracy: 0.863
Precision: 0.863
Recall: 0.864
F1: 0.862

Epoch 1/20
59/59 ————— 2s 20ms/step - accuracy: 0.5056 - loss: 233.7140

Epoch 2/20
59/59 ————— 0s 6ms/step - accuracy: 0.8029 - loss: 0.8187

Epoch 3/20
59/59 ————— 0s 6ms/step - accuracy: 0.8376 - loss: 0.4780

Epoch 4/20
59/59 ————— 0s 6ms/step - accuracy: 0.8672 - loss: 0.3628

Epoch 5/20
59/59 ————— 0s 6ms/step - accuracy: 0.8710 - loss: 0.3527

Epoch 6/20
59/59 ————— 0s 6ms/step - accuracy: 0.8838 - loss: 0.3142

Epoch 7/20
59/59 ————— 0s 6ms/step - accuracy: 0.8877 - loss: 0.2997

Epoch 8/20
59/59 ————— 0s 6ms/step - accuracy: 0.8952 - loss: 0.2779

Epoch 9/20
59/59 ————— 0s 6ms/step - accuracy: 0.8965 - loss: 0.2795

Epoch 10/20
59/59 ————— 0s 6ms/step - accuracy: 0.9093 - loss: 0.2425

Epoch 11/20
59/59 ————— 0s 6ms/step - accuracy: 0.9109 - loss: 0.2429

Epoch 12/20
59/59 ————— 0s 6ms/step - accuracy: 0.9139 - loss: 0.2358

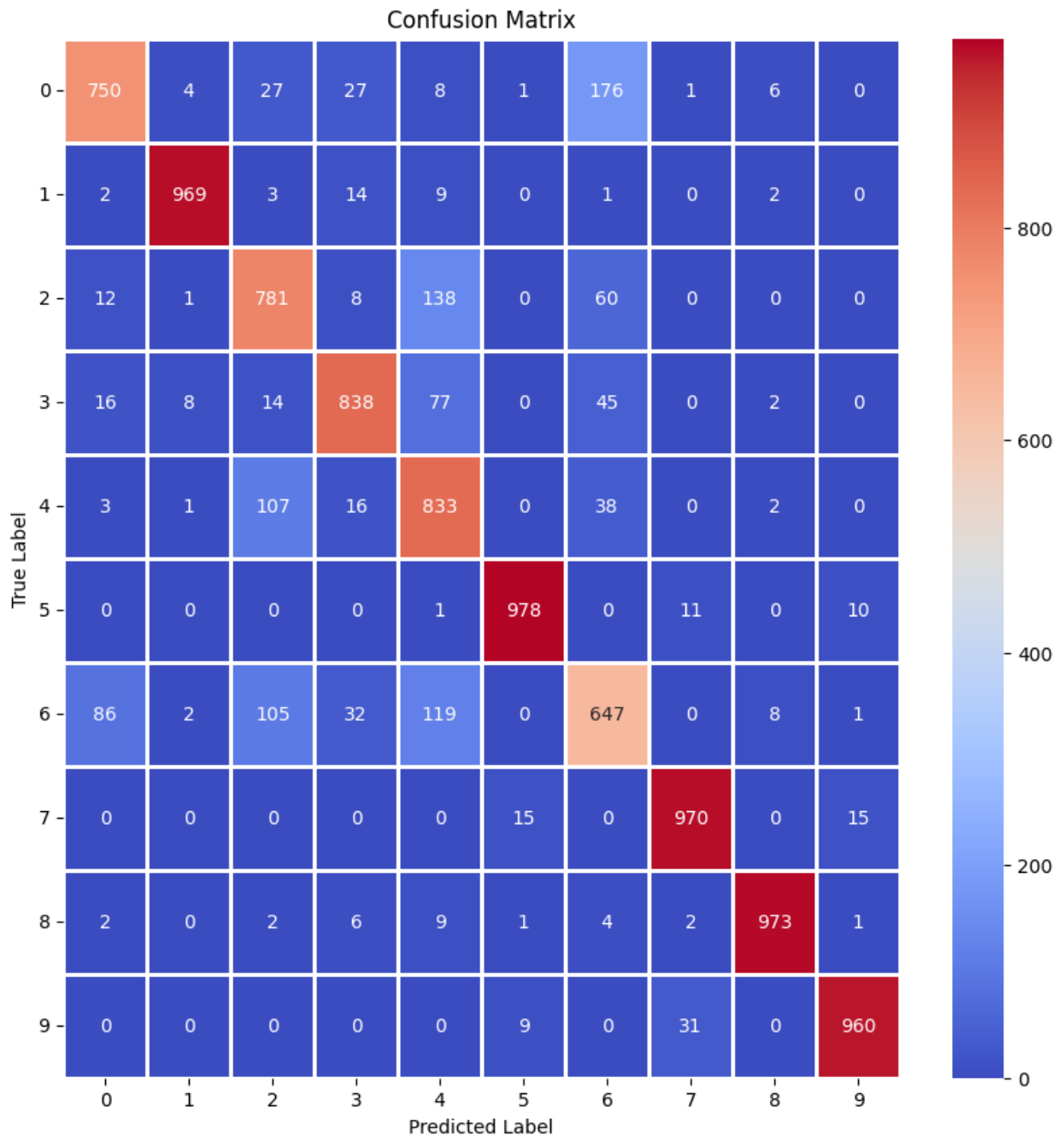
Epoch 13/20
59/59 ————— 0s 6ms/step - accuracy: 0.9163 - loss: 0.2284

Epoch 14/20
59/59 ————— 0s 7ms/step - accuracy: 0.9190 - loss: 0.2216

Epoch 15/20
59/59 ————— 0s 7ms/step - accuracy: 0.9182 - loss: 0.2212

Epoch 16/20

```
59/59 _____ 0s 7ms/step - accuracy: 0.9210 - loss:
0.2138
Epoch 17/20
59/59 _____ 0s 8ms/step - accuracy: 0.9272 - loss:
0.1968
Epoch 18/20
59/59 _____ 0s 8ms/step - accuracy: 0.9276 - loss:
0.1929
Epoch 19/20
59/59 _____ 0s 6ms/step - accuracy: 0.9267 - loss:
0.2077
Epoch 20/20
59/59 _____ 0s 6ms/step - accuracy: 0.9312 - loss:
0.1843
313/313 _____ 1s 2ms/step
y_pred (10000,) (10000,)
--- CNN_1 ---
```



Accuracy: 0.87
Precision: 0.87
Recall: 0.873
F1: 0.87

Epoch 1/40

59/59 ————— 2s 19ms/step - accuracy: 0.4648 - loss: 247.8396

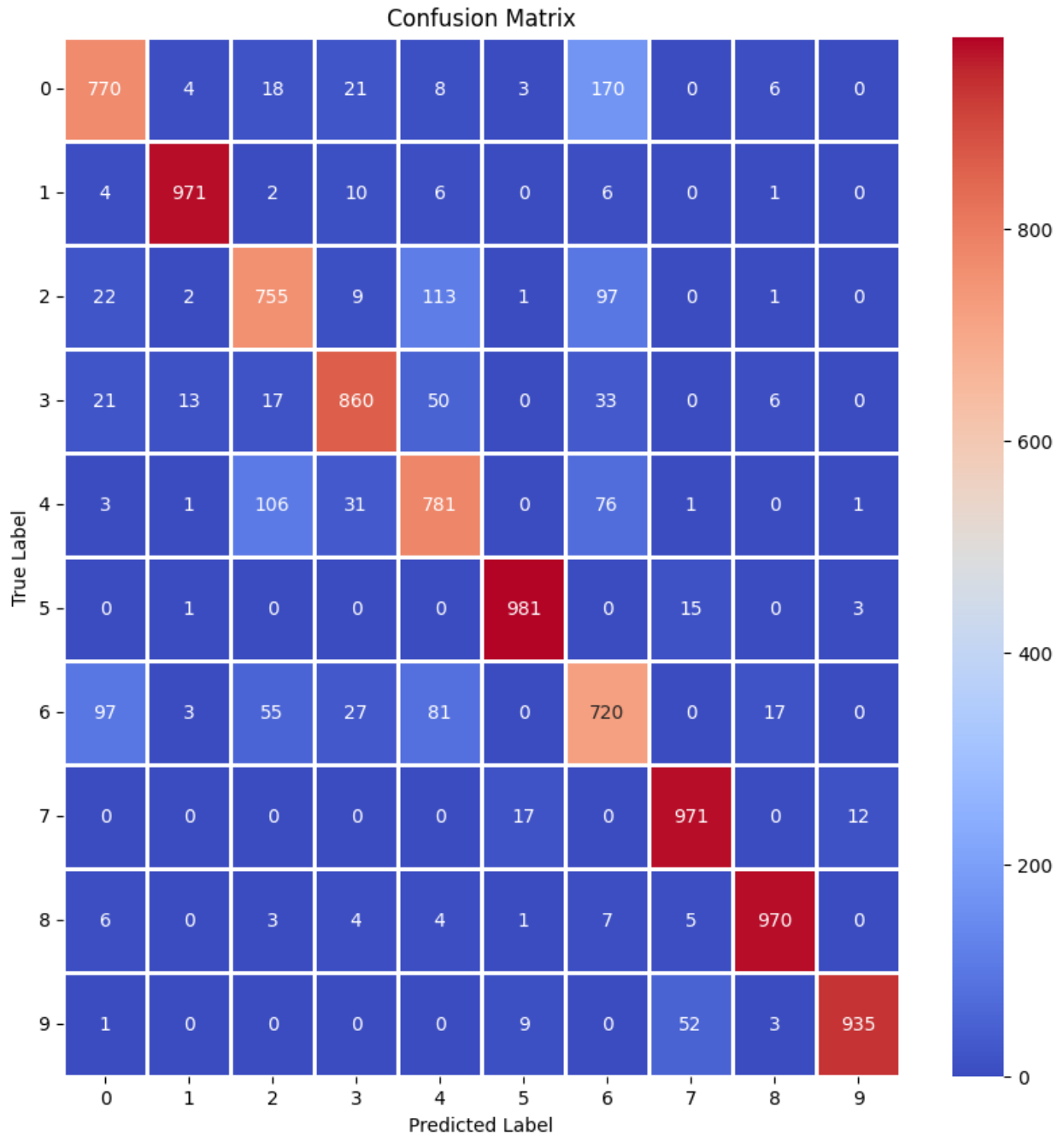
Epoch 2/40

59/59 ————— 0s 6ms/step - accuracy: 0.7984 - loss:

```
1.1778
Epoch 3/40
59/59 _____ 0s 6ms/step - accuracy: 0.8435 - loss:
0.4708
Epoch 4/40
59/59 _____ 0s 6ms/step - accuracy: 0.8742 - loss:
0.3475
Epoch 5/40
59/59 _____ 0s 6ms/step - accuracy: 0.8923 - loss:
0.2881
Epoch 6/40
59/59 _____ 0s 6ms/step - accuracy: 0.8995 - loss:
0.2600
Epoch 7/40
59/59 _____ 0s 6ms/step - accuracy: 0.9121 - loss:
0.2276
Epoch 8/40
59/59 _____ 0s 6ms/step - accuracy: 0.9201 - loss:
0.2072
Epoch 9/40
59/59 _____ 0s 6ms/step - accuracy: 0.9287 - loss:
0.1858
Epoch 10/40
59/59 _____ 0s 6ms/step - accuracy: 0.9325 - loss:
0.1757
Epoch 11/40
59/59 _____ 0s 6ms/step - accuracy: 0.9382 - loss:
0.1616
Epoch 12/40
59/59 _____ 0s 6ms/step - accuracy: 0.9409 - loss:
0.1552
Epoch 13/40
59/59 _____ 0s 6ms/step - accuracy: 0.9453 - loss:
0.1404
Epoch 14/40
59/59 _____ 0s 7ms/step - accuracy: 0.9485 - loss:
0.1341
Epoch 15/40
59/59 _____ 0s 7ms/step - accuracy: 0.9520 - loss:
0.1264
Epoch 16/40
59/59 _____ 0s 7ms/step - accuracy: 0.9547 - loss:
0.1218
Epoch 17/40
59/59 _____ 0s 7ms/step - accuracy: 0.9556 - loss:
0.1195
Epoch 18/40
59/59 _____ 0s 8ms/step - accuracy: 0.9576 - loss:
0.1144
```

```
Epoch 19/40
59/59 _____ 1s 6ms/step - accuracy: 0.9630 - loss:
0.0976
Epoch 20/40
59/59 _____ 0s 6ms/step - accuracy: 0.9591 - loss:
0.1119
Epoch 21/40
59/59 _____ 0s 6ms/step - accuracy: 0.9625 - loss:
0.0993
Epoch 22/40
59/59 _____ 0s 6ms/step - accuracy: 0.9653 - loss:
0.0980
Epoch 23/40
59/59 _____ 0s 6ms/step - accuracy: 0.9676 - loss:
0.0888
Epoch 24/40
59/59 _____ 0s 6ms/step - accuracy: 0.9677 - loss:
0.0902
Epoch 25/40
59/59 _____ 0s 6ms/step - accuracy: 0.9705 - loss:
0.0778
Epoch 26/40
59/59 _____ 0s 6ms/step - accuracy: 0.9709 - loss:
0.0798
Epoch 27/40
59/59 _____ 0s 6ms/step - accuracy: 0.9724 - loss:
0.0739
Epoch 28/40
59/59 _____ 0s 6ms/step - accuracy: 0.9711 - loss:
0.0796
Epoch 29/40
59/59 _____ 0s 6ms/step - accuracy: 0.9746 - loss:
0.0748
Epoch 30/40
59/59 _____ 0s 6ms/step - accuracy: 0.9767 - loss:
0.0668
Epoch 31/40
59/59 _____ 0s 6ms/step - accuracy: 0.9740 - loss:
0.0701
Epoch 32/40
59/59 _____ 0s 6ms/step - accuracy: 0.9744 - loss:
0.0741
Epoch 33/40
59/59 _____ 0s 6ms/step - accuracy: 0.9768 - loss:
0.0667
Epoch 34/40
59/59 _____ 0s 6ms/step - accuracy: 0.9788 - loss:
0.0608
Epoch 35/40
```

```
59/59 _____ 0s 6ms/step - accuracy: 0.9803 - loss:
0.0558
Epoch 36/40
59/59 _____ 0s 6ms/step - accuracy: 0.9787 - loss:
0.0642
Epoch 37/40
59/59 _____ 1s 6ms/step - accuracy: 0.9783 - loss:
0.0613
Epoch 38/40
59/59 _____ 0s 6ms/step - accuracy: 0.9799 - loss:
0.0557
Epoch 39/40
59/59 _____ 0s 6ms/step - accuracy: 0.9808 - loss:
0.0560
Epoch 40/40
59/59 _____ 0s 6ms/step - accuracy: 0.9792 - loss:
0.0631
313/313 _____ 1s 2ms/step
y_pred (10000,) (10000,)
--- CNN_1_ver2 ---
```



Accuracy: 0.871

Precision: 0.871

Recall: 0.874

F1: 0.872

Epoch 1/20

59/59 3s 23ms/step - accuracy: 0.5012 - loss:

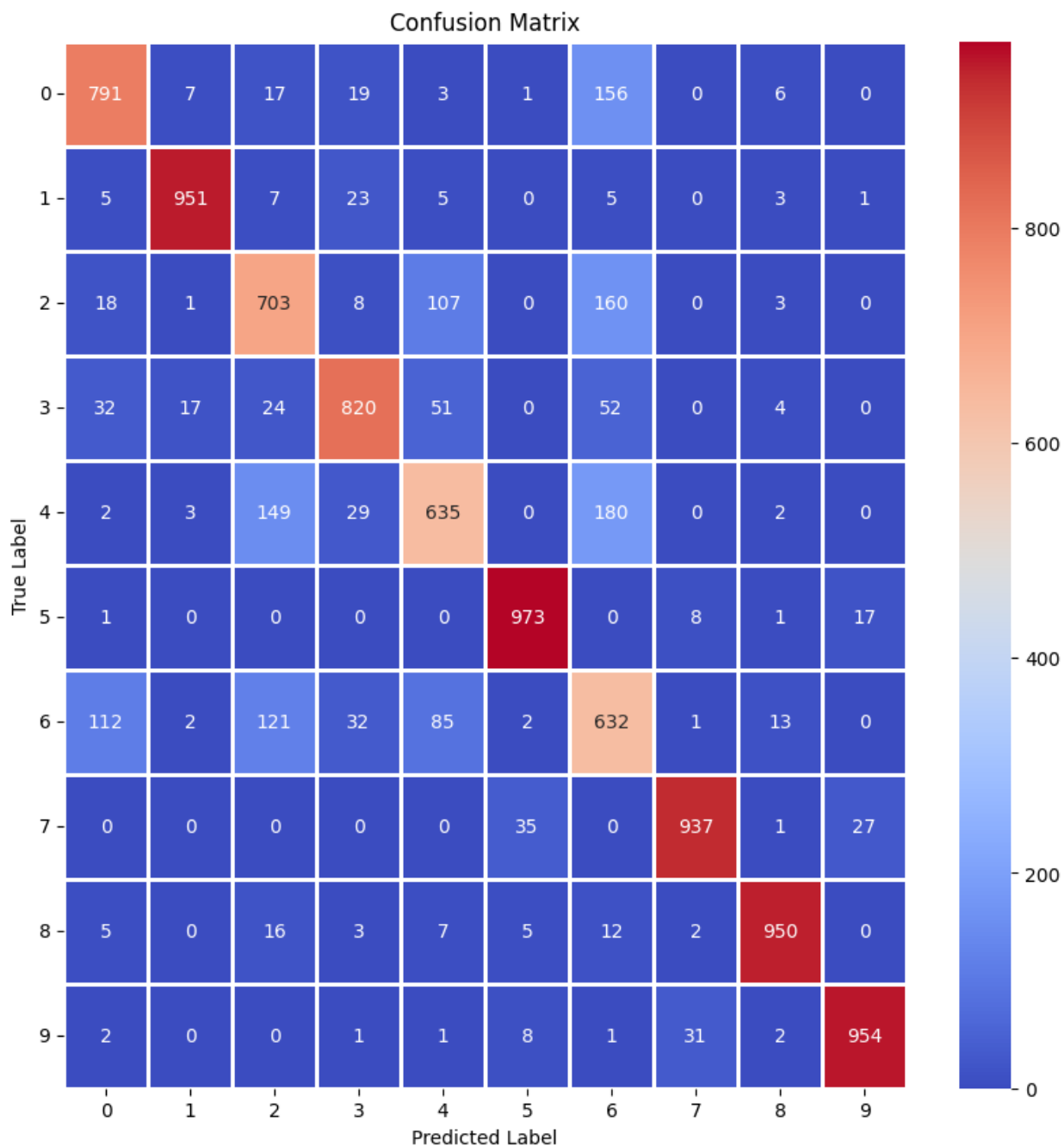
603.9633

Epoch 2/20

59/59 1s 8ms/step - accuracy: 0.7804 - loss:

```
0.7009
Epoch 3/20
59/59 _____ 1s 9ms/step - accuracy: 0.7752 - loss:
0.6950
Epoch 4/20
59/59 _____ 1s 8ms/step - accuracy: 0.8069 - loss:
0.5817
Epoch 5/20
59/59 _____ 1s 9ms/step - accuracy: 0.8255 - loss:
0.4733
Epoch 6/20
59/59 _____ 1s 8ms/step - accuracy: 0.8403 - loss:
0.4706
Epoch 7/20
59/59 _____ 1s 9ms/step - accuracy: 0.8605 - loss:
0.3764
Epoch 8/20
59/59 _____ 1s 8ms/step - accuracy: 0.8617 - loss:
0.4451
Epoch 9/20
59/59 _____ 1s 8ms/step - accuracy: 0.8769 - loss:
0.3310
Epoch 10/20
59/59 _____ 1s 9ms/step - accuracy: 0.8787 - loss:
0.3237
Epoch 11/20
59/59 _____ 1s 8ms/step - accuracy: 0.8870 - loss:
0.3240
Epoch 12/20
59/59 _____ 1s 8ms/step - accuracy: 0.8923 - loss:
0.2849
Epoch 13/20
59/59 _____ 1s 9ms/step - accuracy: 0.8921 - loss:
0.3199
Epoch 14/20
59/59 _____ 1s 8ms/step - accuracy: 0.9017 - loss:
0.2593
Epoch 15/20
59/59 _____ 1s 9ms/step - accuracy: 0.9064 - loss:
0.2615
Epoch 16/20
59/59 _____ 1s 8ms/step - accuracy: 0.9051 - loss:
0.2590
Epoch 17/20
59/59 _____ 1s 9ms/step - accuracy: 0.9119 - loss:
0.2363
Epoch 18/20
59/59 _____ 1s 10ms/step - accuracy: 0.9089 - loss:
0.2610
```

Epoch 19/20
59/59 _____ 1s 10ms/step - accuracy: 0.9167 - loss:
0.2250
Epoch 20/20
59/59 _____ 1s 10ms/step - accuracy: 0.9172 - loss:
0.2294
313/313 _____ 1s 2ms/step
y_pred (10000,) (10000,)
--- CNN_2 ---



Accuracy: 0.835
Precision: 0.835
Recall: 0.841
F1: 0.837

Epoch 1/40
59/59 ————— 3s 23ms/step - accuracy: 0.5551 - loss: 302.1621

Epoch 2/40
59/59 ————— 1s 9ms/step - accuracy: 0.7900 - loss: 0.6417

Epoch 3/40
59/59 ————— 1s 8ms/step - accuracy: 0.7813 - loss: 4.9443

Epoch 4/40
59/59 ————— 1s 9ms/step - accuracy: 0.8146 - loss: 0.7242

Epoch 5/40
59/59 ————— 1s 9ms/step - accuracy: 0.7987 - loss: 1.6219

Epoch 6/40
59/59 ————— 1s 8ms/step - accuracy: 0.8427 - loss: 0.4598

Epoch 7/40
59/59 ————— 1s 9ms/step - accuracy: 0.8482 - loss: 0.4522

Epoch 8/40
59/59 ————— 1s 8ms/step - accuracy: 0.8593 - loss: 0.4067

Epoch 9/40
59/59 ————— 1s 9ms/step - accuracy: 0.8768 - loss: 0.3433

Epoch 10/40
59/59 ————— 1s 9ms/step - accuracy: 0.8741 - loss: 0.3428

Epoch 11/40
59/59 ————— 1s 8ms/step - accuracy: 0.8769 - loss: 0.3464

Epoch 12/40
59/59 ————— 1s 10ms/step - accuracy: 0.8893 - loss: 0.2991

Epoch 13/40
59/59 ————— 1s 10ms/step - accuracy: 0.8923 - loss: 0.2952

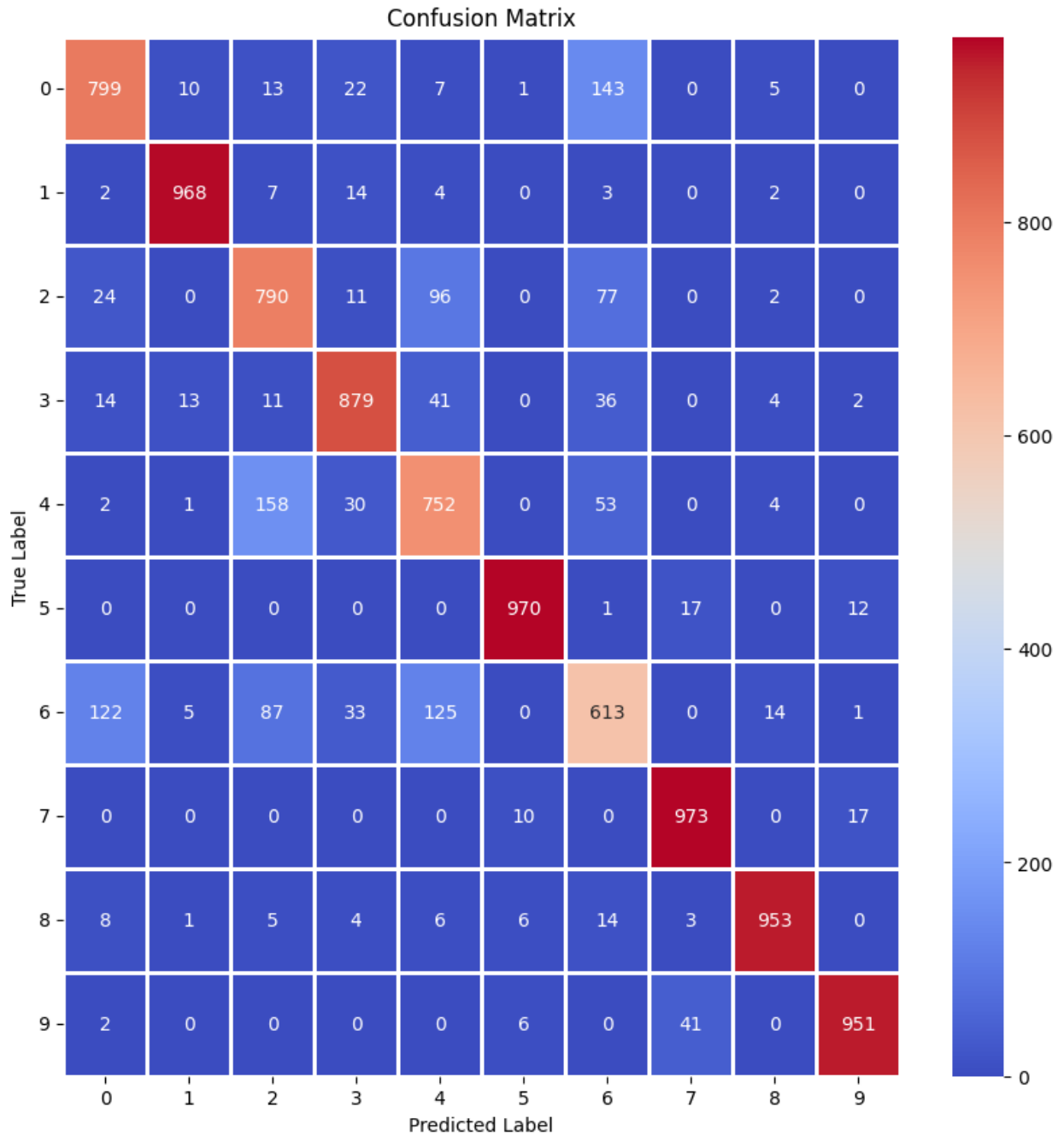
Epoch 14/40
59/59 ————— 1s 9ms/step - accuracy: 0.8924 - loss: 0.3060

Epoch 15/40
59/59 ————— 1s 9ms/step - accuracy: 0.8967 - loss: 0.3009

Epoch 16/40

```
59/59 _____ 1s 9ms/step - accuracy: 0.9029 - loss:
0.2712
Epoch 17/40
59/59 _____ 1s 9ms/step - accuracy: 0.9054 - loss:
0.2640
Epoch 18/40
59/59 _____ 1s 9ms/step - accuracy: 0.9111 - loss:
0.2542
Epoch 19/40
59/59 _____ 1s 9ms/step - accuracy: 0.9105 - loss:
0.2472
Epoch 20/40
59/59 _____ 1s 9ms/step - accuracy: 0.9151 - loss:
0.2375
Epoch 21/40
59/59 _____ 1s 9ms/step - accuracy: 0.9159 - loss:
0.2338
Epoch 22/40
59/59 _____ 1s 9ms/step - accuracy: 0.9169 - loss:
0.2410
Epoch 23/40
59/59 _____ 1s 9ms/step - accuracy: 0.9211 - loss:
0.2208
Epoch 24/40
59/59 _____ 1s 9ms/step - accuracy: 0.9241 - loss:
0.2164
Epoch 25/40
59/59 _____ 1s 9ms/step - accuracy: 0.9237 - loss:
0.2191
Epoch 26/40
59/59 _____ 1s 9ms/step - accuracy: 0.9212 - loss:
0.2366
Epoch 27/40
59/59 _____ 1s 9ms/step - accuracy: 0.9281 - loss:
0.2184
Epoch 28/40
59/59 _____ 1s 9ms/step - accuracy: 0.9312 - loss:
0.1968
Epoch 29/40
59/59 _____ 1s 9ms/step - accuracy: 0.9328 - loss:
0.1930
Epoch 30/40
59/59 _____ 1s 9ms/step - accuracy: 0.9318 - loss:
0.1989
Epoch 31/40
59/59 _____ 1s 9ms/step - accuracy: 0.9330 - loss:
0.2081
Epoch 32/40
59/59 _____ 1s 10ms/step - accuracy: 0.9315 - loss:
0.2069
```

```
Epoch 33/40
59/59 _____ 1s 10ms/step - accuracy: 0.9296 - loss:
0.2243
Epoch 34/40
59/59 _____ 1s 10ms/step - accuracy: 0.9350 - loss:
0.2003
Epoch 35/40
59/59 _____ 1s 10ms/step - accuracy: 0.9330 - loss:
0.2039
Epoch 36/40
59/59 _____ 1s 9ms/step - accuracy: 0.9386 - loss:
0.1870
Epoch 37/40
59/59 _____ 1s 9ms/step - accuracy: 0.9357 - loss:
0.2124
Epoch 38/40
59/59 _____ 1s 9ms/step - accuracy: 0.9376 - loss:
0.1916
Epoch 39/40
59/59 _____ 1s 9ms/step - accuracy: 0.9383 - loss:
0.2031
Epoch 40/40
59/59 _____ 1s 9ms/step - accuracy: 0.9404 - loss:
0.1994
313/313 _____ 1s 2ms/step
y_pred (10000,) (10000,)
--- CNN_2_ver2 ---
```



Accuracy: 0.865

Precision: 0.865

Recall: 0.865

F1: 0.865

Epoch 1/40

59/59 ————— 3s 23ms/step - accuracy: 0.4975 - loss:

186.9950

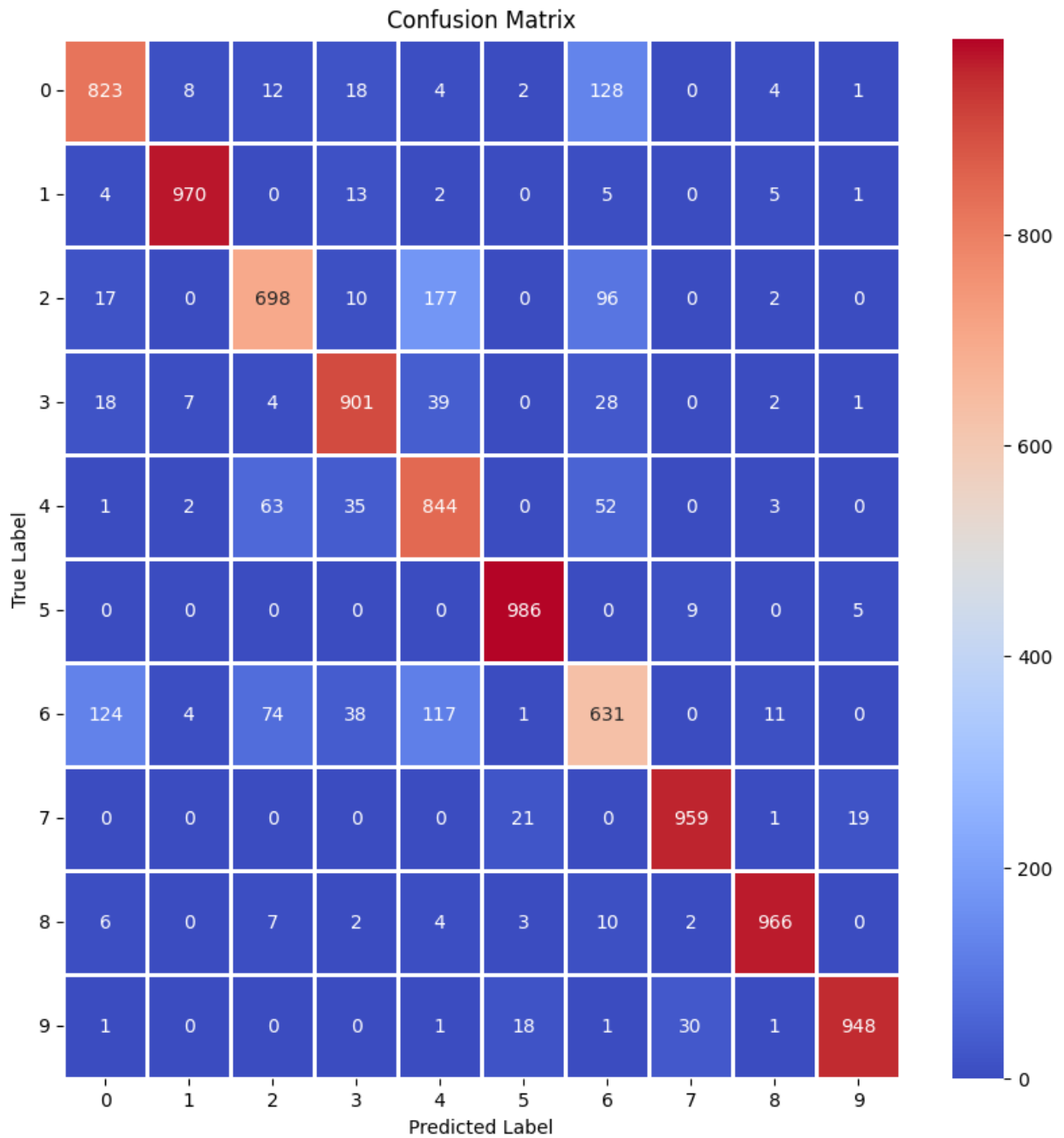
Epoch 2/40

59/59 ————— 1s 9ms/step - accuracy: 0.8139 - loss:

```
0.6020
Epoch 3/40
59/59 _____ 1s 9ms/step - accuracy: 0.8304 - loss:
0.5046
Epoch 4/40
59/59 _____ 1s 8ms/step - accuracy: 0.8562 - loss:
0.4007
Epoch 5/40
59/59 _____ 1s 9ms/step - accuracy: 0.8764 - loss:
0.3328
Epoch 6/40
59/59 _____ 1s 9ms/step - accuracy: 0.8945 - loss:
0.2826
Epoch 7/40
59/59 _____ 1s 10ms/step - accuracy: 0.9021 - loss:
0.2664
Epoch 8/40
59/59 _____ 1s 10ms/step - accuracy: 0.9124 - loss:
0.2310
Epoch 9/40
59/59 _____ 1s 9ms/step - accuracy: 0.9202 - loss:
0.2162
Epoch 10/40
59/59 _____ 1s 9ms/step - accuracy: 0.9276 - loss:
0.1910
Epoch 11/40
59/59 _____ 1s 9ms/step - accuracy: 0.9331 - loss:
0.1837
Epoch 12/40
59/59 _____ 1s 9ms/step - accuracy: 0.9387 - loss:
0.1649
Epoch 13/40
59/59 _____ 1s 9ms/step - accuracy: 0.9440 - loss:
0.1529
Epoch 14/40
59/59 _____ 1s 9ms/step - accuracy: 0.9485 - loss:
0.1412
Epoch 15/40
59/59 _____ 1s 9ms/step - accuracy: 0.9537 - loss:
0.1263
Epoch 16/40
59/59 _____ 1s 9ms/step - accuracy: 0.9557 - loss:
0.1257
Epoch 17/40
59/59 _____ 1s 9ms/step - accuracy: 0.9577 - loss:
0.1289
Epoch 18/40
59/59 _____ 1s 9ms/step - accuracy: 0.9554 - loss:
0.1291
```

```
Epoch 19/40
59/59 _____ 1s 9ms/step - accuracy: 0.9623 - loss:
0.1066
Epoch 20/40
59/59 _____ 1s 9ms/step - accuracy: 0.9669 - loss:
0.0940
Epoch 21/40
59/59 _____ 1s 9ms/step - accuracy: 0.9660 - loss:
0.1017
Epoch 22/40
59/59 _____ 1s 9ms/step - accuracy: 0.9682 - loss:
0.0950
Epoch 23/40
59/59 _____ 1s 9ms/step - accuracy: 0.9693 - loss:
0.0897
Epoch 24/40
59/59 _____ 1s 9ms/step - accuracy: 0.9710 - loss:
0.0878
Epoch 25/40
59/59 _____ 1s 9ms/step - accuracy: 0.9709 - loss:
0.0890
Epoch 26/40
59/59 _____ 1s 9ms/step - accuracy: 0.9713 - loss:
0.0884
Epoch 27/40
59/59 _____ 1s 10ms/step - accuracy: 0.9733 - loss:
0.0826
Epoch 28/40
59/59 _____ 1s 10ms/step - accuracy: 0.9734 - loss:
0.0894
Epoch 29/40
59/59 _____ 1s 9ms/step - accuracy: 0.9737 - loss:
0.0807
Epoch 30/40
59/59 _____ 1s 9ms/step - accuracy: 0.9772 - loss:
0.0713
Epoch 31/40
59/59 _____ 1s 9ms/step - accuracy: 0.9750 - loss:
0.0811
Epoch 32/40
59/59 _____ 1s 9ms/step - accuracy: 0.9770 - loss:
0.0681
Epoch 33/40
59/59 _____ 1s 9ms/step - accuracy: 0.9760 - loss:
0.0785
Epoch 34/40
59/59 _____ 1s 9ms/step - accuracy: 0.9764 - loss:
0.0906
Epoch 35/40
```

```
59/59 _____ 1s 9ms/step - accuracy: 0.9795 - loss:
0.0675
Epoch 36/40
59/59 _____ 1s 9ms/step - accuracy: 0.9811 - loss:
0.0595
Epoch 37/40
59/59 _____ 1s 9ms/step - accuracy: 0.9808 - loss:
0.0618
Epoch 38/40
59/59 _____ 1s 9ms/step - accuracy: 0.9815 - loss:
0.0646
Epoch 39/40
59/59 _____ 1s 9ms/step - accuracy: 0.9797 - loss:
0.0697
Epoch 40/40
59/59 _____ 1s 9ms/step - accuracy: 0.9810 - loss:
0.0641
313/313 _____ 1s 2ms/step
y_pred (10000,) (10000,)
--- CNN_2_ver3 ---
```



Accuracy: 0.873

Precision: 0.873

Recall: 0.874

F1: 0.872

	Model	Epochs	Learning_rate	Random_seed	Accuracy
Precision \					
0	MLP_0	20	0.010	42	0.1000
0.1000					
1	MLP_1	20	0.010	42	0.4680

```

0.4680
2  MLP_1_ver2      40      0.010      42      0.6497
0.6497
3      MLP_2      20      0.010      42      0.1005
0.1005
4  MLP_2_ver2      40      0.005      42      0.8148
0.8148
5      CNN_0      20      0.010      42      0.8629
0.8629
6      CNN_1      20      0.010      42      0.8699
0.8699
7  CNN_1_ver2      40      0.005      42      0.8714
0.8714
8      CNN_2      20      0.010      42      0.8346
0.8346
9  CNN_2_ver2      40      0.010      42      0.8648
0.8648
10 CNN_2_ver3      40      0.005      42      0.8726
0.8726

```

```

      Recall      F1
0  0.010000  0.018182
1  0.472022  0.396580
2  0.671826  0.603363
3  0.143333  0.019375
4  0.813153  0.812767
5  0.864193  0.861637
6  0.873331  0.870496
7  0.873972  0.872163
8  0.840604  0.836743
9  0.864924  0.864637
10 0.873870  0.872131

```

TODO Prepare (**at least 8 new**) configurations using **at least** two new MLP and two new CNN models and perform tests. Try to outperform the results obtained with the sample code.

Present the results in the form of a table. A table can be entered as a markdown: [see the table generator](#). As an alternative, you may load data to Pandas DataFrame and display using `df.head()`.

Configuration	Model name	Hiperparameters	Accuracy	Precision	Recall	F1
MLP_0	MLP_0	epochs=20 learning_rate=0.01 itd	0.67	0.78	0.65	0.72

Second
Configuration

Hiperparameters can be placed in seperate columns...

2.4 Applying convolution to multivariate time series

2.4.1 Data generation

We will first generate multivariate time series data.

Assume that the city is divided into 5 districts and its inhabitants move between the districts during the day. One of the districts (number 2) is marked as the city center, the other are outskirts.

- In the morning people tend to move to the center and in the evening they return back.
- Movements between outskirts is also possible
- During night hours the activity diminishes

The generated time series show hourly counts of people in each district.

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

# --- parameters ---
n_agents = 1000
n_locations = 5
center = 2
steps_per_day = 24
n_days = 30
T = steps_per_day * n_days

# --- parameters controlling moves---
base_move_prob = 0.05
bias_to_center = 0.25
bias_to_home = 0.25
night_suppression = 0.2
random_variation = 0.05

# --- initiation of agents ---
home = np.random.choice(n_locations, n_agents)
location = home.copy()
x = np.zeros((T, n_locations))

def hour_phase(hour):
    if 7 <= hour < 9:
        return 'morning'
    elif 9 <= hour < 17:
        return 'day'
    elif 17 <= hour < 19:
```

```

        return 'evening'
    else:
        return 'night'

# --- main sim loop ---
for t in range(T):
    hour = t % steps_per_day
    phase = hour_phase(hour)

    # basic transition matrix
    P = np.ones((n_locations, n_locations)) * base_move_prob
    np.fill_diagonal(P, 1 - base_move_prob * (n_locations - 1))

    if phase == 'morning':
        P[:, center] += bias_to_center
    elif phase == 'evening':
        for i in range(n_agents):
            P[location[i], home[i]] += bias_to_home
    elif phase == 'night':
        P = 1 - night_suppression + night_suppression * P

    # random perturbations
    P += random_variation * (np.random.rand(*P.shape) - 0.5)
    P = np.clip(P, 0, None)
    P /= P.sum(axis=1, keepdims=True)

    # updating agents locations
    new_locations = []
    for i in range(n_agents):
        loc = location[i]
        probs = P[loc]
        new_loc = np.random.choice(n_locations, p=probs)
        new_locations.append(new_loc)
    location = np.array(new_locations)

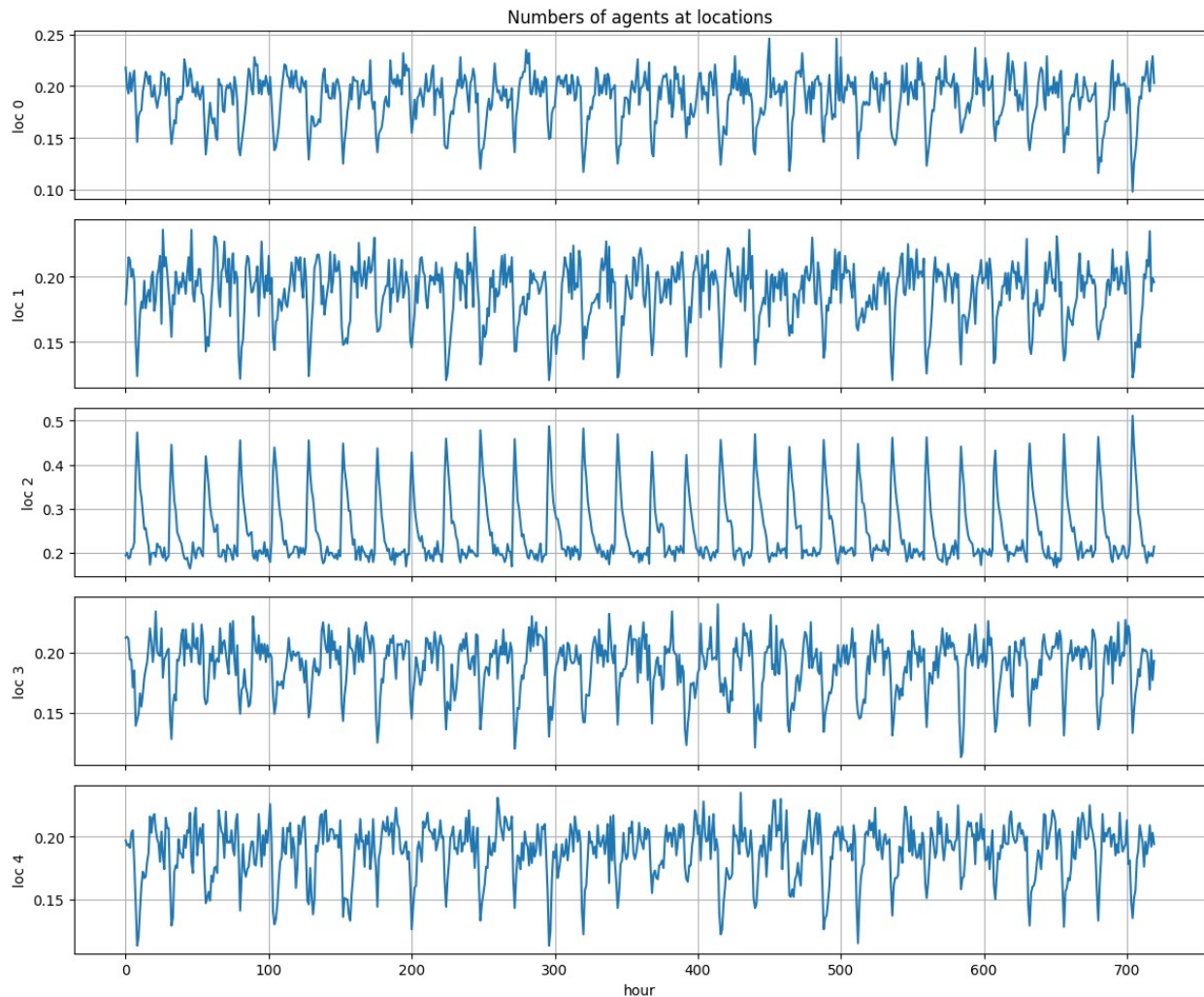
    # saving states (with normalization)
    for j in range(n_locations):
        x[t, j] = np.sum(location == j) / n_agents

# --- plots ---
fig, axes = plt.subplots(n_locations, 1, figsize=(12, 2*n_locations),
sharex=True)

for j in range(n_locations):
    axes[j].plot(x[:, j], color='tab:blue')
    axes[j].set_ylabel(f"loc {j}")
    axes[j].grid(True)
    if j == 0:
        axes[j].set_title("Numbers of agents at locations")

```

```
axes[-1].set_xlabel("hour")
plt.tight_layout()
plt.show()
```



```
import pandas as pd

# assume, x has the shape (T, n_locations)
df = pd.DataFrame(x, columns=[f"loc_{j}" for j in range(n_locations)])
df.to_csv("population_timeseries.csv", index_label="t")
df.head()

{"summary":{"\n  \"name\": \"df\", \n  \"rows\": 720, \n  \"fields\": [\n    {\n      \"column\": \"loc_0\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0.02354707023580318, \n        \"min\": 0.098, \n        \"max\": 0.246, \n        \"num_unique_values\": 114, \n        \"samples\": [\n          0.18, \n          0.195, \n          0.168\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }\n    }, \n    {\n      \"column\": \"loc_1\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0.02354707023580318, \n        \"min\": 0.098, \n        \"max\": 0.246, \n        \"num_unique_values\": 114, \n        \"samples\": [\n          0.18, \n          0.195, \n          0.168\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }\n    }, \n    {\n      \"column\": \"loc_2\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0.02354707023580318, \n        \"min\": 0.098, \n        \"max\": 0.246, \n        \"num_unique_values\": 114, \n        \"samples\": [\n          0.18, \n          0.195, \n          0.168\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }\n    }, \n    {\n      \"column\": \"loc_3\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0.02354707023580318, \n        \"min\": 0.098, \n        \"max\": 0.246, \n        \"num_unique_values\": 114, \n        \"samples\": [\n          0.18, \n          0.195, \n          0.168\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }\n    }, \n    {\n      \"column\": \"loc_4\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0.02354707023580318, \n        \"min\": 0.098, \n        \"max\": 0.246, \n        \"num_unique_values\": 114, \n        \"samples\": [\n          0.18, \n          0.195, \n          0.168\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }\n    }\n  ]\n}}
```

```

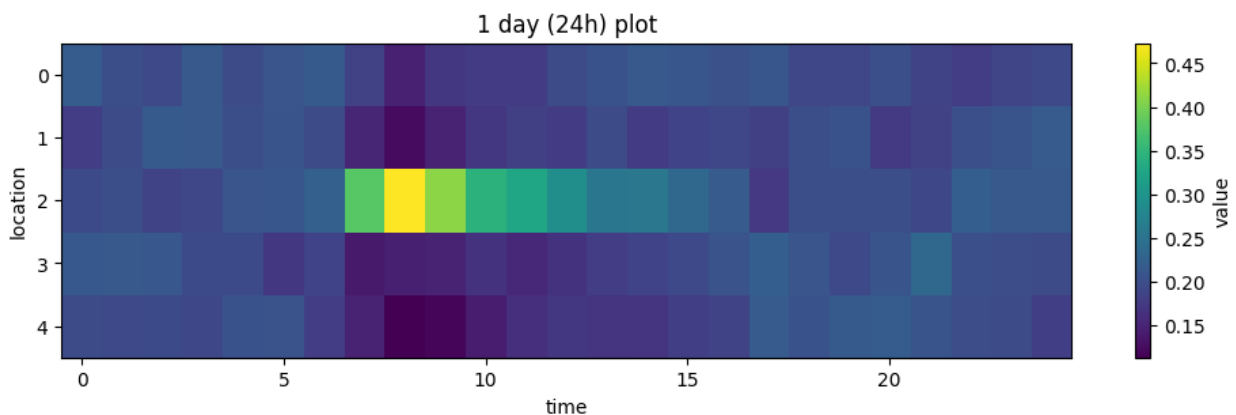
{"column\\": "\\loc_1\\",\\n      \\\"properties\\\": {\\n          \\\"dtype\\\":
\\\"number\\\",\\n          \\\"std\\\": 0.02241805658243809,\\n          \\\"min\\\":
0.121,\\n          \\\"max\\\": 0.238,\\n          \\\"num_unique_values\\\": 109,\\n
\\\"samples\\\": [\\n          0.238,\\n          0.172,\\n          0.2\\n
],\\n          \\\"semantic_type\\\": \\\"\\\",\\n          \\\"description\\\": \\\"\\\"\\n
}\\n      },\\n      {\\n          \\\"column\\\": \\\"loc_2\\\",\\n          \\\"properties\\\":
{\\n          \\\"dtype\\\": \\\"number\\\",\\n          \\\"std\\\":
0.07401748087369608,\\n          \\\"min\\\": 0.164,\\n          \\\"max\\\":
0.511,\\n          \\\"num_unique_values\\\": 219,\\n          \\\"samples\\\": [\\n
0.299,\\n          0.248,\\n          0.442\\n          ],\\n          \\\"semantic_type\\\": \\\"\\\",\\n
          \\\"description\\\": \\\"\\\"\\n          }\\n      },\\n      {\\n          \\\"column\\\": \\\"loc_3\\\",\\n          \\\"properties\\\": {\\n
          \\\"dtype\\\": \\\"number\\\",\\n          \\\"std\\\":
0.022425470193815165,\\n          \\\"min\\\": 0.113,\\n          \\\"max\\\":
0.24,\\n          \\\"num_unique_values\\\": 107,\\n          \\\"samples\\\": [\\n
0.217,\\n          0.155,\\n          0.171\\n          ],\\n          \\\"semantic_type\\\": \\\"\\\",\\n
          \\\"description\\\": \\\"\\\"\\n          }\\n      },\\n      {\\n          \\\"column\\\": \\\"loc_4\\\",\\n          \\\"properties\\\": {\\n
          \\\"dtype\\\": \\\"number\\\",\\n          \\\"std\\\":
0.022006568495482815,\\n          \\\"min\\\": 0.113,\\n          \\\"max\\\":
0.235,\\n          \\\"num_unique_values\\\": 107,\\n          \\\"samples\\\": [\\n
0.126,\\n          0.142,\\n          0.203\\n          ],\\n          \\\"semantic_type\\\": \\\"\\\",\\n
          \\\"description\\\": \\\"\\\"\\n          }\\n      }\\n  ]\\n}\\",\"type\":\"dataframe\", \"variable_name\":\"df\"}

```

```

X = df.to_numpy()
plt.figure(figsize=(12, 3))
plt.imshow(X.T[:, :24+1], cmap='viridis', aspect='auto',
interpolation='none')
plt.colorbar(label='value')
plt.xlabel("time")
plt.ylabel("location")
plt.title("1 day (24h) plot")
plt.show()

```



We will use past w (window) data samples to forecast values in h (horizon) steps.

For example $w=24$ and $h=1$. So we will use matrices X_i of size $5 \times w$ and y_i of size 5 to train a regressor, which will return $\hat{y} \in R^5$.

We will apply multiple convolution 1D filters of size $5 \times k$ s that move only horizontally from left to right. k s stands for *kernel size*. Each convolution reduces the processed sequence by $2 \times k_s // 2$. Finally, the sequences produced by the convolution layers will be delivered to MLP, with 5 output neurons.

As the loss function MSE (suitable for a regression task) will be used.

```
import numpy as np

# parameters
window_size = 24 # use last 24 hours to predict one hour ahead
n_features = n_locations

# tworzymy X i y
X = []
y = []

for t in range(T - window_size):
    X.append(df.iloc[t:t+window_size].values)
    y.append(df.iloc[t+window_size].values)

X = np.array(X) # shape: (samples, window_size, n_features)
y = np.array(y) # shape: (samples, n_features)

print("X shape:", X.shape)
print("y shape:", y.shape)

X shape: (696, 24, 5)
y shape: (696, 5)

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, shuffle=False
)
```

2.4.2 Tensorflow based implementation

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Conv1D, Dense, Flatten, Dropout
from tensorflow.keras import metrics

model = Sequential([
    Input(shape=(window_size, n_features)),
    Conv1D(filters=64, kernel_size=7, activation='relu'),
```

```

    Conv1D(filters=32, kernel_size=7, activation='relu'),
    Flatten(),
    Dense(64, activation='relu'),
    Dense(n_features) # prediction of values in all locations
])

model.compile(optimizer='adam', loss='mse', metrics=[
    metrics.MeanAbsoluteError(name='mae'),
    metrics.RootMeanSquaredError(name='rmse'),
    metrics.R2Score(name='r2')
])
model.summary()

```

Model: "sequential_49"

Layer (type)	Output Shape	
Param #		
conv1d (Conv1D)	(None, 18, 64)	
2,304		
conv1d_1 (Conv1D)	(None, 12, 32)	
14,368		
flatten_49 (Flatten)	(None, 384)	
0		
dense_107 (Dense)	(None, 64)	
24,640		
dense_108 (Dense)	(None, 5)	
325		

Total params: 41,637 (162.64 KB)

Trainable params: 41,637 (162.64 KB)

Non-trainable params: 0 (0.00 B)

```

history = model.fit(
    X_train, y_train,
    validation_data=(X_test, y_test),

```

```

    epochs=50,
    batch_size=32
)

Epoch 1/50
18/18 _____ 6s 177ms/step - loss: 0.0108 - mae: 0.0786
- r2: -16.6989 - rmse: 0.0999 - val_loss: 0.0016 - val_mae: 0.0266 -
val_r2: -0.5195 - val_rmse: 0.0400
Epoch 2/50
18/18 _____ 0s 11ms/step - loss: 0.0015 - mae: 0.0280 -
r2: -0.5545 - rmse: 0.0392 - val_loss: 0.0013 - val_mae: 0.0242 -
val_r2: -0.0597 - val_rmse: 0.0357
Epoch 3/50
18/18 _____ 0s 11ms/step - loss: 0.0012 - mae: 0.0243 -
r2: -0.0376 - rmse: 0.0347 - val_loss: 0.0012 - val_mae: 0.0230 -
val_r2: 0.0114 - val_rmse: 0.0341
Epoch 4/50
18/18 _____ 0s 10ms/step - loss: 0.0011 - mae: 0.0227 -
r2: 0.0701 - rmse: 0.0330 - val_loss: 0.0011 - val_mae: 0.0219 -
val_r2: 0.1082 - val_rmse: 0.0332
Epoch 5/50
18/18 _____ 0s 10ms/step - loss: 9.8346e-04 - mae:
0.0214 - r2: 0.1726 - rmse: 0.0313 - val_loss: 0.0010 - val_mae:
0.0204 - val_r2: 0.2204 - val_rmse: 0.0317
Epoch 6/50
18/18 _____ 0s 11ms/step - loss: 8.8578e-04 - mae:
0.0202 - r2: 0.2548 - rmse: 0.0297 - val_loss: 9.0059e-04 - val_mae:
0.0195 - val_r2: 0.2787 - val_rmse: 0.0300
Epoch 7/50
18/18 _____ 0s 10ms/step - loss: 7.9301e-04 - mae:
0.0193 - r2: 0.3069 - rmse: 0.0281 - val_loss: 8.2056e-04 - val_mae:
0.0189 - val_r2: 0.3086 - val_rmse: 0.0286
Epoch 8/50
18/18 _____ 0s 10ms/step - loss: 7.2457e-04 - mae:
0.0187 - r2: 0.3396 - rmse: 0.0269 - val_loss: 7.5283e-04 - val_mae:
0.0184 - val_r2: 0.3325 - val_rmse: 0.0274
Epoch 9/50
18/18 _____ 0s 6ms/step - loss: 6.6683e-04 - mae:
0.0181 - r2: 0.3654 - rmse: 0.0258 - val_loss: 7.0092e-04 - val_mae:
0.0180 - val_r2: 0.3520 - val_rmse: 0.0265
Epoch 10/50
18/18 _____ 0s 6ms/step - loss: 6.1852e-04 - mae:
0.0176 - r2: 0.3922 - rmse: 0.0248 - val_loss: 6.6621e-04 - val_mae:
0.0179 - val_r2: 0.3689 - val_rmse: 0.0258
Epoch 11/50
18/18 _____ 0s 6ms/step - loss: 5.8039e-04 - mae:
0.0172 - r2: 0.4173 - rmse: 0.0241 - val_loss: 6.3150e-04 - val_mae:
0.0175 - val_r2: 0.3870 - val_rmse: 0.0251
Epoch 12/50
18/18 _____ 0s 6ms/step - loss: 5.4724e-04 - mae:

```

0.0168 - r2: 0.4387 - rmse: 0.0234 - val_loss: 6.0101e-04 - val_mae:
0.0172 - val_r2: 0.4049 - val_rmse: 0.0245
Epoch 13/50
18/18 ————— 0s 6ms/step - loss: 5.1570e-04 - mae:
0.0164 - r2: 0.4609 - rmse: 0.0227 - val_loss: 5.6586e-04 - val_mae:
0.0168 - val_r2: 0.4240 - val_rmse: 0.0238
Epoch 14/50
18/18 ————— 0s 6ms/step - loss: 4.8569e-04 - mae:
0.0160 - r2: 0.4806 - rmse: 0.0220 - val_loss: 5.3787e-04 - val_mae:
0.0166 - val_r2: 0.4398 - val_rmse: 0.0232
Epoch 15/50
18/18 ————— 0s 6ms/step - loss: 4.5907e-04 - mae:
0.0157 - r2: 0.4991 - rmse: 0.0214 - val_loss: 5.0930e-04 - val_mae:
0.0163 - val_r2: 0.4569 - val_rmse: 0.0226
Epoch 16/50
18/18 ————— 0s 6ms/step - loss: 4.3473e-04 - mae:
0.0153 - r2: 0.5163 - rmse: 0.0208 - val_loss: 4.8189e-04 - val_mae:
0.0159 - val_r2: 0.4738 - val_rmse: 0.0220
Epoch 17/50
18/18 ————— 0s 6ms/step - loss: 4.1623e-04 - mae:
0.0151 - r2: 0.5268 - rmse: 0.0204 - val_loss: 4.4714e-04 - val_mae:
0.0151 - val_r2: 0.5044 - val_rmse: 0.0211
Epoch 18/50
18/18 ————— 0s 6ms/step - loss: 4.0054e-04 - mae:
0.0148 - r2: 0.5342 - rmse: 0.0200 - val_loss: 4.4478e-04 - val_mae:
0.0155 - val_r2: 0.5117 - val_rmse: 0.0211
Epoch 19/50
18/18 ————— 0s 6ms/step - loss: 3.8033e-04 - mae:
0.0145 - r2: 0.5592 - rmse: 0.0195 - val_loss: 4.0902e-04 - val_mae:
0.0148 - val_r2: 0.5211 - val_rmse: 0.0202
Epoch 20/50
18/18 ————— 0s 7ms/step - loss: 3.5554e-04 - mae:
0.0142 - r2: 0.5702 - rmse: 0.0188 - val_loss: 3.9021e-04 - val_mae:
0.0145 - val_r2: 0.5359 - val_rmse: 0.0198
Epoch 21/50
18/18 ————— 0s 6ms/step - loss: 3.4074e-04 - mae:
0.0140 - r2: 0.5815 - rmse: 0.0185 - val_loss: 3.7217e-04 - val_mae:
0.0142 - val_r2: 0.5538 - val_rmse: 0.0193
Epoch 22/50
18/18 ————— 0s 6ms/step - loss: 3.2720e-04 - mae:
0.0137 - r2: 0.5938 - rmse: 0.0181 - val_loss: 3.5540e-04 - val_mae:
0.0139 - val_r2: 0.5665 - val_rmse: 0.0189
Epoch 23/50
18/18 ————— 0s 6ms/step - loss: 3.1334e-04 - mae:
0.0135 - r2: 0.6043 - rmse: 0.0177 - val_loss: 3.4285e-04 - val_mae:
0.0137 - val_r2: 0.5782 - val_rmse: 0.0185
Epoch 24/50
18/18 ————— 0s 6ms/step - loss: 3.0129e-04 - mae:
0.0133 - r2: 0.6154 - rmse: 0.0174 - val_loss: 3.2871e-04 - val_mae:

```
0.0134 - val_r2: 0.5916 - val_rmse: 0.0181
Epoch 25/50
18/18 _____ 0s 6ms/step - loss: 2.9102e-04 - mae:
0.0131 - r2: 0.6244 - rmse: 0.0171 - val_loss: 3.1572e-04 - val_mae:
0.0133 - val_r2: 0.6034 - val_rmse: 0.0178
Epoch 26/50
18/18 _____ 0s 6ms/step - loss: 2.8429e-04 - mae:
0.0130 - r2: 0.6302 - rmse: 0.0169 - val_loss: 3.0898e-04 - val_mae:
0.0132 - val_r2: 0.6090 - val_rmse: 0.0176
Epoch 27/50
18/18 _____ 0s 6ms/step - loss: 2.7625e-04 - mae:
0.0128 - r2: 0.6365 - rmse: 0.0166 - val_loss: 2.9963e-04 - val_mae:
0.0131 - val_r2: 0.6175 - val_rmse: 0.0173
Epoch 28/50
18/18 _____ 0s 8ms/step - loss: 2.7183e-04 - mae:
0.0127 - r2: 0.6411 - rmse: 0.0165 - val_loss: 2.9407e-04 - val_mae:
0.0130 - val_r2: 0.6231 - val_rmse: 0.0171
Epoch 29/50
18/18 _____ 0s 6ms/step - loss: 2.6504e-04 - mae:
0.0126 - r2: 0.6471 - rmse: 0.0163 - val_loss: 2.8789e-04 - val_mae:
0.0129 - val_r2: 0.6292 - val_rmse: 0.0170
Epoch 30/50
18/18 _____ 0s 6ms/step - loss: 2.6198e-04 - mae:
0.0126 - r2: 0.6514 - rmse: 0.0162 - val_loss: 2.8216e-04 - val_mae:
0.0128 - val_r2: 0.6325 - val_rmse: 0.0168
Epoch 31/50
18/18 _____ 0s 6ms/step - loss: 2.5832e-04 - mae:
0.0125 - r2: 0.6551 - rmse: 0.0161 - val_loss: 2.7970e-04 - val_mae:
0.0128 - val_r2: 0.6354 - val_rmse: 0.0167
Epoch 32/50
18/18 _____ 0s 6ms/step - loss: 2.5504e-04 - mae:
0.0124 - r2: 0.6585 - rmse: 0.0160 - val_loss: 2.7540e-04 - val_mae:
0.0127 - val_r2: 0.6386 - val_rmse: 0.0166
Epoch 33/50
18/18 _____ 0s 6ms/step - loss: 2.5246e-04 - mae:
0.0124 - r2: 0.6606 - rmse: 0.0159 - val_loss: 2.7282e-04 - val_mae:
0.0127 - val_r2: 0.6412 - val_rmse: 0.0165
Epoch 34/50
18/18 _____ 0s 6ms/step - loss: 2.5014e-04 - mae:
0.0123 - r2: 0.6631 - rmse: 0.0158 - val_loss: 2.7098e-04 - val_mae:
0.0127 - val_r2: 0.6424 - val_rmse: 0.0165
Epoch 35/50
18/18 _____ 0s 7ms/step - loss: 2.4713e-04 - mae:
0.0123 - r2: 0.6654 - rmse: 0.0157 - val_loss: 2.6832e-04 - val_mae:
0.0127 - val_r2: 0.6444 - val_rmse: 0.0164
Epoch 36/50
18/18 _____ 0s 6ms/step - loss: 2.4420e-04 - mae:
0.0122 - r2: 0.6681 - rmse: 0.0156 - val_loss: 2.6523e-04 - val_mae:
0.0126 - val_r2: 0.6469 - val_rmse: 0.0163
```

Epoch 37/50
18/18 ————— 0s 6ms/step - loss: 2.4209e-04 - mae:
0.0122 - r2: 0.6707 - rmse: 0.0156 - val_loss: 2.6396e-04 - val_mae:
0.0126 - val_r2: 0.6473 - val_rmse: 0.0162

Epoch 38/50
18/18 ————— 0s 6ms/step - loss: 2.4075e-04 - mae:
0.0121 - r2: 0.6715 - rmse: 0.0155 - val_loss: 2.6345e-04 - val_mae:
0.0126 - val_r2: 0.6476 - val_rmse: 0.0162

Epoch 39/50
18/18 ————— 0s 6ms/step - loss: 2.3874e-04 - mae:
0.0121 - r2: 0.6732 - rmse: 0.0154 - val_loss: 2.6117e-04 - val_mae:
0.0126 - val_r2: 0.6492 - val_rmse: 0.0162

Epoch 40/50
18/18 ————— 0s 6ms/step - loss: 2.3773e-04 - mae:
0.0121 - r2: 0.6745 - rmse: 0.0154 - val_loss: 2.6051e-04 - val_mae:
0.0126 - val_r2: 0.6496 - val_rmse: 0.0161

Epoch 41/50
18/18 ————— 0s 6ms/step - loss: 2.3582e-04 - mae:
0.0120 - r2: 0.6757 - rmse: 0.0154 - val_loss: 2.6024e-04 - val_mae:
0.0126 - val_r2: 0.6496 - val_rmse: 0.0161

Epoch 42/50
18/18 ————— 0s 6ms/step - loss: 2.3434e-04 - mae:
0.0120 - r2: 0.6773 - rmse: 0.0153 - val_loss: 2.5796e-04 - val_mae:
0.0125 - val_r2: 0.6512 - val_rmse: 0.0161

Epoch 43/50
18/18 ————— 0s 7ms/step - loss: 2.3342e-04 - mae:
0.0120 - r2: 0.6780 - rmse: 0.0153 - val_loss: 2.5741e-04 - val_mae:
0.0125 - val_r2: 0.6513 - val_rmse: 0.0160

Epoch 44/50
18/18 ————— 0s 6ms/step - loss: 2.3172e-04 - mae:
0.0119 - r2: 0.6789 - rmse: 0.0152 - val_loss: 2.5556e-04 - val_mae:
0.0125 - val_r2: 0.6531 - val_rmse: 0.0160

Epoch 45/50
18/18 ————— 0s 6ms/step - loss: 2.2997e-04 - mae:
0.0119 - r2: 0.6805 - rmse: 0.0152 - val_loss: 2.5397e-04 - val_mae:
0.0125 - val_r2: 0.6538 - val_rmse: 0.0159

Epoch 46/50
18/18 ————— 0s 6ms/step - loss: 2.2883e-04 - mae:
0.0119 - r2: 0.6807 - rmse: 0.0151 - val_loss: 2.5244e-04 - val_mae:
0.0124 - val_r2: 0.6562 - val_rmse: 0.0159

Epoch 47/50
18/18 ————— 0s 6ms/step - loss: 2.2785e-04 - mae:
0.0118 - r2: 0.6818 - rmse: 0.0151 - val_loss: 2.5217e-04 - val_mae:
0.0124 - val_r2: 0.6562 - val_rmse: 0.0159

Epoch 48/50
18/18 ————— 0s 6ms/step - loss: 2.2635e-04 - mae:
0.0118 - r2: 0.6823 - rmse: 0.0150 - val_loss: 2.5093e-04 - val_mae:
0.0124 - val_r2: 0.6564 - val_rmse: 0.0158

Epoch 49/50

```

18/18 ————— 0s 9ms/step - loss: 2.2516e-04 - mae:
0.0118 - r2: 0.6834 - rmse: 0.0150 - val_loss: 2.5113e-04 - val_mae:
0.0124 - val_r2: 0.6566 - val_rmse: 0.0158
Epoch 50/50
18/18 ————— 0s 8ms/step - loss: 2.2425e-04 - mae:
0.0118 - r2: 0.6844 - rmse: 0.0150 - val_loss: 2.5166e-04 - val_mae:
0.0124 - val_r2: 0.6556 - val_rmse: 0.0159

import matplotlib.pyplot as plt

# evaluation
loss, mae, rmse, r2 = model.evaluate(X_test, y_test)
print(f"Test MAE: {mae} Test RMSE: {rmse} Test r2 {r2}" )

# prediction
y_pred = model.predict(X_test)

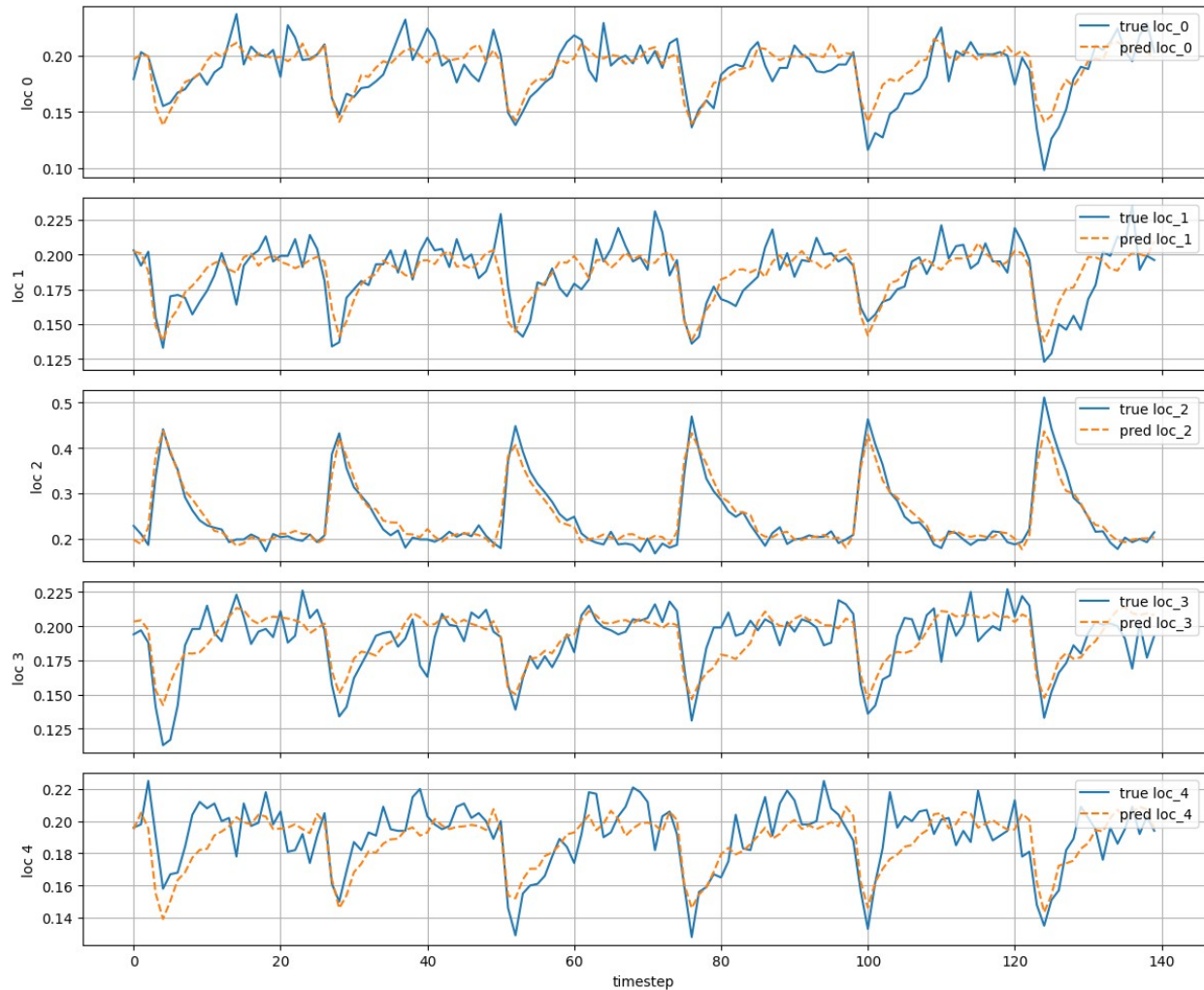
fig, axes = plt.subplots(n_locations, 1, figsize=(12, 2*n_locations),
sharex=True)

for j in range(n_locations):
    axes[j].plot(y_test[:150, j], label=f"true loc_{j}",
color='tab:blue')
    axes[j].plot(y_pred[:150, j], '--', label=f"pred loc_{j}",
color='tab:orange')
    axes[j].set_ylabel(f"loc {j}")
    axes[j].grid(True)
    axes[j].legend(loc='upper right')

axes[-1].set_xlabel("timestep")
plt.tight_layout()
plt.show()

5/5 ————— 0s 8ms/step - loss: 2.4477e-04 - mae: 0.0123
- r2: 0.6504 - rmse: 0.0156
Test MAE: 0.012422414496541023 Test RMSE: 0.015863876789808273 Test r2
0.6555877923965454
5/5 ————— 0s 59ms/step

```



```
from sklearn.metrics import r2_score, mean_squared_error
```

```
r2 = r2_score(y_test, y_pred)
print(f'MSE: {mean_squared_error(y_test, y_pred)} RMSE:
{np.sqrt(mean_squared_error(y_test,y_pred))} R2 score:
{r2_score(y_test, y_pred)}')
```

```
MSE: 0.00025166258583634573 RMSE: 0.01586387675936578 R2 score:
0.6555879750610675
```

2.4.3 PyTorch based implementation

```
configurations={
    'w_6':
    {'window_size':6,'out_channels1':64,'out_channels2':32,'kernel_size1':
    3, 'kernel_size2':3, 'dropout_rate': 0.0},
    'w_12':
    {'window_size':12,'out_channels1':64,'out_channels2':32,'kernel_size1'
```

```

:3, 'kernel_size2':3, 'dropout_rate': 0.0},
    'w_12_ks5':
    {'window_size':12, 'out_channels1':64, 'out_channels2':32, 'kernel_size1':
    :5, 'kernel_size2':5, 'dropout_rate': 0.0},
    'w_24':
    {'window_size':24, 'out_channels1':64, 'out_channels2':32, 'kernel_size1':
    :3, 'kernel_size2':3, 'dropout_rate': 0.0},
    'w_12_ks3_32_16':
    {'window_size':12, 'out_channels1':32, 'out_channels2':16, 'kernel_size1':
    :3, 'kernel_size2':3, 'dropout_rate': 0.0},
    'w_12_ks5_128_64':
    {'window_size':12, 'out_channels1':128, 'out_channels2':64, 'kernel_size1':
    :5, 'kernel_size2':5, 'dropout_rate': 0.0},
    'w_6_do':
    {'window_size':6, 'out_channels1':64, 'out_channels2':32, 'kernel_size1':
    :3, 'kernel_size2':3, 'dropout_rate': 0.2},
    'w_12_do':
    {'window_size':12, 'out_channels1':64, 'out_channels2':32, 'kernel_size1':
    :3, 'kernel_size2':3, 'dropout_rate': 0.2},
    'w_24_do':
    {'window_size':24, 'out_channels1':64, 'out_channels2':32, 'kernel_size1':
    :3, 'kernel_size2':3, 'dropout_rate': 0.2},
}

```

```

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_squared_error

```

```

batch_size_torch = 32
num_epochs = 100

```

```

summary_1dConv = []

```

```

for cname, cfg in configurations.items():

```

```

    ws = cfg['window_size']
    out1 = cfg['out_channels1']
    out2 = cfg['out_channels2']
    k1 = cfg['kernel_size1']
    k2 = cfg['kernel_size2']
    dropout_rate = cfg['dropout_rate']
    print(f'\n--- Running configuration: {cname} ---\n')
    print(f'Window Size: {ws}, Out Channels 1: {out1}, Out Channels 2:
{out2}, Kernel 1: {k1}, Kernel 2: {k2}, Dropout: {dropout_rate}')

```

```

    X_current = []
    y_current = []

```

```

for t in range(T - ws):
    X_current.append(df.iloc[t:t+ws].values)
    y_current.append(df.iloc[t:t+ws].values)

X_current = np.array(X_current)
y_current = np.array(y_current)

X_train_curr, X_test_curr, y_train_curr, y_test_curr =
train_test_split(
    X_current, y_current, test_size=0.2, shuffle=False
)

# --- Konwersja danych NumPy -> Torch ---
X_train_tensor = torch.tensor(X_train_curr, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train_curr, dtype=torch.float32)
X_test_tensor = torch.tensor(X_test_curr, dtype=torch.float32)
y_test_tensor = torch.tensor(y_test_curr, dtype=torch.float32)

# --- Dataset i DataLoader ---
train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
train_loader = DataLoader(train_dataset,
batch_size=batch_size_torch, shuffle=True)

val_dataset = TensorDataset(X_test_tensor, y_test_tensor)
val_loader = DataLoader(val_dataset, batch_size=batch_size_torch,
shuffle=False)

# --- Model ---
class Conv1DModel(nn.Module):
    def __init__(self, input_window_size, n_features, out1, out2,
k1, k2, dropout=0.0):
        super(Conv1DModel, self).__init__()
        self.conv1 = nn.Conv1d(in_channels=n_features,
out_channels=out1, kernel_size=k1)
        conv1_output_len = input_window_size - k1 + 1
        self.conv2 = nn.Conv1d(in_channels=out1, out_channels=out2,
kernel_size=k2)
        conv2_output_len = conv1_output_len - k2 + 1

        if conv2_output_len <= 0:
            raise ValueError("Convolutional layers result in an
output sequence length of 0 or less. Increase window_size or decrease
kernel_sizes.")

        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(dropout)
        self.flatten = nn.Flatten()

        self.fc1 = nn.Linear(out2 * conv2_output_len, 64)

```

```

        self.fc2 = nn.Linear(64, n_features)

    def forward(self, x):
        x = x.permute(0, 2, 1)
        x = self.relu(self.conv1(x))
        x = self.relu(self.conv2(x))
        x = self.flatten(x)
        x = self.dropout(x)
        x = self.relu(self.fc1(x))
        x = self.fc2(x)
        return x

    model = Conv1DModel(input_window_size=ws, n_features=n_features,
                        out1=out1, out2=out2, k1=k1, k2=k2, dropout=dropout_rate)

    optimizer = optim.Adam(model.parameters(), lr=0.001)
    criterion = nn.MSELoss()
    mae_metric = nn.L1Loss(reduction='mean')

    # --- Trening z validacją ---
    for epoch in range(num_epochs):
        # --- TRAIN ---
        model.train()
        train_loss = 0
        train_mae = 0
        for xb, yb in train_loader:
            optimizer.zero_grad()
            pred = model(xb)
            loss = criterion(pred, yb)
            loss.backward()
            optimizer.step()
            train_loss += loss.item() * xb.size(0)
            train_mae += mae_metric(pred, yb).item() * xb.size(0)
        train_loss /= len(train_loader.dataset)
        train_mae /= len(train_loader.dataset)

        # --- VALIDATION ---
        model.eval()
        val_loss = 0
        val_mae = 0
        with torch.no_grad():
            for xb, yb in val_loader:
                pred = model(xb)
                loss = criterion(pred, yb)
                val_loss += loss.item() * xb.size(0)
                val_mae += mae_metric(pred, yb).item() * xb.size(0)
        val_loss /= len(val_loader.dataset)
        val_mae /= len(val_loader.dataset)

    print(f"Epoch {epoch+1}/{num_epochs} | ")

```

```

        f"Train Loss: {train_loss:.4f}, Train MAE: {train_mae:.4f}
| "
        f"Val Loss: {val_loss:.4f}, Val MAE: {val_mae:.4f}")

# --- Evaluation on test set ---
model.eval()
with torch.no_grad():
    y_pred_tensor = model(X_test_tensor)
    y_pred = y_pred_tensor.numpy()

    mse = mean_squared_error(y_test_curr, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test_curr, y_pred)
    mae_test = mae_metric(y_pred_tensor, y_test_tensor).item()

    print(f'MSE: {mse:.6f}, RMSE: {rmse:.6f}, MAE: {mae_test:.6f}, R2
score: {r2:.6f}')

    summary_1dConv.append({
        'Config': cname,
        'Window Size': ws,
        'Out Channels 1': out1,
        'Out Channels 2': out2,
        'Kernel Size 1': k1,
        'Kernel Size 2': k2,
        'Dropout Rate': dropout_rate,
        'MAE': mae_test,
        'RMSE': rmse,
        'R2': r2
    })

summary_1dConv_df = pd.DataFrame(summary_1dConv)
display(summary_1dConv_df)

```

--- Running configuration: w_6 ---

```

Window Size: 6, Out Channels 1: 64, Out Channels 2: 32, Kernel 1: 3,
Kernel 2: 3, Dropout: 0.0
Epoch 1/100 | Train Loss: 0.0154, Train MAE: 0.0939 | Val Loss:
0.0031, Val MAE: 0.0436
Epoch 2/100 | Train Loss: 0.0023, Train MAE: 0.0366 | Val Loss:
0.0018, Val MAE: 0.0301
Epoch 3/100 | Train Loss: 0.0016, Train MAE: 0.0265 | Val Loss:
0.0016, Val MAE: 0.0263
Epoch 4/100 | Train Loss: 0.0015, Train MAE: 0.0265 | Val Loss:
0.0015, Val MAE: 0.0257
Epoch 5/100 | Train Loss: 0.0014, Train MAE: 0.0257 | Val Loss:
0.0015, Val MAE: 0.0258
Epoch 6/100 | Train Loss: 0.0014, Train MAE: 0.0255 | Val Loss:

```

0.0015, Val MAE: 0.0259
Epoch 7/100 | Train Loss: 0.0014, Train MAE: 0.0250 | Val Loss:
0.0014, Val MAE: 0.0252
Epoch 8/100 | Train Loss: 0.0013, Train MAE: 0.0242 | Val Loss:
0.0013, Val MAE: 0.0238
Epoch 9/100 | Train Loss: 0.0012, Train MAE: 0.0234 | Val Loss:
0.0013, Val MAE: 0.0218
Epoch 10/100 | Train Loss: 0.0011, Train MAE: 0.0218 | Val Loss:
0.0012, Val MAE: 0.0234
Epoch 11/100 | Train Loss: 0.0011, Train MAE: 0.0211 | Val Loss:
0.0011, Val MAE: 0.0226
Epoch 12/100 | Train Loss: 0.0010, Train MAE: 0.0207 | Val Loss:
0.0010, Val MAE: 0.0190
Epoch 13/100 | Train Loss: 0.0010, Train MAE: 0.0191 | Val Loss:
0.0010, Val MAE: 0.0187
Epoch 14/100 | Train Loss: 0.0009, Train MAE: 0.0188 | Val Loss:
0.0009, Val MAE: 0.0191
Epoch 15/100 | Train Loss: 0.0009, Train MAE: 0.0184 | Val Loss:
0.0009, Val MAE: 0.0171
Epoch 16/100 | Train Loss: 0.0008, Train MAE: 0.0181 | Val Loss:
0.0008, Val MAE: 0.0183
Epoch 17/100 | Train Loss: 0.0008, Train MAE: 0.0176 | Val Loss:
0.0008, Val MAE: 0.0177
Epoch 18/100 | Train Loss: 0.0007, Train MAE: 0.0172 | Val Loss:
0.0008, Val MAE: 0.0165
Epoch 19/100 | Train Loss: 0.0007, Train MAE: 0.0170 | Val Loss:
0.0007, Val MAE: 0.0166
Epoch 20/100 | Train Loss: 0.0007, Train MAE: 0.0169 | Val Loss:
0.0007, Val MAE: 0.0158
Epoch 21/100 | Train Loss: 0.0007, Train MAE: 0.0166 | Val Loss:
0.0007, Val MAE: 0.0179
Epoch 22/100 | Train Loss: 0.0006, Train MAE: 0.0165 | Val Loss:
0.0007, Val MAE: 0.0154
Epoch 23/100 | Train Loss: 0.0006, Train MAE: 0.0163 | Val Loss:
0.0006, Val MAE: 0.0163
Epoch 24/100 | Train Loss: 0.0006, Train MAE: 0.0159 | Val Loss:
0.0006, Val MAE: 0.0151
Epoch 25/100 | Train Loss: 0.0006, Train MAE: 0.0158 | Val Loss:
0.0006, Val MAE: 0.0148
Epoch 26/100 | Train Loss: 0.0006, Train MAE: 0.0153 | Val Loss:
0.0006, Val MAE: 0.0154
Epoch 27/100 | Train Loss: 0.0005, Train MAE: 0.0153 | Val Loss:
0.0006, Val MAE: 0.0143
Epoch 28/100 | Train Loss: 0.0005, Train MAE: 0.0150 | Val Loss:
0.0005, Val MAE: 0.0142
Epoch 29/100 | Train Loss: 0.0005, Train MAE: 0.0147 | Val Loss:
0.0005, Val MAE: 0.0141
Epoch 30/100 | Train Loss: 0.0005, Train MAE: 0.0148 | Val Loss:
0.0005, Val MAE: 0.0144

Epoch 31/100 | Train Loss: 0.0005, Train MAE: 0.0151 | Val Loss: 0.0005, Val MAE: 0.0141
Epoch 32/100 | Train Loss: 0.0005, Train MAE: 0.0150 | Val Loss: 0.0005, Val MAE: 0.0142
Epoch 33/100 | Train Loss: 0.0005, Train MAE: 0.0149 | Val Loss: 0.0005, Val MAE: 0.0147
Epoch 34/100 | Train Loss: 0.0005, Train MAE: 0.0147 | Val Loss: 0.0005, Val MAE: 0.0141
Epoch 35/100 | Train Loss: 0.0005, Train MAE: 0.0145 | Val Loss: 0.0005, Val MAE: 0.0137
Epoch 36/100 | Train Loss: 0.0005, Train MAE: 0.0146 | Val Loss: 0.0005, Val MAE: 0.0148
Epoch 37/100 | Train Loss: 0.0005, Train MAE: 0.0143 | Val Loss: 0.0005, Val MAE: 0.0136
Epoch 38/100 | Train Loss: 0.0005, Train MAE: 0.0141 | Val Loss: 0.0005, Val MAE: 0.0136
Epoch 39/100 | Train Loss: 0.0005, Train MAE: 0.0141 | Val Loss: 0.0005, Val MAE: 0.0149
Epoch 40/100 | Train Loss: 0.0005, Train MAE: 0.0144 | Val Loss: 0.0005, Val MAE: 0.0135
Epoch 41/100 | Train Loss: 0.0005, Train MAE: 0.0140 | Val Loss: 0.0005, Val MAE: 0.0136
Epoch 42/100 | Train Loss: 0.0005, Train MAE: 0.0142 | Val Loss: 0.0005, Val MAE: 0.0149
Epoch 43/100 | Train Loss: 0.0005, Train MAE: 0.0141 | Val Loss: 0.0005, Val MAE: 0.0137
Epoch 44/100 | Train Loss: 0.0004, Train MAE: 0.0139 | Val Loss: 0.0005, Val MAE: 0.0136
Epoch 45/100 | Train Loss: 0.0004, Train MAE: 0.0140 | Val Loss: 0.0005, Val MAE: 0.0137
Epoch 46/100 | Train Loss: 0.0004, Train MAE: 0.0141 | Val Loss: 0.0005, Val MAE: 0.0141
Epoch 47/100 | Train Loss: 0.0004, Train MAE: 0.0138 | Val Loss: 0.0005, Val MAE: 0.0141
Epoch 48/100 | Train Loss: 0.0005, Train MAE: 0.0142 | Val Loss: 0.0005, Val MAE: 0.0137
Epoch 49/100 | Train Loss: 0.0004, Train MAE: 0.0137 | Val Loss: 0.0005, Val MAE: 0.0135
Epoch 50/100 | Train Loss: 0.0004, Train MAE: 0.0140 | Val Loss: 0.0005, Val MAE: 0.0138
Epoch 51/100 | Train Loss: 0.0004, Train MAE: 0.0137 | Val Loss: 0.0005, Val MAE: 0.0135
Epoch 52/100 | Train Loss: 0.0004, Train MAE: 0.0137 | Val Loss: 0.0005, Val MAE: 0.0143
Epoch 53/100 | Train Loss: 0.0004, Train MAE: 0.0141 | Val Loss: 0.0005, Val MAE: 0.0134
Epoch 54/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0005, Val MAE: 0.0135
Epoch 55/100 | Train Loss: 0.0004, Train MAE: 0.0138 | Val Loss:

0.0005, Val MAE: 0.0135
Epoch 56/100 | Train Loss: 0.0004, Train MAE: 0.0138 | Val Loss: 0.0005, Val MAE: 0.0134
Epoch 57/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0005, Val MAE: 0.0137
Epoch 58/100 | Train Loss: 0.0004, Train MAE: 0.0140 | Val Loss: 0.0005, Val MAE: 0.0134
Epoch 59/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0005, Val MAE: 0.0133
Epoch 60/100 | Train Loss: 0.0004, Train MAE: 0.0138 | Val Loss: 0.0005, Val MAE: 0.0138
Epoch 61/100 | Train Loss: 0.0004, Train MAE: 0.0139 | Val Loss: 0.0005, Val MAE: 0.0133
Epoch 62/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0005, Val MAE: 0.0133
Epoch 63/100 | Train Loss: 0.0004, Train MAE: 0.0137 | Val Loss: 0.0005, Val MAE: 0.0133
Epoch 64/100 | Train Loss: 0.0004, Train MAE: 0.0137 | Val Loss: 0.0005, Val MAE: 0.0133
Epoch 65/100 | Train Loss: 0.0004, Train MAE: 0.0137 | Val Loss: 0.0005, Val MAE: 0.0131
Epoch 66/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0005, Val MAE: 0.0136
Epoch 67/100 | Train Loss: 0.0004, Train MAE: 0.0137 | Val Loss: 0.0005, Val MAE: 0.0131
Epoch 68/100 | Train Loss: 0.0004, Train MAE: 0.0135 | Val Loss: 0.0005, Val MAE: 0.0135
Epoch 69/100 | Train Loss: 0.0004, Train MAE: 0.0138 | Val Loss: 0.0005, Val MAE: 0.0132
Epoch 70/100 | Train Loss: 0.0004, Train MAE: 0.0135 | Val Loss: 0.0005, Val MAE: 0.0133
Epoch 71/100 | Train Loss: 0.0004, Train MAE: 0.0139 | Val Loss: 0.0005, Val MAE: 0.0135
Epoch 72/100 | Train Loss: 0.0005, Train MAE: 0.0145 | Val Loss: 0.0005, Val MAE: 0.0148
Epoch 73/100 | Train Loss: 0.0005, Train MAE: 0.0141 | Val Loss: 0.0005, Val MAE: 0.0138
Epoch 74/100 | Train Loss: 0.0004, Train MAE: 0.0143 | Val Loss: 0.0005, Val MAE: 0.0138
Epoch 75/100 | Train Loss: 0.0004, Train MAE: 0.0138 | Val Loss: 0.0005, Val MAE: 0.0137
Epoch 76/100 | Train Loss: 0.0004, Train MAE: 0.0142 | Val Loss: 0.0005, Val MAE: 0.0139
Epoch 77/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0005, Val MAE: 0.0133
Epoch 78/100 | Train Loss: 0.0004, Train MAE: 0.0137 | Val Loss: 0.0005, Val MAE: 0.0132
Epoch 79/100 | Train Loss: 0.0004, Train MAE: 0.0137 | Val Loss: 0.0005, Val MAE: 0.0133

Epoch 80/100 | Train Loss: 0.0004, Train MAE: 0.0134 | Val Loss: 0.0005, Val MAE: 0.0136
Epoch 81/100 | Train Loss: 0.0004, Train MAE: 0.0138 | Val Loss: 0.0005, Val MAE: 0.0133
Epoch 82/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0005, Val MAE: 0.0134
Epoch 83/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0005, Val MAE: 0.0133
Epoch 84/100 | Train Loss: 0.0004, Train MAE: 0.0135 | Val Loss: 0.0005, Val MAE: 0.0136
Epoch 85/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0005, Val MAE: 0.0131
Epoch 86/100 | Train Loss: 0.0004, Train MAE: 0.0134 | Val Loss: 0.0005, Val MAE: 0.0136
Epoch 87/100 | Train Loss: 0.0004, Train MAE: 0.0138 | Val Loss: 0.0005, Val MAE: 0.0134
Epoch 88/100 | Train Loss: 0.0004, Train MAE: 0.0135 | Val Loss: 0.0005, Val MAE: 0.0132
Epoch 89/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0005, Val MAE: 0.0132
Epoch 90/100 | Train Loss: 0.0004, Train MAE: 0.0135 | Val Loss: 0.0005, Val MAE: 0.0138
Epoch 91/100 | Train Loss: 0.0004, Train MAE: 0.0137 | Val Loss: 0.0005, Val MAE: 0.0134
Epoch 92/100 | Train Loss: 0.0004, Train MAE: 0.0137 | Val Loss: 0.0005, Val MAE: 0.0131
Epoch 93/100 | Train Loss: 0.0004, Train MAE: 0.0135 | Val Loss: 0.0005, Val MAE: 0.0131
Epoch 94/100 | Train Loss: 0.0004, Train MAE: 0.0137 | Val Loss: 0.0005, Val MAE: 0.0144
Epoch 95/100 | Train Loss: 0.0004, Train MAE: 0.0138 | Val Loss: 0.0005, Val MAE: 0.0133
Epoch 96/100 | Train Loss: 0.0004, Train MAE: 0.0137 | Val Loss: 0.0005, Val MAE: 0.0134
Epoch 97/100 | Train Loss: 0.0004, Train MAE: 0.0137 | Val Loss: 0.0005, Val MAE: 0.0136
Epoch 98/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0005, Val MAE: 0.0134
Epoch 99/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0005, Val MAE: 0.0135
Epoch 100/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0005, Val MAE: 0.0132
MSE: 0.000467, RMSE: 0.021613, MAE: 0.013154, R2 score: 0.566087

--- Running configuration: w_12 ---

Window Size: 12, Out Channels 1: 64, Out Channels 2: 32, Kernel 1: 3, Kernel 2: 3, Dropout: 0.0
Epoch 1/100 | Train Loss: 0.0224, Train MAE: 0.1160 | Val Loss:

0.0020, Val MAE: 0.0327
Epoch 2/100 | Train Loss: 0.0021, Train MAE: 0.0306 | Val Loss: 0.0018, Val MAE: 0.0301
Epoch 3/100 | Train Loss: 0.0016, Train MAE: 0.0280 | Val Loss: 0.0017, Val MAE: 0.0256
Epoch 4/100 | Train Loss: 0.0015, Train MAE: 0.0255 | Val Loss: 0.0016, Val MAE: 0.0270
Epoch 5/100 | Train Loss: 0.0015, Train MAE: 0.0260 | Val Loss: 0.0015, Val MAE: 0.0250
Epoch 6/100 | Train Loss: 0.0014, Train MAE: 0.0253 | Val Loss: 0.0015, Val MAE: 0.0263
Epoch 7/100 | Train Loss: 0.0014, Train MAE: 0.0251 | Val Loss: 0.0015, Val MAE: 0.0254
Epoch 8/100 | Train Loss: 0.0014, Train MAE: 0.0249 | Val Loss: 0.0015, Val MAE: 0.0256
Epoch 9/100 | Train Loss: 0.0014, Train MAE: 0.0251 | Val Loss: 0.0015, Val MAE: 0.0250
Epoch 10/100 | Train Loss: 0.0014, Train MAE: 0.0246 | Val Loss: 0.0014, Val MAE: 0.0245
Epoch 11/100 | Train Loss: 0.0013, Train MAE: 0.0241 | Val Loss: 0.0014, Val MAE: 0.0232
Epoch 12/100 | Train Loss: 0.0013, Train MAE: 0.0236 | Val Loss: 0.0013, Val MAE: 0.0228
Epoch 13/100 | Train Loss: 0.0012, Train MAE: 0.0226 | Val Loss: 0.0013, Val MAE: 0.0233
Epoch 14/100 | Train Loss: 0.0011, Train MAE: 0.0225 | Val Loss: 0.0012, Val MAE: 0.0213
Epoch 15/100 | Train Loss: 0.0011, Train MAE: 0.0214 | Val Loss: 0.0011, Val MAE: 0.0211
Epoch 16/100 | Train Loss: 0.0010, Train MAE: 0.0206 | Val Loss: 0.0011, Val MAE: 0.0212
Epoch 17/100 | Train Loss: 0.0010, Train MAE: 0.0203 | Val Loss: 0.0010, Val MAE: 0.0207
Epoch 18/100 | Train Loss: 0.0010, Train MAE: 0.0205 | Val Loss: 0.0010, Val MAE: 0.0201
Epoch 19/100 | Train Loss: 0.0008, Train MAE: 0.0190 | Val Loss: 0.0009, Val MAE: 0.0197
Epoch 20/100 | Train Loss: 0.0008, Train MAE: 0.0187 | Val Loss: 0.0009, Val MAE: 0.0195
Epoch 21/100 | Train Loss: 0.0008, Train MAE: 0.0184 | Val Loss: 0.0008, Val MAE: 0.0190
Epoch 22/100 | Train Loss: 0.0007, Train MAE: 0.0179 | Val Loss: 0.0008, Val MAE: 0.0187
Epoch 23/100 | Train Loss: 0.0007, Train MAE: 0.0178 | Val Loss: 0.0008, Val MAE: 0.0177
Epoch 24/100 | Train Loss: 0.0007, Train MAE: 0.0173 | Val Loss: 0.0007, Val MAE: 0.0174
Epoch 25/100 | Train Loss: 0.0006, Train MAE: 0.0170 | Val Loss: 0.0007, Val MAE: 0.0175

Epoch 26/100 | Train Loss: 0.0006, Train MAE: 0.0166 | Val Loss: 0.0006, Val MAE: 0.0168
Epoch 27/100 | Train Loss: 0.0006, Train MAE: 0.0162 | Val Loss: 0.0006, Val MAE: 0.0166
Epoch 28/100 | Train Loss: 0.0006, Train MAE: 0.0160 | Val Loss: 0.0006, Val MAE: 0.0159
Epoch 29/100 | Train Loss: 0.0005, Train MAE: 0.0159 | Val Loss: 0.0006, Val MAE: 0.0157
Epoch 30/100 | Train Loss: 0.0005, Train MAE: 0.0157 | Val Loss: 0.0006, Val MAE: 0.0160
Epoch 31/100 | Train Loss: 0.0005, Train MAE: 0.0160 | Val Loss: 0.0006, Val MAE: 0.0161
Epoch 32/100 | Train Loss: 0.0005, Train MAE: 0.0158 | Val Loss: 0.0006, Val MAE: 0.0161
Epoch 33/100 | Train Loss: 0.0005, Train MAE: 0.0155 | Val Loss: 0.0005, Val MAE: 0.0155
Epoch 34/100 | Train Loss: 0.0005, Train MAE: 0.0150 | Val Loss: 0.0006, Val MAE: 0.0167
Epoch 35/100 | Train Loss: 0.0005, Train MAE: 0.0150 | Val Loss: 0.0005, Val MAE: 0.0151
Epoch 36/100 | Train Loss: 0.0005, Train MAE: 0.0152 | Val Loss: 0.0005, Val MAE: 0.0149
Epoch 37/100 | Train Loss: 0.0005, Train MAE: 0.0148 | Val Loss: 0.0005, Val MAE: 0.0151
Epoch 38/100 | Train Loss: 0.0005, Train MAE: 0.0149 | Val Loss: 0.0005, Val MAE: 0.0149
Epoch 39/100 | Train Loss: 0.0004, Train MAE: 0.0147 | Val Loss: 0.0005, Val MAE: 0.0151
Epoch 40/100 | Train Loss: 0.0004, Train MAE: 0.0148 | Val Loss: 0.0006, Val MAE: 0.0157
Epoch 41/100 | Train Loss: 0.0004, Train MAE: 0.0147 | Val Loss: 0.0005, Val MAE: 0.0145
Epoch 42/100 | Train Loss: 0.0004, Train MAE: 0.0143 | Val Loss: 0.0005, Val MAE: 0.0150
Epoch 43/100 | Train Loss: 0.0004, Train MAE: 0.0143 | Val Loss: 0.0005, Val MAE: 0.0148
Epoch 44/100 | Train Loss: 0.0004, Train MAE: 0.0145 | Val Loss: 0.0005, Val MAE: 0.0145
Epoch 45/100 | Train Loss: 0.0004, Train MAE: 0.0143 | Val Loss: 0.0005, Val MAE: 0.0148
Epoch 46/100 | Train Loss: 0.0005, Train MAE: 0.0146 | Val Loss: 0.0005, Val MAE: 0.0150
Epoch 47/100 | Train Loss: 0.0004, Train MAE: 0.0142 | Val Loss: 0.0005, Val MAE: 0.0146
Epoch 48/100 | Train Loss: 0.0004, Train MAE: 0.0144 | Val Loss: 0.0005, Val MAE: 0.0143
Epoch 49/100 | Train Loss: 0.0004, Train MAE: 0.0141 | Val Loss: 0.0005, Val MAE: 0.0146
Epoch 50/100 | Train Loss: 0.0004, Train MAE: 0.0139 | Val Loss:

0.0005, Val MAE: 0.0142
Epoch 51/100 | Train Loss: 0.0004, Train MAE: 0.0140 | Val Loss: 0.0005, Val MAE: 0.0141
Epoch 52/100 | Train Loss: 0.0004, Train MAE: 0.0142 | Val Loss: 0.0005, Val MAE: 0.0142
Epoch 53/100 | Train Loss: 0.0004, Train MAE: 0.0140 | Val Loss: 0.0005, Val MAE: 0.0141
Epoch 54/100 | Train Loss: 0.0004, Train MAE: 0.0141 | Val Loss: 0.0005, Val MAE: 0.0148
Epoch 55/100 | Train Loss: 0.0004, Train MAE: 0.0140 | Val Loss: 0.0005, Val MAE: 0.0142
Epoch 56/100 | Train Loss: 0.0004, Train MAE: 0.0139 | Val Loss: 0.0005, Val MAE: 0.0145
Epoch 57/100 | Train Loss: 0.0004, Train MAE: 0.0139 | Val Loss: 0.0005, Val MAE: 0.0147
Epoch 58/100 | Train Loss: 0.0004, Train MAE: 0.0139 | Val Loss: 0.0005, Val MAE: 0.0141
Epoch 59/100 | Train Loss: 0.0004, Train MAE: 0.0140 | Val Loss: 0.0005, Val MAE: 0.0149
Epoch 60/100 | Train Loss: 0.0004, Train MAE: 0.0140 | Val Loss: 0.0005, Val MAE: 0.0144
Epoch 61/100 | Train Loss: 0.0004, Train MAE: 0.0138 | Val Loss: 0.0005, Val MAE: 0.0140
Epoch 62/100 | Train Loss: 0.0004, Train MAE: 0.0137 | Val Loss: 0.0005, Val MAE: 0.0142
Epoch 63/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0005, Val MAE: 0.0139
Epoch 64/100 | Train Loss: 0.0004, Train MAE: 0.0138 | Val Loss: 0.0005, Val MAE: 0.0150
Epoch 65/100 | Train Loss: 0.0004, Train MAE: 0.0140 | Val Loss: 0.0005, Val MAE: 0.0143
Epoch 66/100 | Train Loss: 0.0004, Train MAE: 0.0139 | Val Loss: 0.0004, Val MAE: 0.0139
Epoch 67/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0005, Val MAE: 0.0137
Epoch 68/100 | Train Loss: 0.0004, Train MAE: 0.0139 | Val Loss: 0.0005, Val MAE: 0.0138
Epoch 69/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0004, Val MAE: 0.0138
Epoch 70/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0005, Val MAE: 0.0141
Epoch 71/100 | Train Loss: 0.0004, Train MAE: 0.0137 | Val Loss: 0.0005, Val MAE: 0.0148
Epoch 72/100 | Train Loss: 0.0004, Train MAE: 0.0138 | Val Loss: 0.0005, Val MAE: 0.0139
Epoch 73/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0004, Val MAE: 0.0139
Epoch 74/100 | Train Loss: 0.0004, Train MAE: 0.0135 | Val Loss: 0.0004, Val MAE: 0.0137

Epoch 75/100 | Train Loss: 0.0004, Train MAE: 0.0133 | Val Loss: 0.0004, Val MAE: 0.0137
Epoch 76/100 | Train Loss: 0.0004, Train MAE: 0.0133 | Val Loss: 0.0005, Val MAE: 0.0141
Epoch 77/100 | Train Loss: 0.0004, Train MAE: 0.0133 | Val Loss: 0.0004, Val MAE: 0.0136
Epoch 78/100 | Train Loss: 0.0004, Train MAE: 0.0134 | Val Loss: 0.0005, Val MAE: 0.0138
Epoch 79/100 | Train Loss: 0.0004, Train MAE: 0.0135 | Val Loss: 0.0004, Val MAE: 0.0138
Epoch 80/100 | Train Loss: 0.0004, Train MAE: 0.0140 | Val Loss: 0.0005, Val MAE: 0.0139
Epoch 81/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0004, Val MAE: 0.0141
Epoch 82/100 | Train Loss: 0.0004, Train MAE: 0.0137 | Val Loss: 0.0005, Val MAE: 0.0141
Epoch 83/100 | Train Loss: 0.0004, Train MAE: 0.0134 | Val Loss: 0.0004, Val MAE: 0.0138
Epoch 84/100 | Train Loss: 0.0004, Train MAE: 0.0135 | Val Loss: 0.0005, Val MAE: 0.0138
Epoch 85/100 | Train Loss: 0.0004, Train MAE: 0.0134 | Val Loss: 0.0005, Val MAE: 0.0142
Epoch 86/100 | Train Loss: 0.0004, Train MAE: 0.0134 | Val Loss: 0.0005, Val MAE: 0.0138
Epoch 87/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0005, Val MAE: 0.0141
Epoch 88/100 | Train Loss: 0.0004, Train MAE: 0.0133 | Val Loss: 0.0004, Val MAE: 0.0139
Epoch 89/100 | Train Loss: 0.0004, Train MAE: 0.0134 | Val Loss: 0.0004, Val MAE: 0.0138
Epoch 90/100 | Train Loss: 0.0004, Train MAE: 0.0133 | Val Loss: 0.0005, Val MAE: 0.0137
Epoch 91/100 | Train Loss: 0.0004, Train MAE: 0.0133 | Val Loss: 0.0004, Val MAE: 0.0138
Epoch 92/100 | Train Loss: 0.0004, Train MAE: 0.0135 | Val Loss: 0.0005, Val MAE: 0.0145
Epoch 93/100 | Train Loss: 0.0004, Train MAE: 0.0139 | Val Loss: 0.0005, Val MAE: 0.0143
Epoch 94/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0005, Val MAE: 0.0144
Epoch 95/100 | Train Loss: 0.0004, Train MAE: 0.0135 | Val Loss: 0.0005, Val MAE: 0.0139
Epoch 96/100 | Train Loss: 0.0004, Train MAE: 0.0135 | Val Loss: 0.0004, Val MAE: 0.0135
Epoch 97/100 | Train Loss: 0.0004, Train MAE: 0.0132 | Val Loss: 0.0004, Val MAE: 0.0137
Epoch 98/100 | Train Loss: 0.0004, Train MAE: 0.0133 | Val Loss: 0.0004, Val MAE: 0.0139
Epoch 99/100 | Train Loss: 0.0004, Train MAE: 0.0133 | Val Loss:

0.0005, Val MAE: 0.0144
Epoch 100/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss:
0.0004, Val MAE: 0.0139
MSE: 0.000447, RMSE: 0.021150, MAE: 0.013873, R2 score: 0.544978

--- Running configuration: w_12_ks5 ---

Window Size: 12, Out Channels 1: 64, Out Channels 2: 32, Kernel 1: 5,
Kernel 2: 5, Dropout: 0.0

Epoch 1/100 | Train Loss: 0.0093, Train MAE: 0.0742 | Val Loss:
0.0026, Val MAE: 0.0335
Epoch 2/100 | Train Loss: 0.0021, Train MAE: 0.0320 | Val Loss:
0.0016, Val MAE: 0.0265
Epoch 3/100 | Train Loss: 0.0015, Train MAE: 0.0251 | Val Loss:
0.0016, Val MAE: 0.0263
Epoch 4/100 | Train Loss: 0.0015, Train MAE: 0.0264 | Val Loss:
0.0016, Val MAE: 0.0260
Epoch 5/100 | Train Loss: 0.0014, Train MAE: 0.0255 | Val Loss:
0.0016, Val MAE: 0.0255
Epoch 6/100 | Train Loss: 0.0014, Train MAE: 0.0260 | Val Loss:
0.0016, Val MAE: 0.0254
Epoch 7/100 | Train Loss: 0.0014, Train MAE: 0.0248 | Val Loss:
0.0015, Val MAE: 0.0262
Epoch 8/100 | Train Loss: 0.0014, Train MAE: 0.0263 | Val Loss:
0.0016, Val MAE: 0.0249
Epoch 9/100 | Train Loss: 0.0014, Train MAE: 0.0252 | Val Loss:
0.0015, Val MAE: 0.0260
Epoch 10/100 | Train Loss: 0.0014, Train MAE: 0.0256 | Val Loss:
0.0015, Val MAE: 0.0252
Epoch 11/100 | Train Loss: 0.0014, Train MAE: 0.0252 | Val Loss:
0.0015, Val MAE: 0.0260
Epoch 12/100 | Train Loss: 0.0014, Train MAE: 0.0253 | Val Loss:
0.0015, Val MAE: 0.0250
Epoch 13/100 | Train Loss: 0.0014, Train MAE: 0.0248 | Val Loss:
0.0014, Val MAE: 0.0255
Epoch 14/100 | Train Loss: 0.0013, Train MAE: 0.0254 | Val Loss:
0.0015, Val MAE: 0.0242
Epoch 15/100 | Train Loss: 0.0013, Train MAE: 0.0245 | Val Loss:
0.0014, Val MAE: 0.0244
Epoch 16/100 | Train Loss: 0.0013, Train MAE: 0.0245 | Val Loss:
0.0014, Val MAE: 0.0235
Epoch 17/100 | Train Loss: 0.0012, Train MAE: 0.0240 | Val Loss:
0.0013, Val MAE: 0.0235
Epoch 18/100 | Train Loss: 0.0012, Train MAE: 0.0234 | Val Loss:
0.0013, Val MAE: 0.0239
Epoch 19/100 | Train Loss: 0.0012, Train MAE: 0.0231 | Val Loss:
0.0012, Val MAE: 0.0222
Epoch 20/100 | Train Loss: 0.0011, Train MAE: 0.0222 | Val Loss:
0.0012, Val MAE: 0.0227

Epoch 21/100 | Train Loss: 0.0011, Train MAE: 0.0220 | Val Loss: 0.0011, Val MAE: 0.0226
Epoch 22/100 | Train Loss: 0.0011, Train MAE: 0.0214 | Val Loss: 0.0011, Val MAE: 0.0200
Epoch 23/100 | Train Loss: 0.0010, Train MAE: 0.0207 | Val Loss: 0.0010, Val MAE: 0.0203
Epoch 24/100 | Train Loss: 0.0009, Train MAE: 0.0200 | Val Loss: 0.0010, Val MAE: 0.0202
Epoch 25/100 | Train Loss: 0.0009, Train MAE: 0.0199 | Val Loss: 0.0009, Val MAE: 0.0198
Epoch 26/100 | Train Loss: 0.0009, Train MAE: 0.0193 | Val Loss: 0.0009, Val MAE: 0.0195
Epoch 27/100 | Train Loss: 0.0008, Train MAE: 0.0190 | Val Loss: 0.0009, Val MAE: 0.0177
Epoch 28/100 | Train Loss: 0.0008, Train MAE: 0.0182 | Val Loss: 0.0008, Val MAE: 0.0178
Epoch 29/100 | Train Loss: 0.0008, Train MAE: 0.0184 | Val Loss: 0.0010, Val MAE: 0.0185
Epoch 30/100 | Train Loss: 0.0007, Train MAE: 0.0179 | Val Loss: 0.0008, Val MAE: 0.0172
Epoch 31/100 | Train Loss: 0.0007, Train MAE: 0.0171 | Val Loss: 0.0007, Val MAE: 0.0172
Epoch 32/100 | Train Loss: 0.0007, Train MAE: 0.0169 | Val Loss: 0.0007, Val MAE: 0.0163
Epoch 33/100 | Train Loss: 0.0007, Train MAE: 0.0178 | Val Loss: 0.0008, Val MAE: 0.0167
Epoch 34/100 | Train Loss: 0.0006, Train MAE: 0.0165 | Val Loss: 0.0007, Val MAE: 0.0162
Epoch 35/100 | Train Loss: 0.0006, Train MAE: 0.0167 | Val Loss: 0.0006, Val MAE: 0.0166
Epoch 36/100 | Train Loss: 0.0006, Train MAE: 0.0161 | Val Loss: 0.0006, Val MAE: 0.0162
Epoch 37/100 | Train Loss: 0.0005, Train MAE: 0.0157 | Val Loss: 0.0006, Val MAE: 0.0154
Epoch 38/100 | Train Loss: 0.0006, Train MAE: 0.0159 | Val Loss: 0.0006, Val MAE: 0.0156
Epoch 39/100 | Train Loss: 0.0005, Train MAE: 0.0157 | Val Loss: 0.0005, Val MAE: 0.0154
Epoch 40/100 | Train Loss: 0.0005, Train MAE: 0.0152 | Val Loss: 0.0005, Val MAE: 0.0151
Epoch 41/100 | Train Loss: 0.0005, Train MAE: 0.0160 | Val Loss: 0.0006, Val MAE: 0.0158
Epoch 42/100 | Train Loss: 0.0005, Train MAE: 0.0160 | Val Loss: 0.0005, Val MAE: 0.0146
Epoch 43/100 | Train Loss: 0.0005, Train MAE: 0.0152 | Val Loss: 0.0006, Val MAE: 0.0166
Epoch 44/100 | Train Loss: 0.0005, Train MAE: 0.0150 | Val Loss: 0.0005, Val MAE: 0.0149
Epoch 45/100 | Train Loss: 0.0005, Train MAE: 0.0151 | Val Loss: 0.0005, Val MAE: 0.0154

Epoch 46/100 | Train Loss: 0.0005, Train MAE: 0.0149 | Val Loss: 0.0005, Val MAE: 0.0146
Epoch 47/100 | Train Loss: 0.0004, Train MAE: 0.0145 | Val Loss: 0.0005, Val MAE: 0.0146
Epoch 48/100 | Train Loss: 0.0004, Train MAE: 0.0144 | Val Loss: 0.0005, Val MAE: 0.0161
Epoch 49/100 | Train Loss: 0.0005, Train MAE: 0.0150 | Val Loss: 0.0005, Val MAE: 0.0147
Epoch 50/100 | Train Loss: 0.0004, Train MAE: 0.0147 | Val Loss: 0.0006, Val MAE: 0.0178
Epoch 51/100 | Train Loss: 0.0005, Train MAE: 0.0153 | Val Loss: 0.0005, Val MAE: 0.0145
Epoch 52/100 | Train Loss: 0.0005, Train MAE: 0.0155 | Val Loss: 0.0005, Val MAE: 0.0142
Epoch 53/100 | Train Loss: 0.0004, Train MAE: 0.0145 | Val Loss: 0.0005, Val MAE: 0.0160
Epoch 54/100 | Train Loss: 0.0004, Train MAE: 0.0145 | Val Loss: 0.0005, Val MAE: 0.0148
Epoch 55/100 | Train Loss: 0.0004, Train MAE: 0.0142 | Val Loss: 0.0005, Val MAE: 0.0144
Epoch 56/100 | Train Loss: 0.0004, Train MAE: 0.0141 | Val Loss: 0.0005, Val MAE: 0.0140
Epoch 57/100 | Train Loss: 0.0004, Train MAE: 0.0138 | Val Loss: 0.0005, Val MAE: 0.0142
Epoch 58/100 | Train Loss: 0.0004, Train MAE: 0.0137 | Val Loss: 0.0004, Val MAE: 0.0140
Epoch 59/100 | Train Loss: 0.0004, Train MAE: 0.0139 | Val Loss: 0.0005, Val MAE: 0.0141
Epoch 60/100 | Train Loss: 0.0004, Train MAE: 0.0138 | Val Loss: 0.0005, Val MAE: 0.0138
Epoch 61/100 | Train Loss: 0.0004, Train MAE: 0.0137 | Val Loss: 0.0004, Val MAE: 0.0140
Epoch 62/100 | Train Loss: 0.0004, Train MAE: 0.0138 | Val Loss: 0.0004, Val MAE: 0.0139
Epoch 63/100 | Train Loss: 0.0004, Train MAE: 0.0138 | Val Loss: 0.0005, Val MAE: 0.0139
Epoch 64/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0005, Val MAE: 0.0137
Epoch 65/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0005, Val MAE: 0.0139
Epoch 66/100 | Train Loss: 0.0004, Train MAE: 0.0137 | Val Loss: 0.0005, Val MAE: 0.0147
Epoch 67/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0005, Val MAE: 0.0142
Epoch 68/100 | Train Loss: 0.0004, Train MAE: 0.0137 | Val Loss: 0.0004, Val MAE: 0.0139
Epoch 69/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0005, Val MAE: 0.0142
Epoch 70/100 | Train Loss: 0.0004, Train MAE: 0.0137 | Val Loss:

0.0005, Val MAE: 0.0137
Epoch 71/100 | Train Loss: 0.0004, Train MAE: 0.0143 | Val Loss: 0.0005, Val MAE: 0.0146
Epoch 72/100 | Train Loss: 0.0004, Train MAE: 0.0137 | Val Loss: 0.0004, Val MAE: 0.0138
Epoch 73/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0004, Val MAE: 0.0141
Epoch 74/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0005, Val MAE: 0.0138
Epoch 75/100 | Train Loss: 0.0004, Train MAE: 0.0139 | Val Loss: 0.0004, Val MAE: 0.0141
Epoch 76/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0005, Val MAE: 0.0141
Epoch 77/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0005, Val MAE: 0.0144
Epoch 78/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0004, Val MAE: 0.0138
Epoch 79/100 | Train Loss: 0.0004, Train MAE: 0.0133 | Val Loss: 0.0005, Val MAE: 0.0149
Epoch 80/100 | Train Loss: 0.0004, Train MAE: 0.0137 | Val Loss: 0.0004, Val MAE: 0.0136
Epoch 81/100 | Train Loss: 0.0004, Train MAE: 0.0135 | Val Loss: 0.0004, Val MAE: 0.0136
Epoch 82/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0004, Val MAE: 0.0139
Epoch 83/100 | Train Loss: 0.0004, Train MAE: 0.0137 | Val Loss: 0.0005, Val MAE: 0.0142
Epoch 84/100 | Train Loss: 0.0004, Train MAE: 0.0139 | Val Loss: 0.0005, Val MAE: 0.0150
Epoch 85/100 | Train Loss: 0.0004, Train MAE: 0.0135 | Val Loss: 0.0005, Val MAE: 0.0146
Epoch 86/100 | Train Loss: 0.0004, Train MAE: 0.0137 | Val Loss: 0.0005, Val MAE: 0.0141
Epoch 87/100 | Train Loss: 0.0004, Train MAE: 0.0134 | Val Loss: 0.0004, Val MAE: 0.0140
Epoch 88/100 | Train Loss: 0.0004, Train MAE: 0.0138 | Val Loss: 0.0004, Val MAE: 0.0137
Epoch 89/100 | Train Loss: 0.0004, Train MAE: 0.0135 | Val Loss: 0.0004, Val MAE: 0.0140
Epoch 90/100 | Train Loss: 0.0004, Train MAE: 0.0139 | Val Loss: 0.0005, Val MAE: 0.0138
Epoch 91/100 | Train Loss: 0.0004, Train MAE: 0.0138 | Val Loss: 0.0005, Val MAE: 0.0146
Epoch 92/100 | Train Loss: 0.0004, Train MAE: 0.0140 | Val Loss: 0.0004, Val MAE: 0.0136
Epoch 93/100 | Train Loss: 0.0004, Train MAE: 0.0135 | Val Loss: 0.0004, Val MAE: 0.0136
Epoch 94/100 | Train Loss: 0.0004, Train MAE: 0.0132 | Val Loss: 0.0004, Val MAE: 0.0137

Epoch 95/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0005, Val MAE: 0.0144
Epoch 96/100 | Train Loss: 0.0004, Train MAE: 0.0135 | Val Loss: 0.0004, Val MAE: 0.0138
Epoch 97/100 | Train Loss: 0.0004, Train MAE: 0.0135 | Val Loss: 0.0004, Val MAE: 0.0139
Epoch 98/100 | Train Loss: 0.0004, Train MAE: 0.0132 | Val Loss: 0.0005, Val MAE: 0.0144
Epoch 99/100 | Train Loss: 0.0004, Train MAE: 0.0137 | Val Loss: 0.0004, Val MAE: 0.0137
Epoch 100/100 | Train Loss: 0.0004, Train MAE: 0.0133 | Val Loss: 0.0004, Val MAE: 0.0137
MSE: 0.000443, RMSE: 0.021044, MAE: 0.013654, R2 score: 0.570417

--- Running configuration: w_24 ---

Window Size: 24, Out Channels 1: 64, Out Channels 2: 32, Kernel 1: 3, Kernel 2: 3, Dropout: 0.0

Epoch 1/100 | Train Loss: 0.0066, Train MAE: 0.0593 | Val Loss: 0.0026, Val MAE: 0.0340
Epoch 2/100 | Train Loss: 0.0018, Train MAE: 0.0295 | Val Loss: 0.0017, Val MAE: 0.0260
Epoch 3/100 | Train Loss: 0.0015, Train MAE: 0.0260 | Val Loss: 0.0016, Val MAE: 0.0255
Epoch 4/100 | Train Loss: 0.0015, Train MAE: 0.0250 | Val Loss: 0.0016, Val MAE: 0.0255
Epoch 5/100 | Train Loss: 0.0014, Train MAE: 0.0247 | Val Loss: 0.0015, Val MAE: 0.0245
Epoch 6/100 | Train Loss: 0.0013, Train MAE: 0.0236 | Val Loss: 0.0013, Val MAE: 0.0228
Epoch 7/100 | Train Loss: 0.0011, Train MAE: 0.0215 | Val Loss: 0.0011, Val MAE: 0.0221
Epoch 8/100 | Train Loss: 0.0009, Train MAE: 0.0204 | Val Loss: 0.0010, Val MAE: 0.0201
Epoch 9/100 | Train Loss: 0.0008, Train MAE: 0.0192 | Val Loss: 0.0009, Val MAE: 0.0190
Epoch 10/100 | Train Loss: 0.0007, Train MAE: 0.0181 | Val Loss: 0.0008, Val MAE: 0.0184
Epoch 11/100 | Train Loss: 0.0007, Train MAE: 0.0178 | Val Loss: 0.0007, Val MAE: 0.0171
Epoch 12/100 | Train Loss: 0.0006, Train MAE: 0.0173 | Val Loss: 0.0007, Val MAE: 0.0174
Epoch 13/100 | Train Loss: 0.0006, Train MAE: 0.0166 | Val Loss: 0.0006, Val MAE: 0.0160
Epoch 14/100 | Train Loss: 0.0005, Train MAE: 0.0160 | Val Loss: 0.0005, Val MAE: 0.0157
Epoch 15/100 | Train Loss: 0.0005, Train MAE: 0.0158 | Val Loss: 0.0005, Val MAE: 0.0155
Epoch 16/100 | Train Loss: 0.0005, Train MAE: 0.0157 | Val Loss:

0.0005, Val MAE: 0.0160
Epoch 17/100 | Train Loss: 0.0005, Train MAE: 0.0156 | Val Loss: 0.0005, Val MAE: 0.0151
Epoch 18/100 | Train Loss: 0.0004, Train MAE: 0.0146 | Val Loss: 0.0004, Val MAE: 0.0144
Epoch 19/100 | Train Loss: 0.0004, Train MAE: 0.0142 | Val Loss: 0.0004, Val MAE: 0.0138
Epoch 20/100 | Train Loss: 0.0003, Train MAE: 0.0138 | Val Loss: 0.0004, Val MAE: 0.0136
Epoch 21/100 | Train Loss: 0.0003, Train MAE: 0.0136 | Val Loss: 0.0004, Val MAE: 0.0142
Epoch 22/100 | Train Loss: 0.0003, Train MAE: 0.0134 | Val Loss: 0.0003, Val MAE: 0.0130
Epoch 23/100 | Train Loss: 0.0003, Train MAE: 0.0133 | Val Loss: 0.0003, Val MAE: 0.0129
Epoch 24/100 | Train Loss: 0.0003, Train MAE: 0.0130 | Val Loss: 0.0003, Val MAE: 0.0132
Epoch 25/100 | Train Loss: 0.0003, Train MAE: 0.0130 | Val Loss: 0.0003, Val MAE: 0.0126
Epoch 26/100 | Train Loss: 0.0003, Train MAE: 0.0126 | Val Loss: 0.0003, Val MAE: 0.0122
Epoch 27/100 | Train Loss: 0.0003, Train MAE: 0.0127 | Val Loss: 0.0003, Val MAE: 0.0125
Epoch 28/100 | Train Loss: 0.0003, Train MAE: 0.0126 | Val Loss: 0.0003, Val MAE: 0.0122
Epoch 29/100 | Train Loss: 0.0003, Train MAE: 0.0124 | Val Loss: 0.0003, Val MAE: 0.0122
Epoch 30/100 | Train Loss: 0.0002, Train MAE: 0.0122 | Val Loss: 0.0002, Val MAE: 0.0118
Epoch 31/100 | Train Loss: 0.0002, Train MAE: 0.0120 | Val Loss: 0.0003, Val MAE: 0.0123
Epoch 32/100 | Train Loss: 0.0002, Train MAE: 0.0123 | Val Loss: 0.0002, Val MAE: 0.0123
Epoch 33/100 | Train Loss: 0.0002, Train MAE: 0.0120 | Val Loss: 0.0002, Val MAE: 0.0121
Epoch 34/100 | Train Loss: 0.0002, Train MAE: 0.0120 | Val Loss: 0.0002, Val MAE: 0.0120
Epoch 35/100 | Train Loss: 0.0002, Train MAE: 0.0119 | Val Loss: 0.0002, Val MAE: 0.0119
Epoch 36/100 | Train Loss: 0.0002, Train MAE: 0.0118 | Val Loss: 0.0002, Val MAE: 0.0119
Epoch 37/100 | Train Loss: 0.0002, Train MAE: 0.0118 | Val Loss: 0.0002, Val MAE: 0.0122
Epoch 38/100 | Train Loss: 0.0002, Train MAE: 0.0118 | Val Loss: 0.0002, Val MAE: 0.0115
Epoch 39/100 | Train Loss: 0.0002, Train MAE: 0.0117 | Val Loss: 0.0002, Val MAE: 0.0118
Epoch 40/100 | Train Loss: 0.0002, Train MAE: 0.0116 | Val Loss: 0.0003, Val MAE: 0.0124

Epoch 41/100 | Train Loss: 0.0002, Train MAE: 0.0119 | Val Loss: 0.0002, Val MAE: 0.0118
Epoch 42/100 | Train Loss: 0.0002, Train MAE: 0.0116 | Val Loss: 0.0002, Val MAE: 0.0115
Epoch 43/100 | Train Loss: 0.0002, Train MAE: 0.0115 | Val Loss: 0.0002, Val MAE: 0.0121
Epoch 44/100 | Train Loss: 0.0002, Train MAE: 0.0116 | Val Loss: 0.0002, Val MAE: 0.0115
Epoch 45/100 | Train Loss: 0.0002, Train MAE: 0.0116 | Val Loss: 0.0002, Val MAE: 0.0121
Epoch 46/100 | Train Loss: 0.0002, Train MAE: 0.0116 | Val Loss: 0.0002, Val MAE: 0.0115
Epoch 47/100 | Train Loss: 0.0002, Train MAE: 0.0113 | Val Loss: 0.0002, Val MAE: 0.0118
Epoch 48/100 | Train Loss: 0.0002, Train MAE: 0.0114 | Val Loss: 0.0002, Val MAE: 0.0116
Epoch 49/100 | Train Loss: 0.0002, Train MAE: 0.0114 | Val Loss: 0.0002, Val MAE: 0.0113
Epoch 50/100 | Train Loss: 0.0002, Train MAE: 0.0114 | Val Loss: 0.0002, Val MAE: 0.0116
Epoch 51/100 | Train Loss: 0.0002, Train MAE: 0.0113 | Val Loss: 0.0003, Val MAE: 0.0124
Epoch 52/100 | Train Loss: 0.0002, Train MAE: 0.0118 | Val Loss: 0.0002, Val MAE: 0.0121
Epoch 53/100 | Train Loss: 0.0002, Train MAE: 0.0116 | Val Loss: 0.0002, Val MAE: 0.0118
Epoch 54/100 | Train Loss: 0.0002, Train MAE: 0.0114 | Val Loss: 0.0002, Val MAE: 0.0114
Epoch 55/100 | Train Loss: 0.0002, Train MAE: 0.0113 | Val Loss: 0.0002, Val MAE: 0.0118
Epoch 56/100 | Train Loss: 0.0002, Train MAE: 0.0112 | Val Loss: 0.0002, Val MAE: 0.0115
Epoch 57/100 | Train Loss: 0.0002, Train MAE: 0.0115 | Val Loss: 0.0002, Val MAE: 0.0119
Epoch 58/100 | Train Loss: 0.0002, Train MAE: 0.0114 | Val Loss: 0.0002, Val MAE: 0.0117
Epoch 59/100 | Train Loss: 0.0002, Train MAE: 0.0114 | Val Loss: 0.0002, Val MAE: 0.0116
Epoch 60/100 | Train Loss: 0.0002, Train MAE: 0.0114 | Val Loss: 0.0003, Val MAE: 0.0126
Epoch 61/100 | Train Loss: 0.0002, Train MAE: 0.0119 | Val Loss: 0.0002, Val MAE: 0.0118
Epoch 62/100 | Train Loss: 0.0002, Train MAE: 0.0113 | Val Loss: 0.0002, Val MAE: 0.0114
Epoch 63/100 | Train Loss: 0.0002, Train MAE: 0.0113 | Val Loss: 0.0002, Val MAE: 0.0115
Epoch 64/100 | Train Loss: 0.0002, Train MAE: 0.0114 | Val Loss: 0.0002, Val MAE: 0.0116
Epoch 65/100 | Train Loss: 0.0002, Train MAE: 0.0113 | Val Loss:

0.0002, Val MAE: 0.0117
Epoch 66/100 | Train Loss: 0.0002, Train MAE: 0.0112 | Val Loss: 0.0002, Val MAE: 0.0119
Epoch 67/100 | Train Loss: 0.0002, Train MAE: 0.0114 | Val Loss: 0.0002, Val MAE: 0.0121
Epoch 68/100 | Train Loss: 0.0002, Train MAE: 0.0115 | Val Loss: 0.0002, Val MAE: 0.0117
Epoch 69/100 | Train Loss: 0.0002, Train MAE: 0.0111 | Val Loss: 0.0002, Val MAE: 0.0114
Epoch 70/100 | Train Loss: 0.0002, Train MAE: 0.0112 | Val Loss: 0.0002, Val MAE: 0.0117
Epoch 71/100 | Train Loss: 0.0002, Train MAE: 0.0113 | Val Loss: 0.0002, Val MAE: 0.0118
Epoch 72/100 | Train Loss: 0.0002, Train MAE: 0.0116 | Val Loss: 0.0002, Val MAE: 0.0115
Epoch 73/100 | Train Loss: 0.0002, Train MAE: 0.0115 | Val Loss: 0.0002, Val MAE: 0.0119
Epoch 74/100 | Train Loss: 0.0002, Train MAE: 0.0112 | Val Loss: 0.0002, Val MAE: 0.0116
Epoch 75/100 | Train Loss: 0.0002, Train MAE: 0.0114 | Val Loss: 0.0002, Val MAE: 0.0114
Epoch 76/100 | Train Loss: 0.0002, Train MAE: 0.0112 | Val Loss: 0.0002, Val MAE: 0.0115
Epoch 77/100 | Train Loss: 0.0002, Train MAE: 0.0113 | Val Loss: 0.0002, Val MAE: 0.0115
Epoch 78/100 | Train Loss: 0.0002, Train MAE: 0.0113 | Val Loss: 0.0002, Val MAE: 0.0117
Epoch 79/100 | Train Loss: 0.0002, Train MAE: 0.0110 | Val Loss: 0.0002, Val MAE: 0.0115
Epoch 80/100 | Train Loss: 0.0002, Train MAE: 0.0110 | Val Loss: 0.0002, Val MAE: 0.0115
Epoch 81/100 | Train Loss: 0.0002, Train MAE: 0.0111 | Val Loss: 0.0002, Val MAE: 0.0117
Epoch 82/100 | Train Loss: 0.0002, Train MAE: 0.0111 | Val Loss: 0.0002, Val MAE: 0.0115
Epoch 83/100 | Train Loss: 0.0002, Train MAE: 0.0117 | Val Loss: 0.0002, Val MAE: 0.0115
Epoch 84/100 | Train Loss: 0.0002, Train MAE: 0.0115 | Val Loss: 0.0002, Val MAE: 0.0113
Epoch 85/100 | Train Loss: 0.0002, Train MAE: 0.0113 | Val Loss: 0.0002, Val MAE: 0.0116
Epoch 86/100 | Train Loss: 0.0002, Train MAE: 0.0112 | Val Loss: 0.0002, Val MAE: 0.0114
Epoch 87/100 | Train Loss: 0.0002, Train MAE: 0.0110 | Val Loss: 0.0002, Val MAE: 0.0113
Epoch 88/100 | Train Loss: 0.0002, Train MAE: 0.0111 | Val Loss: 0.0002, Val MAE: 0.0115
Epoch 89/100 | Train Loss: 0.0002, Train MAE: 0.0114 | Val Loss: 0.0002, Val MAE: 0.0112

Epoch 90/100 | Train Loss: 0.0002, Train MAE: 0.0112 | Val Loss: 0.0002, Val MAE: 0.0115
Epoch 91/100 | Train Loss: 0.0002, Train MAE: 0.0113 | Val Loss: 0.0002, Val MAE: 0.0118
Epoch 92/100 | Train Loss: 0.0002, Train MAE: 0.0110 | Val Loss: 0.0002, Val MAE: 0.0112
Epoch 93/100 | Train Loss: 0.0002, Train MAE: 0.0110 | Val Loss: 0.0002, Val MAE: 0.0112
Epoch 94/100 | Train Loss: 0.0002, Train MAE: 0.0110 | Val Loss: 0.0002, Val MAE: 0.0118
Epoch 95/100 | Train Loss: 0.0002, Train MAE: 0.0113 | Val Loss: 0.0002, Val MAE: 0.0116
Epoch 96/100 | Train Loss: 0.0002, Train MAE: 0.0112 | Val Loss: 0.0002, Val MAE: 0.0118
Epoch 97/100 | Train Loss: 0.0002, Train MAE: 0.0113 | Val Loss: 0.0002, Val MAE: 0.0115
Epoch 98/100 | Train Loss: 0.0002, Train MAE: 0.0111 | Val Loss: 0.0002, Val MAE: 0.0115
Epoch 99/100 | Train Loss: 0.0002, Train MAE: 0.0112 | Val Loss: 0.0002, Val MAE: 0.0116
Epoch 100/100 | Train Loss: 0.0002, Train MAE: 0.0110 | Val Loss: 0.0002, Val MAE: 0.0115
MSE: 0.000213, RMSE: 0.014579, MAE: 0.011473, R2 score: 0.677510

--- Running configuration: w_12_ks3_32_16 ---

Window Size: 12, Out Channels 1: 32, Out Channels 2: 16, Kernel 1: 3, Kernel 2: 3, Dropout: 0.0

Epoch 1/100 | Train Loss: 0.0169, Train MAE: 0.1019 | Val Loss: 0.0035, Val MAE: 0.0429
Epoch 2/100 | Train Loss: 0.0022, Train MAE: 0.0346 | Val Loss: 0.0018, Val MAE: 0.0272
Epoch 3/100 | Train Loss: 0.0016, Train MAE: 0.0277 | Val Loss: 0.0016, Val MAE: 0.0268
Epoch 4/100 | Train Loss: 0.0015, Train MAE: 0.0257 | Val Loss: 0.0016, Val MAE: 0.0257
Epoch 5/100 | Train Loss: 0.0015, Train MAE: 0.0251 | Val Loss: 0.0016, Val MAE: 0.0263
Epoch 6/100 | Train Loss: 0.0015, Train MAE: 0.0258 | Val Loss: 0.0016, Val MAE: 0.0251
Epoch 7/100 | Train Loss: 0.0015, Train MAE: 0.0255 | Val Loss: 0.0016, Val MAE: 0.0260
Epoch 8/100 | Train Loss: 0.0015, Train MAE: 0.0254 | Val Loss: 0.0016, Val MAE: 0.0249
Epoch 9/100 | Train Loss: 0.0015, Train MAE: 0.0250 | Val Loss: 0.0016, Val MAE: 0.0271
Epoch 10/100 | Train Loss: 0.0014, Train MAE: 0.0253 | Val Loss: 0.0016, Val MAE: 0.0253
Epoch 11/100 | Train Loss: 0.0014, Train MAE: 0.0251 | Val Loss:

0.0015, Val MAE: 0.0250
Epoch 12/100 | Train Loss: 0.0014, Train MAE: 0.0255 | Val Loss: 0.0016, Val MAE: 0.0242
Epoch 13/100 | Train Loss: 0.0014, Train MAE: 0.0251 | Val Loss: 0.0015, Val MAE: 0.0254
Epoch 14/100 | Train Loss: 0.0014, Train MAE: 0.0248 | Val Loss: 0.0015, Val MAE: 0.0239
Epoch 15/100 | Train Loss: 0.0014, Train MAE: 0.0247 | Val Loss: 0.0015, Val MAE: 0.0245
Epoch 16/100 | Train Loss: 0.0014, Train MAE: 0.0242 | Val Loss: 0.0014, Val MAE: 0.0255
Epoch 17/100 | Train Loss: 0.0013, Train MAE: 0.0241 | Val Loss: 0.0014, Val MAE: 0.0248
Epoch 18/100 | Train Loss: 0.0013, Train MAE: 0.0234 | Val Loss: 0.0014, Val MAE: 0.0240
Epoch 19/100 | Train Loss: 0.0012, Train MAE: 0.0231 | Val Loss: 0.0014, Val MAE: 0.0220
Epoch 20/100 | Train Loss: 0.0012, Train MAE: 0.0224 | Val Loss: 0.0013, Val MAE: 0.0242
Epoch 21/100 | Train Loss: 0.0012, Train MAE: 0.0221 | Val Loss: 0.0012, Val MAE: 0.0227
Epoch 22/100 | Train Loss: 0.0011, Train MAE: 0.0217 | Val Loss: 0.0012, Val MAE: 0.0228
Epoch 23/100 | Train Loss: 0.0011, Train MAE: 0.0219 | Val Loss: 0.0012, Val MAE: 0.0219
Epoch 24/100 | Train Loss: 0.0011, Train MAE: 0.0217 | Val Loss: 0.0012, Val MAE: 0.0217
Epoch 25/100 | Train Loss: 0.0011, Train MAE: 0.0215 | Val Loss: 0.0011, Val MAE: 0.0216
Epoch 26/100 | Train Loss: 0.0011, Train MAE: 0.0212 | Val Loss: 0.0011, Val MAE: 0.0211
Epoch 27/100 | Train Loss: 0.0010, Train MAE: 0.0210 | Val Loss: 0.0011, Val MAE: 0.0211
Epoch 28/100 | Train Loss: 0.0010, Train MAE: 0.0206 | Val Loss: 0.0011, Val MAE: 0.0210
Epoch 29/100 | Train Loss: 0.0010, Train MAE: 0.0205 | Val Loss: 0.0011, Val MAE: 0.0230
Epoch 30/100 | Train Loss: 0.0010, Train MAE: 0.0207 | Val Loss: 0.0010, Val MAE: 0.0200
Epoch 31/100 | Train Loss: 0.0009, Train MAE: 0.0198 | Val Loss: 0.0010, Val MAE: 0.0200
Epoch 32/100 | Train Loss: 0.0009, Train MAE: 0.0196 | Val Loss: 0.0010, Val MAE: 0.0194
Epoch 33/100 | Train Loss: 0.0009, Train MAE: 0.0195 | Val Loss: 0.0009, Val MAE: 0.0193
Epoch 34/100 | Train Loss: 0.0008, Train MAE: 0.0189 | Val Loss: 0.0009, Val MAE: 0.0190
Epoch 35/100 | Train Loss: 0.0008, Train MAE: 0.0182 | Val Loss: 0.0009, Val MAE: 0.0188

Epoch 36/100 | Train Loss: 0.0008, Train MAE: 0.0181 | Val Loss: 0.0008, Val MAE: 0.0191
Epoch 37/100 | Train Loss: 0.0007, Train MAE: 0.0178 | Val Loss: 0.0008, Val MAE: 0.0186
Epoch 38/100 | Train Loss: 0.0007, Train MAE: 0.0174 | Val Loss: 0.0008, Val MAE: 0.0175
Epoch 39/100 | Train Loss: 0.0007, Train MAE: 0.0168 | Val Loss: 0.0007, Val MAE: 0.0178
Epoch 40/100 | Train Loss: 0.0006, Train MAE: 0.0166 | Val Loss: 0.0007, Val MAE: 0.0170
Epoch 41/100 | Train Loss: 0.0006, Train MAE: 0.0166 | Val Loss: 0.0007, Val MAE: 0.0167
Epoch 42/100 | Train Loss: 0.0006, Train MAE: 0.0167 | Val Loss: 0.0007, Val MAE: 0.0184
Epoch 43/100 | Train Loss: 0.0006, Train MAE: 0.0164 | Val Loss: 0.0007, Val MAE: 0.0175
Epoch 44/100 | Train Loss: 0.0006, Train MAE: 0.0161 | Val Loss: 0.0006, Val MAE: 0.0163
Epoch 45/100 | Train Loss: 0.0006, Train MAE: 0.0160 | Val Loss: 0.0006, Val MAE: 0.0161
Epoch 46/100 | Train Loss: 0.0006, Train MAE: 0.0161 | Val Loss: 0.0006, Val MAE: 0.0163
Epoch 47/100 | Train Loss: 0.0005, Train MAE: 0.0156 | Val Loss: 0.0006, Val MAE: 0.0162
Epoch 48/100 | Train Loss: 0.0005, Train MAE: 0.0155 | Val Loss: 0.0006, Val MAE: 0.0160
Epoch 49/100 | Train Loss: 0.0005, Train MAE: 0.0155 | Val Loss: 0.0007, Val MAE: 0.0164
Epoch 50/100 | Train Loss: 0.0006, Train MAE: 0.0164 | Val Loss: 0.0006, Val MAE: 0.0162
Epoch 51/100 | Train Loss: 0.0005, Train MAE: 0.0159 | Val Loss: 0.0006, Val MAE: 0.0156
Epoch 52/100 | Train Loss: 0.0005, Train MAE: 0.0154 | Val Loss: 0.0005, Val MAE: 0.0159
Epoch 53/100 | Train Loss: 0.0005, Train MAE: 0.0151 | Val Loss: 0.0006, Val MAE: 0.0153
Epoch 54/100 | Train Loss: 0.0005, Train MAE: 0.0149 | Val Loss: 0.0006, Val MAE: 0.0154
Epoch 55/100 | Train Loss: 0.0005, Train MAE: 0.0155 | Val Loss: 0.0006, Val MAE: 0.0175
Epoch 56/100 | Train Loss: 0.0005, Train MAE: 0.0155 | Val Loss: 0.0006, Val MAE: 0.0160
Epoch 57/100 | Train Loss: 0.0005, Train MAE: 0.0151 | Val Loss: 0.0006, Val MAE: 0.0151
Epoch 58/100 | Train Loss: 0.0005, Train MAE: 0.0148 | Val Loss: 0.0005, Val MAE: 0.0155
Epoch 59/100 | Train Loss: 0.0005, Train MAE: 0.0148 | Val Loss: 0.0005, Val MAE: 0.0147
Epoch 60/100 | Train Loss: 0.0005, Train MAE: 0.0148 | Val Loss:

0.0005, Val MAE: 0.0151
Epoch 61/100 | Train Loss: 0.0005, Train MAE: 0.0146 | Val Loss:
0.0005, Val MAE: 0.0150
Epoch 62/100 | Train Loss: 0.0004, Train MAE: 0.0149 | Val Loss:
0.0005, Val MAE: 0.0149
Epoch 63/100 | Train Loss: 0.0004, Train MAE: 0.0144 | Val Loss:
0.0005, Val MAE: 0.0145
Epoch 64/100 | Train Loss: 0.0005, Train MAE: 0.0149 | Val Loss:
0.0005, Val MAE: 0.0147
Epoch 65/100 | Train Loss: 0.0004, Train MAE: 0.0144 | Val Loss:
0.0005, Val MAE: 0.0143
Epoch 66/100 | Train Loss: 0.0005, Train MAE: 0.0147 | Val Loss:
0.0005, Val MAE: 0.0144
Epoch 67/100 | Train Loss: 0.0004, Train MAE: 0.0143 | Val Loss:
0.0005, Val MAE: 0.0157
Epoch 68/100 | Train Loss: 0.0004, Train MAE: 0.0145 | Val Loss:
0.0005, Val MAE: 0.0146
Epoch 69/100 | Train Loss: 0.0004, Train MAE: 0.0145 | Val Loss:
0.0005, Val MAE: 0.0147
Epoch 70/100 | Train Loss: 0.0004, Train MAE: 0.0142 | Val Loss:
0.0005, Val MAE: 0.0144
Epoch 71/100 | Train Loss: 0.0004, Train MAE: 0.0142 | Val Loss:
0.0005, Val MAE: 0.0144
Epoch 72/100 | Train Loss: 0.0005, Train MAE: 0.0145 | Val Loss:
0.0005, Val MAE: 0.0143
Epoch 73/100 | Train Loss: 0.0004, Train MAE: 0.0145 | Val Loss:
0.0005, Val MAE: 0.0149
Epoch 74/100 | Train Loss: 0.0004, Train MAE: 0.0144 | Val Loss:
0.0005, Val MAE: 0.0140
Epoch 75/100 | Train Loss: 0.0004, Train MAE: 0.0141 | Val Loss:
0.0005, Val MAE: 0.0140
Epoch 76/100 | Train Loss: 0.0004, Train MAE: 0.0142 | Val Loss:
0.0005, Val MAE: 0.0143
Epoch 77/100 | Train Loss: 0.0004, Train MAE: 0.0138 | Val Loss:
0.0005, Val MAE: 0.0146
Epoch 78/100 | Train Loss: 0.0004, Train MAE: 0.0145 | Val Loss:
0.0005, Val MAE: 0.0141
Epoch 79/100 | Train Loss: 0.0004, Train MAE: 0.0140 | Val Loss:
0.0005, Val MAE: 0.0142
Epoch 80/100 | Train Loss: 0.0004, Train MAE: 0.0138 | Val Loss:
0.0005, Val MAE: 0.0144
Epoch 81/100 | Train Loss: 0.0004, Train MAE: 0.0140 | Val Loss:
0.0005, Val MAE: 0.0139
Epoch 82/100 | Train Loss: 0.0004, Train MAE: 0.0140 | Val Loss:
0.0004, Val MAE: 0.0140
Epoch 83/100 | Train Loss: 0.0004, Train MAE: 0.0138 | Val Loss:
0.0005, Val MAE: 0.0142
Epoch 84/100 | Train Loss: 0.0004, Train MAE: 0.0139 | Val Loss:
0.0005, Val MAE: 0.0139

Epoch 85/100 | Train Loss: 0.0004, Train MAE: 0.0139 | Val Loss: 0.0005, Val MAE: 0.0142
Epoch 86/100 | Train Loss: 0.0004, Train MAE: 0.0140 | Val Loss: 0.0005, Val MAE: 0.0139
Epoch 87/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0005, Val MAE: 0.0138
Epoch 88/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0005, Val MAE: 0.0148
Epoch 89/100 | Train Loss: 0.0004, Train MAE: 0.0144 | Val Loss: 0.0004, Val MAE: 0.0137
Epoch 90/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0004, Val MAE: 0.0140
Epoch 91/100 | Train Loss: 0.0004, Train MAE: 0.0138 | Val Loss: 0.0005, Val MAE: 0.0141
Epoch 92/100 | Train Loss: 0.0004, Train MAE: 0.0135 | Val Loss: 0.0005, Val MAE: 0.0142
Epoch 93/100 | Train Loss: 0.0004, Train MAE: 0.0135 | Val Loss: 0.0005, Val MAE: 0.0148
Epoch 94/100 | Train Loss: 0.0004, Train MAE: 0.0139 | Val Loss: 0.0005, Val MAE: 0.0137
Epoch 95/100 | Train Loss: 0.0004, Train MAE: 0.0139 | Val Loss: 0.0005, Val MAE: 0.0152
Epoch 96/100 | Train Loss: 0.0004, Train MAE: 0.0140 | Val Loss: 0.0005, Val MAE: 0.0141
Epoch 97/100 | Train Loss: 0.0004, Train MAE: 0.0135 | Val Loss: 0.0004, Val MAE: 0.0138
Epoch 98/100 | Train Loss: 0.0004, Train MAE: 0.0139 | Val Loss: 0.0005, Val MAE: 0.0150
Epoch 99/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0004, Val MAE: 0.0138
Epoch 100/100 | Train Loss: 0.0004, Train MAE: 0.0137 | Val Loss: 0.0004, Val MAE: 0.0138
MSE: 0.000445, RMSE: 0.021086, MAE: 0.013809, R2 score: 0.559088

--- Running configuration: w_12_ks5_128_64 ---

Window Size: 12, Out Channels 1: 128, Out Channels 2: 64, Kernel 1: 5, Kernel 2: 5, Dropout: 0.0
Epoch 1/100 | Train Loss: 0.0093, Train MAE: 0.0751 | Val Loss: 0.0031, Val MAE: 0.0410
Epoch 2/100 | Train Loss: 0.0019, Train MAE: 0.0315 | Val Loss: 0.0017, Val MAE: 0.0273
Epoch 3/100 | Train Loss: 0.0015, Train MAE: 0.0262 | Val Loss: 0.0016, Val MAE: 0.0267
Epoch 4/100 | Train Loss: 0.0014, Train MAE: 0.0258 | Val Loss: 0.0015, Val MAE: 0.0252
Epoch 5/100 | Train Loss: 0.0014, Train MAE: 0.0252 | Val Loss: 0.0015, Val MAE: 0.0259
Epoch 6/100 | Train Loss: 0.0014, Train MAE: 0.0254 | Val Loss: 0.0015, Val MAE: 0.0250

Epoch 7/100 | Train Loss: 0.0013, Train MAE: 0.0248 | Val Loss: 0.0014, Val MAE: 0.0239
Epoch 8/100 | Train Loss: 0.0013, Train MAE: 0.0241 | Val Loss: 0.0013, Val MAE: 0.0237
Epoch 9/100 | Train Loss: 0.0012, Train MAE: 0.0238 | Val Loss: 0.0013, Val MAE: 0.0234
Epoch 10/100 | Train Loss: 0.0012, Train MAE: 0.0234 | Val Loss: 0.0013, Val MAE: 0.0233
Epoch 11/100 | Train Loss: 0.0012, Train MAE: 0.0229 | Val Loss: 0.0012, Val MAE: 0.0229
Epoch 12/100 | Train Loss: 0.0011, Train MAE: 0.0218 | Val Loss: 0.0011, Val MAE: 0.0224
Epoch 13/100 | Train Loss: 0.0010, Train MAE: 0.0211 | Val Loss: 0.0011, Val MAE: 0.0205
Epoch 14/100 | Train Loss: 0.0010, Train MAE: 0.0204 | Val Loss: 0.0010, Val MAE: 0.0204
Epoch 15/100 | Train Loss: 0.0009, Train MAE: 0.0205 | Val Loss: 0.0015, Val MAE: 0.0224
Epoch 16/100 | Train Loss: 0.0010, Train MAE: 0.0215 | Val Loss: 0.0009, Val MAE: 0.0194
Epoch 17/100 | Train Loss: 0.0009, Train MAE: 0.0191 | Val Loss: 0.0010, Val MAE: 0.0183
Epoch 18/100 | Train Loss: 0.0008, Train MAE: 0.0189 | Val Loss: 0.0009, Val MAE: 0.0185
Epoch 19/100 | Train Loss: 0.0007, Train MAE: 0.0177 | Val Loss: 0.0009, Val MAE: 0.0204
Epoch 20/100 | Train Loss: 0.0007, Train MAE: 0.0177 | Val Loss: 0.0007, Val MAE: 0.0172
Epoch 21/100 | Train Loss: 0.0007, Train MAE: 0.0169 | Val Loss: 0.0007, Val MAE: 0.0166
Epoch 22/100 | Train Loss: 0.0006, Train MAE: 0.0163 | Val Loss: 0.0007, Val MAE: 0.0172
Epoch 23/100 | Train Loss: 0.0006, Train MAE: 0.0164 | Val Loss: 0.0007, Val MAE: 0.0162
Epoch 24/100 | Train Loss: 0.0006, Train MAE: 0.0164 | Val Loss: 0.0006, Val MAE: 0.0156
Epoch 25/100 | Train Loss: 0.0006, Train MAE: 0.0158 | Val Loss: 0.0006, Val MAE: 0.0160
Epoch 26/100 | Train Loss: 0.0005, Train MAE: 0.0156 | Val Loss: 0.0007, Val MAE: 0.0180
Epoch 27/100 | Train Loss: 0.0006, Train MAE: 0.0160 | Val Loss: 0.0006, Val MAE: 0.0164
Epoch 28/100 | Train Loss: 0.0005, Train MAE: 0.0155 | Val Loss: 0.0007, Val MAE: 0.0160
Epoch 29/100 | Train Loss: 0.0005, Train MAE: 0.0152 | Val Loss: 0.0005, Val MAE: 0.0152
Epoch 30/100 | Train Loss: 0.0005, Train MAE: 0.0148 | Val Loss: 0.0005, Val MAE: 0.0153
Epoch 31/100 | Train Loss: 0.0005, Train MAE: 0.0147 | Val Loss:

0.0006, Val MAE: 0.0152
Epoch 32/100 | Train Loss: 0.0005, Train MAE: 0.0151 | Val Loss: 0.0005, Val MAE: 0.0155
Epoch 33/100 | Train Loss: 0.0005, Train MAE: 0.0150 | Val Loss: 0.0005, Val MAE: 0.0151
Epoch 34/100 | Train Loss: 0.0004, Train MAE: 0.0147 | Val Loss: 0.0005, Val MAE: 0.0152
Epoch 35/100 | Train Loss: 0.0004, Train MAE: 0.0147 | Val Loss: 0.0005, Val MAE: 0.0150
Epoch 36/100 | Train Loss: 0.0004, Train MAE: 0.0141 | Val Loss: 0.0005, Val MAE: 0.0143
Epoch 37/100 | Train Loss: 0.0005, Train MAE: 0.0147 | Val Loss: 0.0006, Val MAE: 0.0165
Epoch 38/100 | Train Loss: 0.0004, Train MAE: 0.0145 | Val Loss: 0.0005, Val MAE: 0.0151
Epoch 39/100 | Train Loss: 0.0004, Train MAE: 0.0146 | Val Loss: 0.0005, Val MAE: 0.0145
Epoch 40/100 | Train Loss: 0.0004, Train MAE: 0.0139 | Val Loss: 0.0005, Val MAE: 0.0142
Epoch 41/100 | Train Loss: 0.0004, Train MAE: 0.0140 | Val Loss: 0.0005, Val MAE: 0.0146
Epoch 42/100 | Train Loss: 0.0004, Train MAE: 0.0139 | Val Loss: 0.0005, Val MAE: 0.0154
Epoch 43/100 | Train Loss: 0.0004, Train MAE: 0.0141 | Val Loss: 0.0005, Val MAE: 0.0145
Epoch 44/100 | Train Loss: 0.0004, Train MAE: 0.0139 | Val Loss: 0.0005, Val MAE: 0.0139
Epoch 45/100 | Train Loss: 0.0004, Train MAE: 0.0139 | Val Loss: 0.0005, Val MAE: 0.0148
Epoch 46/100 | Train Loss: 0.0004, Train MAE: 0.0139 | Val Loss: 0.0005, Val MAE: 0.0137
Epoch 47/100 | Train Loss: 0.0004, Train MAE: 0.0138 | Val Loss: 0.0005, Val MAE: 0.0142
Epoch 48/100 | Train Loss: 0.0004, Train MAE: 0.0143 | Val Loss: 0.0005, Val MAE: 0.0156
Epoch 49/100 | Train Loss: 0.0004, Train MAE: 0.0135 | Val Loss: 0.0005, Val MAE: 0.0141
Epoch 50/100 | Train Loss: 0.0004, Train MAE: 0.0135 | Val Loss: 0.0005, Val MAE: 0.0142
Epoch 51/100 | Train Loss: 0.0004, Train MAE: 0.0137 | Val Loss: 0.0005, Val MAE: 0.0143
Epoch 52/100 | Train Loss: 0.0004, Train MAE: 0.0134 | Val Loss: 0.0005, Val MAE: 0.0149
Epoch 53/100 | Train Loss: 0.0004, Train MAE: 0.0136 | Val Loss: 0.0005, Val MAE: 0.0142
Epoch 54/100 | Train Loss: 0.0004, Train MAE: 0.0134 | Val Loss: 0.0005, Val MAE: 0.0141
Epoch 55/100 | Train Loss: 0.0004, Train MAE: 0.0134 | Val Loss: 0.0005, Val MAE: 0.0148

Epoch 56/100 | Train Loss: 0.0004, Train MAE: 0.0133 | Val Loss: 0.0005, Val MAE: 0.0143
Epoch 57/100 | Train Loss: 0.0004, Train MAE: 0.0134 | Val Loss: 0.0005, Val MAE: 0.0140
Epoch 58/100 | Train Loss: 0.0004, Train MAE: 0.0132 | Val Loss: 0.0005, Val MAE: 0.0138
Epoch 59/100 | Train Loss: 0.0004, Train MAE: 0.0132 | Val Loss: 0.0005, Val MAE: 0.0139
Epoch 60/100 | Train Loss: 0.0004, Train MAE: 0.0140 | Val Loss: 0.0005, Val MAE: 0.0163
Epoch 61/100 | Train Loss: 0.0004, Train MAE: 0.0140 | Val Loss: 0.0005, Val MAE: 0.0140
Epoch 62/100 | Train Loss: 0.0004, Train MAE: 0.0134 | Val Loss: 0.0005, Val MAE: 0.0139
Epoch 63/100 | Train Loss: 0.0004, Train MAE: 0.0131 | Val Loss: 0.0004, Val MAE: 0.0139
Epoch 64/100 | Train Loss: 0.0004, Train MAE: 0.0132 | Val Loss: 0.0005, Val MAE: 0.0138
Epoch 65/100 | Train Loss: 0.0004, Train MAE: 0.0134 | Val Loss: 0.0005, Val MAE: 0.0153
Epoch 66/100 | Train Loss: 0.0004, Train MAE: 0.0132 | Val Loss: 0.0005, Val MAE: 0.0139
Epoch 67/100 | Train Loss: 0.0004, Train MAE: 0.0134 | Val Loss: 0.0005, Val MAE: 0.0141
Epoch 68/100 | Train Loss: 0.0004, Train MAE: 0.0131 | Val Loss: 0.0005, Val MAE: 0.0137
Epoch 69/100 | Train Loss: 0.0003, Train MAE: 0.0131 | Val Loss: 0.0005, Val MAE: 0.0140
Epoch 70/100 | Train Loss: 0.0004, Train MAE: 0.0129 | Val Loss: 0.0005, Val MAE: 0.0148
Epoch 71/100 | Train Loss: 0.0004, Train MAE: 0.0135 | Val Loss: 0.0005, Val MAE: 0.0138
Epoch 72/100 | Train Loss: 0.0003, Train MAE: 0.0131 | Val Loss: 0.0005, Val MAE: 0.0139
Epoch 73/100 | Train Loss: 0.0003, Train MAE: 0.0130 | Val Loss: 0.0005, Val MAE: 0.0139
Epoch 74/100 | Train Loss: 0.0004, Train MAE: 0.0131 | Val Loss: 0.0005, Val MAE: 0.0149
Epoch 75/100 | Train Loss: 0.0004, Train MAE: 0.0131 | Val Loss: 0.0005, Val MAE: 0.0140
Epoch 76/100 | Train Loss: 0.0003, Train MAE: 0.0130 | Val Loss: 0.0005, Val MAE: 0.0142
Epoch 77/100 | Train Loss: 0.0004, Train MAE: 0.0132 | Val Loss: 0.0005, Val MAE: 0.0144
Epoch 78/100 | Train Loss: 0.0003, Train MAE: 0.0129 | Val Loss: 0.0005, Val MAE: 0.0143
Epoch 79/100 | Train Loss: 0.0004, Train MAE: 0.0134 | Val Loss: 0.0006, Val MAE: 0.0174
Epoch 80/100 | Train Loss: 0.0004, Train MAE: 0.0140 | Val Loss:

0.0005, Val MAE: 0.0142
Epoch 81/100 | Train Loss: 0.0003, Train MAE: 0.0132 | Val Loss: 0.0005, Val MAE: 0.0141
Epoch 82/100 | Train Loss: 0.0003, Train MAE: 0.0130 | Val Loss: 0.0005, Val MAE: 0.0140
Epoch 83/100 | Train Loss: 0.0003, Train MAE: 0.0129 | Val Loss: 0.0005, Val MAE: 0.0141
Epoch 84/100 | Train Loss: 0.0003, Train MAE: 0.0129 | Val Loss: 0.0005, Val MAE: 0.0149
Epoch 85/100 | Train Loss: 0.0004, Train MAE: 0.0137 | Val Loss: 0.0005, Val MAE: 0.0139
Epoch 86/100 | Train Loss: 0.0004, Train MAE: 0.0133 | Val Loss: 0.0005, Val MAE: 0.0139
Epoch 87/100 | Train Loss: 0.0003, Train MAE: 0.0130 | Val Loss: 0.0005, Val MAE: 0.0140
Epoch 88/100 | Train Loss: 0.0003, Train MAE: 0.0129 | Val Loss: 0.0005, Val MAE: 0.0140
Epoch 89/100 | Train Loss: 0.0003, Train MAE: 0.0131 | Val Loss: 0.0005, Val MAE: 0.0139
Epoch 90/100 | Train Loss: 0.0004, Train MAE: 0.0134 | Val Loss: 0.0005, Val MAE: 0.0141
Epoch 91/100 | Train Loss: 0.0003, Train MAE: 0.0129 | Val Loss: 0.0005, Val MAE: 0.0140
Epoch 92/100 | Train Loss: 0.0003, Train MAE: 0.0128 | Val Loss: 0.0005, Val MAE: 0.0142
Epoch 93/100 | Train Loss: 0.0003, Train MAE: 0.0129 | Val Loss: 0.0005, Val MAE: 0.0142
Epoch 94/100 | Train Loss: 0.0003, Train MAE: 0.0129 | Val Loss: 0.0005, Val MAE: 0.0143
Epoch 95/100 | Train Loss: 0.0003, Train MAE: 0.0131 | Val Loss: 0.0005, Val MAE: 0.0140
Epoch 96/100 | Train Loss: 0.0003, Train MAE: 0.0128 | Val Loss: 0.0005, Val MAE: 0.0142
Epoch 97/100 | Train Loss: 0.0003, Train MAE: 0.0129 | Val Loss: 0.0005, Val MAE: 0.0140
Epoch 98/100 | Train Loss: 0.0003, Train MAE: 0.0130 | Val Loss: 0.0005, Val MAE: 0.0138
Epoch 99/100 | Train Loss: 0.0004, Train MAE: 0.0135 | Val Loss: 0.0005, Val MAE: 0.0142
Epoch 100/100 | Train Loss: 0.0004, Train MAE: 0.0131 | Val Loss: 0.0005, Val MAE: 0.0149
MSE: 0.000498, RMSE: 0.022325, MAE: 0.014876, R2 score: 0.526659

--- Running configuration: w_6_do ---

Window Size: 6, Out Channels 1: 64, Out Channels 2: 32, Kernel 1: 3, Kernel 2: 3, Dropout: 0.2
Epoch 1/100 | Train Loss: 0.0245, Train MAE: 0.1229 | Val Loss: 0.0052, Val MAE: 0.0572

Epoch 2/100 | Train Loss: 0.0033, Train MAE: 0.0438 | Val Loss: 0.0019, Val MAE: 0.0310
Epoch 3/100 | Train Loss: 0.0022, Train MAE: 0.0341 | Val Loss: 0.0017, Val MAE: 0.0273
Epoch 4/100 | Train Loss: 0.0020, Train MAE: 0.0297 | Val Loss: 0.0016, Val MAE: 0.0253
Epoch 5/100 | Train Loss: 0.0019, Train MAE: 0.0305 | Val Loss: 0.0016, Val MAE: 0.0263
Epoch 6/100 | Train Loss: 0.0019, Train MAE: 0.0297 | Val Loss: 0.0016, Val MAE: 0.0255
Epoch 7/100 | Train Loss: 0.0018, Train MAE: 0.0287 | Val Loss: 0.0016, Val MAE: 0.0264
Epoch 8/100 | Train Loss: 0.0018, Train MAE: 0.0287 | Val Loss: 0.0016, Val MAE: 0.0259
Epoch 9/100 | Train Loss: 0.0017, Train MAE: 0.0284 | Val Loss: 0.0016, Val MAE: 0.0258
Epoch 10/100 | Train Loss: 0.0017, Train MAE: 0.0275 | Val Loss: 0.0015, Val MAE: 0.0257
Epoch 11/100 | Train Loss: 0.0017, Train MAE: 0.0281 | Val Loss: 0.0015, Val MAE: 0.0256
Epoch 12/100 | Train Loss: 0.0017, Train MAE: 0.0278 | Val Loss: 0.0015, Val MAE: 0.0257
Epoch 13/100 | Train Loss: 0.0016, Train MAE: 0.0279 | Val Loss: 0.0015, Val MAE: 0.0251
Epoch 14/100 | Train Loss: 0.0016, Train MAE: 0.0270 | Val Loss: 0.0015, Val MAE: 0.0252
Epoch 15/100 | Train Loss: 0.0015, Train MAE: 0.0267 | Val Loss: 0.0014, Val MAE: 0.0249
Epoch 16/100 | Train Loss: 0.0015, Train MAE: 0.0268 | Val Loss: 0.0015, Val MAE: 0.0247
Epoch 17/100 | Train Loss: 0.0015, Train MAE: 0.0260 | Val Loss: 0.0014, Val MAE: 0.0244
Epoch 18/100 | Train Loss: 0.0014, Train MAE: 0.0260 | Val Loss: 0.0014, Val MAE: 0.0242
Epoch 19/100 | Train Loss: 0.0014, Train MAE: 0.0252 | Val Loss: 0.0014, Val MAE: 0.0229
Epoch 20/100 | Train Loss: 0.0013, Train MAE: 0.0243 | Val Loss: 0.0012, Val MAE: 0.0217
Epoch 21/100 | Train Loss: 0.0013, Train MAE: 0.0235 | Val Loss: 0.0011, Val MAE: 0.0206
Epoch 22/100 | Train Loss: 0.0012, Train MAE: 0.0223 | Val Loss: 0.0010, Val MAE: 0.0207
Epoch 23/100 | Train Loss: 0.0011, Train MAE: 0.0217 | Val Loss: 0.0010, Val MAE: 0.0187
Epoch 24/100 | Train Loss: 0.0011, Train MAE: 0.0208 | Val Loss: 0.0009, Val MAE: 0.0172
Epoch 25/100 | Train Loss: 0.0010, Train MAE: 0.0199 | Val Loss: 0.0008, Val MAE: 0.0180
Epoch 26/100 | Train Loss: 0.0010, Train MAE: 0.0200 | Val Loss:

0.0008, Val MAE: 0.0172
Epoch 27/100 | Train Loss: 0.0009, Train MAE: 0.0187 | Val Loss: 0.0008, Val MAE: 0.0167
Epoch 28/100 | Train Loss: 0.0009, Train MAE: 0.0190 | Val Loss: 0.0008, Val MAE: 0.0164
Epoch 29/100 | Train Loss: 0.0008, Train MAE: 0.0184 | Val Loss: 0.0007, Val MAE: 0.0159
Epoch 30/100 | Train Loss: 0.0008, Train MAE: 0.0182 | Val Loss: 0.0007, Val MAE: 0.0161
Epoch 31/100 | Train Loss: 0.0007, Train MAE: 0.0174 | Val Loss: 0.0007, Val MAE: 0.0154
Epoch 32/100 | Train Loss: 0.0008, Train MAE: 0.0176 | Val Loss: 0.0006, Val MAE: 0.0164
Epoch 33/100 | Train Loss: 0.0006, Train MAE: 0.0167 | Val Loss: 0.0006, Val MAE: 0.0149
Epoch 34/100 | Train Loss: 0.0007, Train MAE: 0.0167 | Val Loss: 0.0006, Val MAE: 0.0152
Epoch 35/100 | Train Loss: 0.0006, Train MAE: 0.0162 | Val Loss: 0.0006, Val MAE: 0.0164
Epoch 36/100 | Train Loss: 0.0006, Train MAE: 0.0160 | Val Loss: 0.0006, Val MAE: 0.0147
Epoch 37/100 | Train Loss: 0.0006, Train MAE: 0.0159 | Val Loss: 0.0006, Val MAE: 0.0158
Epoch 38/100 | Train Loss: 0.0006, Train MAE: 0.0160 | Val Loss: 0.0005, Val MAE: 0.0147
Epoch 39/100 | Train Loss: 0.0006, Train MAE: 0.0161 | Val Loss: 0.0006, Val MAE: 0.0146
Epoch 40/100 | Train Loss: 0.0006, Train MAE: 0.0160 | Val Loss: 0.0006, Val MAE: 0.0147
Epoch 41/100 | Train Loss: 0.0006, Train MAE: 0.0159 | Val Loss: 0.0005, Val MAE: 0.0140
Epoch 42/100 | Train Loss: 0.0006, Train MAE: 0.0158 | Val Loss: 0.0005, Val MAE: 0.0146
Epoch 43/100 | Train Loss: 0.0005, Train MAE: 0.0152 | Val Loss: 0.0005, Val MAE: 0.0139
Epoch 44/100 | Train Loss: 0.0005, Train MAE: 0.0153 | Val Loss: 0.0005, Val MAE: 0.0140
Epoch 45/100 | Train Loss: 0.0005, Train MAE: 0.0148 | Val Loss: 0.0005, Val MAE: 0.0139
Epoch 46/100 | Train Loss: 0.0005, Train MAE: 0.0155 | Val Loss: 0.0005, Val MAE: 0.0139
Epoch 47/100 | Train Loss: 0.0005, Train MAE: 0.0147 | Val Loss: 0.0005, Val MAE: 0.0136
Epoch 48/100 | Train Loss: 0.0005, Train MAE: 0.0150 | Val Loss: 0.0005, Val MAE: 0.0145
Epoch 49/100 | Train Loss: 0.0005, Train MAE: 0.0150 | Val Loss: 0.0005, Val MAE: 0.0156
Epoch 50/100 | Train Loss: 0.0005, Train MAE: 0.0149 | Val Loss: 0.0005, Val MAE: 0.0149

Epoch 51/100 | Train Loss: 0.0005, Train MAE: 0.0150 | Val Loss: 0.0005, Val MAE: 0.0141
Epoch 52/100 | Train Loss: 0.0005, Train MAE: 0.0146 | Val Loss: 0.0005, Val MAE: 0.0136
Epoch 53/100 | Train Loss: 0.0005, Train MAE: 0.0148 | Val Loss: 0.0005, Val MAE: 0.0135
Epoch 54/100 | Train Loss: 0.0005, Train MAE: 0.0144 | Val Loss: 0.0005, Val MAE: 0.0133
Epoch 55/100 | Train Loss: 0.0005, Train MAE: 0.0145 | Val Loss: 0.0005, Val MAE: 0.0136
Epoch 56/100 | Train Loss: 0.0005, Train MAE: 0.0147 | Val Loss: 0.0005, Val MAE: 0.0135
Epoch 57/100 | Train Loss: 0.0005, Train MAE: 0.0148 | Val Loss: 0.0005, Val MAE: 0.0136
Epoch 58/100 | Train Loss: 0.0005, Train MAE: 0.0146 | Val Loss: 0.0005, Val MAE: 0.0135
Epoch 59/100 | Train Loss: 0.0005, Train MAE: 0.0146 | Val Loss: 0.0005, Val MAE: 0.0136
Epoch 60/100 | Train Loss: 0.0005, Train MAE: 0.0147 | Val Loss: 0.0005, Val MAE: 0.0134
Epoch 61/100 | Train Loss: 0.0005, Train MAE: 0.0147 | Val Loss: 0.0005, Val MAE: 0.0136
Epoch 62/100 | Train Loss: 0.0005, Train MAE: 0.0147 | Val Loss: 0.0005, Val MAE: 0.0134
Epoch 63/100 | Train Loss: 0.0005, Train MAE: 0.0146 | Val Loss: 0.0005, Val MAE: 0.0134
Epoch 64/100 | Train Loss: 0.0005, Train MAE: 0.0145 | Val Loss: 0.0005, Val MAE: 0.0133
Epoch 65/100 | Train Loss: 0.0005, Train MAE: 0.0144 | Val Loss: 0.0005, Val MAE: 0.0136
Epoch 66/100 | Train Loss: 0.0005, Train MAE: 0.0144 | Val Loss: 0.0005, Val MAE: 0.0133
Epoch 67/100 | Train Loss: 0.0005, Train MAE: 0.0143 | Val Loss: 0.0005, Val MAE: 0.0134
Epoch 68/100 | Train Loss: 0.0005, Train MAE: 0.0141 | Val Loss: 0.0005, Val MAE: 0.0133
Epoch 69/100 | Train Loss: 0.0005, Train MAE: 0.0143 | Val Loss: 0.0005, Val MAE: 0.0135
Epoch 70/100 | Train Loss: 0.0005, Train MAE: 0.0143 | Val Loss: 0.0005, Val MAE: 0.0133
Epoch 71/100 | Train Loss: 0.0005, Train MAE: 0.0143 | Val Loss: 0.0005, Val MAE: 0.0137
Epoch 72/100 | Train Loss: 0.0005, Train MAE: 0.0144 | Val Loss: 0.0005, Val MAE: 0.0133
Epoch 73/100 | Train Loss: 0.0005, Train MAE: 0.0145 | Val Loss: 0.0005, Val MAE: 0.0134
Epoch 74/100 | Train Loss: 0.0004, Train MAE: 0.0138 | Val Loss: 0.0005, Val MAE: 0.0144
Epoch 75/100 | Train Loss: 0.0005, Train MAE: 0.0146 | Val Loss:

0.0005, Val MAE: 0.0140
Epoch 76/100 | Train Loss: 0.0005, Train MAE: 0.0141 | Val Loss: 0.0005, Val MAE: 0.0135
Epoch 77/100 | Train Loss: 0.0004, Train MAE: 0.0140 | Val Loss: 0.0005, Val MAE: 0.0137
Epoch 78/100 | Train Loss: 0.0005, Train MAE: 0.0142 | Val Loss: 0.0005, Val MAE: 0.0135
Epoch 79/100 | Train Loss: 0.0005, Train MAE: 0.0143 | Val Loss: 0.0005, Val MAE: 0.0135
Epoch 80/100 | Train Loss: 0.0005, Train MAE: 0.0144 | Val Loss: 0.0005, Val MAE: 0.0135
Epoch 81/100 | Train Loss: 0.0005, Train MAE: 0.0143 | Val Loss: 0.0005, Val MAE: 0.0135
Epoch 82/100 | Train Loss: 0.0004, Train MAE: 0.0142 | Val Loss: 0.0005, Val MAE: 0.0132
Epoch 83/100 | Train Loss: 0.0005, Train MAE: 0.0142 | Val Loss: 0.0005, Val MAE: 0.0133
Epoch 84/100 | Train Loss: 0.0004, Train MAE: 0.0141 | Val Loss: 0.0005, Val MAE: 0.0134
Epoch 85/100 | Train Loss: 0.0005, Train MAE: 0.0141 | Val Loss: 0.0005, Val MAE: 0.0139
Epoch 86/100 | Train Loss: 0.0004, Train MAE: 0.0139 | Val Loss: 0.0005, Val MAE: 0.0137
Epoch 87/100 | Train Loss: 0.0005, Train MAE: 0.0143 | Val Loss: 0.0005, Val MAE: 0.0135
Epoch 88/100 | Train Loss: 0.0005, Train MAE: 0.0140 | Val Loss: 0.0005, Val MAE: 0.0131
Epoch 89/100 | Train Loss: 0.0005, Train MAE: 0.0140 | Val Loss: 0.0005, Val MAE: 0.0134
Epoch 90/100 | Train Loss: 0.0004, Train MAE: 0.0140 | Val Loss: 0.0005, Val MAE: 0.0137
Epoch 91/100 | Train Loss: 0.0004, Train MAE: 0.0140 | Val Loss: 0.0005, Val MAE: 0.0135
Epoch 92/100 | Train Loss: 0.0004, Train MAE: 0.0141 | Val Loss: 0.0005, Val MAE: 0.0139
Epoch 93/100 | Train Loss: 0.0004, Train MAE: 0.0139 | Val Loss: 0.0005, Val MAE: 0.0140
Epoch 94/100 | Train Loss: 0.0004, Train MAE: 0.0141 | Val Loss: 0.0005, Val MAE: 0.0135
Epoch 95/100 | Train Loss: 0.0004, Train MAE: 0.0140 | Val Loss: 0.0005, Val MAE: 0.0132
Epoch 96/100 | Train Loss: 0.0004, Train MAE: 0.0140 | Val Loss: 0.0005, Val MAE: 0.0137
Epoch 97/100 | Train Loss: 0.0004, Train MAE: 0.0139 | Val Loss: 0.0005, Val MAE: 0.0133
Epoch 98/100 | Train Loss: 0.0004, Train MAE: 0.0143 | Val Loss: 0.0005, Val MAE: 0.0139
Epoch 99/100 | Train Loss: 0.0005, Train MAE: 0.0144 | Val Loss: 0.0005, Val MAE: 0.0138

Epoch 100/100 | Train Loss: 0.0005, Train MAE: 0.0142 | Val Loss: 0.0005, Val MAE: 0.0134
MSE: 0.000458, RMSE: 0.021409, MAE: 0.013393, R2 score: 0.561150

--- Running configuration: w_12_do ---

Window Size: 12, Out Channels 1: 64, Out Channels 2: 32, Kernel 1: 3, Kernel 2: 3, Dropout: 0.2

Epoch 1/100 | Train Loss: 0.0139, Train MAE: 0.0890 | Val Loss: 0.0036, Val MAE: 0.0451

Epoch 2/100 | Train Loss: 0.0025, Train MAE: 0.0357 | Val Loss: 0.0017, Val MAE: 0.0286

Epoch 3/100 | Train Loss: 0.0019, Train MAE: 0.0309 | Val Loss: 0.0016, Val MAE: 0.0262

Epoch 4/100 | Train Loss: 0.0017, Train MAE: 0.0287 | Val Loss: 0.0015, Val MAE: 0.0254

Epoch 5/100 | Train Loss: 0.0016, Train MAE: 0.0281 | Val Loss: 0.0014, Val MAE: 0.0244

Epoch 6/100 | Train Loss: 0.0016, Train MAE: 0.0277 | Val Loss: 0.0014, Val MAE: 0.0237

Epoch 7/100 | Train Loss: 0.0015, Train MAE: 0.0265 | Val Loss: 0.0014, Val MAE: 0.0243

Epoch 8/100 | Train Loss: 0.0015, Train MAE: 0.0264 | Val Loss: 0.0013, Val MAE: 0.0258

Epoch 9/100 | Train Loss: 0.0014, Train MAE: 0.0253 | Val Loss: 0.0013, Val MAE: 0.0243

Epoch 10/100 | Train Loss: 0.0013, Train MAE: 0.0251 | Val Loss: 0.0012, Val MAE: 0.0220

Epoch 11/100 | Train Loss: 0.0012, Train MAE: 0.0239 | Val Loss: 0.0011, Val MAE: 0.0204

Epoch 12/100 | Train Loss: 0.0012, Train MAE: 0.0237 | Val Loss: 0.0011, Val MAE: 0.0197

Epoch 13/100 | Train Loss: 0.0011, Train MAE: 0.0232 | Val Loss: 0.0010, Val MAE: 0.0204

Epoch 14/100 | Train Loss: 0.0011, Train MAE: 0.0228 | Val Loss: 0.0010, Val MAE: 0.0211

Epoch 15/100 | Train Loss: 0.0011, Train MAE: 0.0220 | Val Loss: 0.0010, Val MAE: 0.0210

Epoch 16/100 | Train Loss: 0.0011, Train MAE: 0.0222 | Val Loss: 0.0009, Val MAE: 0.0191

Epoch 17/100 | Train Loss: 0.0010, Train MAE: 0.0214 | Val Loss: 0.0009, Val MAE: 0.0195

Epoch 18/100 | Train Loss: 0.0010, Train MAE: 0.0220 | Val Loss: 0.0009, Val MAE: 0.0189

Epoch 19/100 | Train Loss: 0.0010, Train MAE: 0.0213 | Val Loss: 0.0009, Val MAE: 0.0182

Epoch 20/100 | Train Loss: 0.0010, Train MAE: 0.0212 | Val Loss: 0.0008, Val MAE: 0.0188

Epoch 21/100 | Train Loss: 0.0009, Train MAE: 0.0210 | Val Loss:

0.0009, Val MAE: 0.0198
Epoch 22/100 | Train Loss: 0.0009, Train MAE: 0.0201 | Val Loss: 0.0008, Val MAE: 0.0198
Epoch 23/100 | Train Loss: 0.0009, Train MAE: 0.0208 | Val Loss: 0.0008, Val MAE: 0.0179
Epoch 24/100 | Train Loss: 0.0008, Train MAE: 0.0203 | Val Loss: 0.0007, Val MAE: 0.0179
Epoch 25/100 | Train Loss: 0.0008, Train MAE: 0.0195 | Val Loss: 0.0008, Val MAE: 0.0186
Epoch 26/100 | Train Loss: 0.0008, Train MAE: 0.0192 | Val Loss: 0.0007, Val MAE: 0.0182
Epoch 27/100 | Train Loss: 0.0007, Train MAE: 0.0189 | Val Loss: 0.0007, Val MAE: 0.0170
Epoch 28/100 | Train Loss: 0.0007, Train MAE: 0.0189 | Val Loss: 0.0007, Val MAE: 0.0171
Epoch 29/100 | Train Loss: 0.0007, Train MAE: 0.0185 | Val Loss: 0.0006, Val MAE: 0.0161
Epoch 30/100 | Train Loss: 0.0007, Train MAE: 0.0193 | Val Loss: 0.0006, Val MAE: 0.0162
Epoch 31/100 | Train Loss: 0.0007, Train MAE: 0.0189 | Val Loss: 0.0006, Val MAE: 0.0159
Epoch 32/100 | Train Loss: 0.0007, Train MAE: 0.0185 | Val Loss: 0.0006, Val MAE: 0.0157
Epoch 33/100 | Train Loss: 0.0007, Train MAE: 0.0184 | Val Loss: 0.0006, Val MAE: 0.0159
Epoch 34/100 | Train Loss: 0.0006, Train MAE: 0.0179 | Val Loss: 0.0006, Val MAE: 0.0158
Epoch 35/100 | Train Loss: 0.0006, Train MAE: 0.0178 | Val Loss: 0.0006, Val MAE: 0.0165
Epoch 36/100 | Train Loss: 0.0007, Train MAE: 0.0185 | Val Loss: 0.0006, Val MAE: 0.0166
Epoch 37/100 | Train Loss: 0.0006, Train MAE: 0.0176 | Val Loss: 0.0006, Val MAE: 0.0157
Epoch 38/100 | Train Loss: 0.0006, Train MAE: 0.0175 | Val Loss: 0.0006, Val MAE: 0.0155
Epoch 39/100 | Train Loss: 0.0006, Train MAE: 0.0172 | Val Loss: 0.0005, Val MAE: 0.0155
Epoch 40/100 | Train Loss: 0.0006, Train MAE: 0.0173 | Val Loss: 0.0005, Val MAE: 0.0149
Epoch 41/100 | Train Loss: 0.0006, Train MAE: 0.0169 | Val Loss: 0.0005, Val MAE: 0.0150
Epoch 42/100 | Train Loss: 0.0006, Train MAE: 0.0172 | Val Loss: 0.0005, Val MAE: 0.0153
Epoch 43/100 | Train Loss: 0.0006, Train MAE: 0.0168 | Val Loss: 0.0005, Val MAE: 0.0149
Epoch 44/100 | Train Loss: 0.0005, Train MAE: 0.0166 | Val Loss: 0.0005, Val MAE: 0.0147
Epoch 45/100 | Train Loss: 0.0006, Train MAE: 0.0171 | Val Loss: 0.0005, Val MAE: 0.0146

Epoch 46/100 | Train Loss: 0.0006, Train MAE: 0.0172 | Val Loss: 0.0006, Val MAE: 0.0179
Epoch 47/100 | Train Loss: 0.0006, Train MAE: 0.0175 | Val Loss: 0.0005, Val MAE: 0.0149
Epoch 48/100 | Train Loss: 0.0006, Train MAE: 0.0167 | Val Loss: 0.0005, Val MAE: 0.0146
Epoch 49/100 | Train Loss: 0.0005, Train MAE: 0.0161 | Val Loss: 0.0005, Val MAE: 0.0146
Epoch 50/100 | Train Loss: 0.0006, Train MAE: 0.0169 | Val Loss: 0.0005, Val MAE: 0.0154
Epoch 51/100 | Train Loss: 0.0005, Train MAE: 0.0163 | Val Loss: 0.0005, Val MAE: 0.0147
Epoch 52/100 | Train Loss: 0.0005, Train MAE: 0.0163 | Val Loss: 0.0005, Val MAE: 0.0147
Epoch 53/100 | Train Loss: 0.0005, Train MAE: 0.0161 | Val Loss: 0.0005, Val MAE: 0.0146
Epoch 54/100 | Train Loss: 0.0005, Train MAE: 0.0163 | Val Loss: 0.0005, Val MAE: 0.0149
Epoch 55/100 | Train Loss: 0.0005, Train MAE: 0.0160 | Val Loss: 0.0005, Val MAE: 0.0143
Epoch 56/100 | Train Loss: 0.0005, Train MAE: 0.0159 | Val Loss: 0.0005, Val MAE: 0.0146
Epoch 57/100 | Train Loss: 0.0005, Train MAE: 0.0159 | Val Loss: 0.0005, Val MAE: 0.0148
Epoch 58/100 | Train Loss: 0.0005, Train MAE: 0.0158 | Val Loss: 0.0005, Val MAE: 0.0142
Epoch 59/100 | Train Loss: 0.0005, Train MAE: 0.0159 | Val Loss: 0.0005, Val MAE: 0.0143
Epoch 60/100 | Train Loss: 0.0005, Train MAE: 0.0160 | Val Loss: 0.0005, Val MAE: 0.0147
Epoch 61/100 | Train Loss: 0.0005, Train MAE: 0.0157 | Val Loss: 0.0005, Val MAE: 0.0144
Epoch 62/100 | Train Loss: 0.0005, Train MAE: 0.0158 | Val Loss: 0.0005, Val MAE: 0.0146
Epoch 63/100 | Train Loss: 0.0005, Train MAE: 0.0153 | Val Loss: 0.0005, Val MAE: 0.0141
Epoch 64/100 | Train Loss: 0.0005, Train MAE: 0.0157 | Val Loss: 0.0005, Val MAE: 0.0146
Epoch 65/100 | Train Loss: 0.0005, Train MAE: 0.0157 | Val Loss: 0.0005, Val MAE: 0.0139
Epoch 66/100 | Train Loss: 0.0005, Train MAE: 0.0152 | Val Loss: 0.0005, Val MAE: 0.0144
Epoch 67/100 | Train Loss: 0.0005, Train MAE: 0.0152 | Val Loss: 0.0005, Val MAE: 0.0143
Epoch 68/100 | Train Loss: 0.0005, Train MAE: 0.0150 | Val Loss: 0.0005, Val MAE: 0.0146
Epoch 69/100 | Train Loss: 0.0005, Train MAE: 0.0151 | Val Loss: 0.0005, Val MAE: 0.0149
Epoch 70/100 | Train Loss: 0.0004, Train MAE: 0.0149 | Val Loss: 0.0005, Val MAE: 0.0143

Epoch 71/100 | Train Loss: 0.0005, Train MAE: 0.0151 | Val Loss: 0.0005, Val MAE: 0.0139
Epoch 72/100 | Train Loss: 0.0004, Train MAE: 0.0146 | Val Loss: 0.0005, Val MAE: 0.0140
Epoch 73/100 | Train Loss: 0.0004, Train MAE: 0.0145 | Val Loss: 0.0005, Val MAE: 0.0143
Epoch 74/100 | Train Loss: 0.0004, Train MAE: 0.0148 | Val Loss: 0.0005, Val MAE: 0.0139
Epoch 75/100 | Train Loss: 0.0005, Train MAE: 0.0149 | Val Loss: 0.0005, Val MAE: 0.0141
Epoch 76/100 | Train Loss: 0.0005, Train MAE: 0.0149 | Val Loss: 0.0005, Val MAE: 0.0150
Epoch 77/100 | Train Loss: 0.0004, Train MAE: 0.0148 | Val Loss: 0.0005, Val MAE: 0.0149
Epoch 78/100 | Train Loss: 0.0005, Train MAE: 0.0152 | Val Loss: 0.0005, Val MAE: 0.0139
Epoch 79/100 | Train Loss: 0.0005, Train MAE: 0.0147 | Val Loss: 0.0005, Val MAE: 0.0139
Epoch 80/100 | Train Loss: 0.0004, Train MAE: 0.0145 | Val Loss: 0.0005, Val MAE: 0.0140
Epoch 81/100 | Train Loss: 0.0005, Train MAE: 0.0147 | Val Loss: 0.0005, Val MAE: 0.0138
Epoch 82/100 | Train Loss: 0.0004, Train MAE: 0.0144 | Val Loss: 0.0005, Val MAE: 0.0139
Epoch 83/100 | Train Loss: 0.0004, Train MAE: 0.0145 | Val Loss: 0.0005, Val MAE: 0.0140
Epoch 84/100 | Train Loss: 0.0004, Train MAE: 0.0146 | Val Loss: 0.0004, Val MAE: 0.0143
Epoch 85/100 | Train Loss: 0.0004, Train MAE: 0.0145 | Val Loss: 0.0004, Val MAE: 0.0137
Epoch 86/100 | Train Loss: 0.0004, Train MAE: 0.0141 | Val Loss: 0.0005, Val MAE: 0.0146
Epoch 87/100 | Train Loss: 0.0004, Train MAE: 0.0142 | Val Loss: 0.0004, Val MAE: 0.0137
Epoch 88/100 | Train Loss: 0.0004, Train MAE: 0.0143 | Val Loss: 0.0004, Val MAE: 0.0139
Epoch 89/100 | Train Loss: 0.0004, Train MAE: 0.0140 | Val Loss: 0.0004, Val MAE: 0.0138
Epoch 90/100 | Train Loss: 0.0004, Train MAE: 0.0140 | Val Loss: 0.0004, Val MAE: 0.0137
Epoch 91/100 | Train Loss: 0.0004, Train MAE: 0.0143 | Val Loss: 0.0004, Val MAE: 0.0137
Epoch 92/100 | Train Loss: 0.0004, Train MAE: 0.0141 | Val Loss: 0.0004, Val MAE: 0.0137
Epoch 93/100 | Train Loss: 0.0004, Train MAE: 0.0143 | Val Loss: 0.0004, Val MAE: 0.0138
Epoch 94/100 | Train Loss: 0.0004, Train MAE: 0.0143 | Val Loss: 0.0004, Val MAE: 0.0139
Epoch 95/100 | Train Loss: 0.0004, Train MAE: 0.0138 | Val Loss:

0.0004, Val MAE: 0.0139
Epoch 96/100 | Train Loss: 0.0004, Train MAE: 0.0138 | Val Loss:
0.0004, Val MAE: 0.0138
Epoch 97/100 | Train Loss: 0.0004, Train MAE: 0.0140 | Val Loss:
0.0004, Val MAE: 0.0139
Epoch 98/100 | Train Loss: 0.0004, Train MAE: 0.0137 | Val Loss:
0.0004, Val MAE: 0.0136
Epoch 99/100 | Train Loss: 0.0004, Train MAE: 0.0137 | Val Loss:
0.0004, Val MAE: 0.0138
Epoch 100/100 | Train Loss: 0.0004, Train MAE: 0.0140 | Val Loss:
0.0005, Val MAE: 0.0136
MSE: 0.000453, RMSE: 0.021290, MAE: 0.013603, R2 score: 0.565402

--- Running configuration: w_24_do ---

Window Size: 24, Out Channels 1: 64, Out Channels 2: 32, Kernel 1: 3,
Kernel 2: 3, Dropout: 0.2
Epoch 1/100 | Train Loss: 0.0105, Train MAE: 0.0776 | Val Loss:
0.0027, Val MAE: 0.0400
Epoch 2/100 | Train Loss: 0.0020, Train MAE: 0.0321 | Val Loss:
0.0016, Val MAE: 0.0263
Epoch 3/100 | Train Loss: 0.0016, Train MAE: 0.0278 | Val Loss:
0.0015, Val MAE: 0.0261
Epoch 4/100 | Train Loss: 0.0015, Train MAE: 0.0265 | Val Loss:
0.0015, Val MAE: 0.0233
Epoch 5/100 | Train Loss: 0.0014, Train MAE: 0.0251 | Val Loss:
0.0013, Val MAE: 0.0245
Epoch 6/100 | Train Loss: 0.0012, Train MAE: 0.0242 | Val Loss:
0.0012, Val MAE: 0.0245
Epoch 7/100 | Train Loss: 0.0011, Train MAE: 0.0232 | Val Loss:
0.0010, Val MAE: 0.0218
Epoch 8/100 | Train Loss: 0.0010, Train MAE: 0.0212 | Val Loss:
0.0009, Val MAE: 0.0197
Epoch 9/100 | Train Loss: 0.0009, Train MAE: 0.0202 | Val Loss:
0.0008, Val MAE: 0.0183
Epoch 10/100 | Train Loss: 0.0008, Train MAE: 0.0201 | Val Loss:
0.0008, Val MAE: 0.0184
Epoch 11/100 | Train Loss: 0.0008, Train MAE: 0.0200 | Val Loss:
0.0007, Val MAE: 0.0176
Epoch 12/100 | Train Loss: 0.0007, Train MAE: 0.0186 | Val Loss:
0.0007, Val MAE: 0.0173
Epoch 13/100 | Train Loss: 0.0007, Train MAE: 0.0185 | Val Loss:
0.0006, Val MAE: 0.0162
Epoch 14/100 | Train Loss: 0.0006, Train MAE: 0.0179 | Val Loss:
0.0006, Val MAE: 0.0170
Epoch 15/100 | Train Loss: 0.0006, Train MAE: 0.0173 | Val Loss:
0.0005, Val MAE: 0.0151
Epoch 16/100 | Train Loss: 0.0006, Train MAE: 0.0176 | Val Loss:
0.0005, Val MAE: 0.0150

Epoch 17/100 | Train Loss: 0.0006, Train MAE: 0.0169 | Val Loss: 0.0005, Val MAE: 0.0148
Epoch 18/100 | Train Loss: 0.0005, Train MAE: 0.0169 | Val Loss: 0.0005, Val MAE: 0.0149
Epoch 19/100 | Train Loss: 0.0005, Train MAE: 0.0163 | Val Loss: 0.0004, Val MAE: 0.0145
Epoch 20/100 | Train Loss: 0.0005, Train MAE: 0.0160 | Val Loss: 0.0004, Val MAE: 0.0145
Epoch 21/100 | Train Loss: 0.0005, Train MAE: 0.0162 | Val Loss: 0.0004, Val MAE: 0.0136
Epoch 22/100 | Train Loss: 0.0005, Train MAE: 0.0157 | Val Loss: 0.0004, Val MAE: 0.0134
Epoch 23/100 | Train Loss: 0.0005, Train MAE: 0.0158 | Val Loss: 0.0004, Val MAE: 0.0137
Epoch 24/100 | Train Loss: 0.0005, Train MAE: 0.0158 | Val Loss: 0.0004, Val MAE: 0.0137
Epoch 25/100 | Train Loss: 0.0004, Train MAE: 0.0151 | Val Loss: 0.0003, Val MAE: 0.0131
Epoch 26/100 | Train Loss: 0.0004, Train MAE: 0.0150 | Val Loss: 0.0003, Val MAE: 0.0130
Epoch 27/100 | Train Loss: 0.0004, Train MAE: 0.0155 | Val Loss: 0.0003, Val MAE: 0.0131
Epoch 28/100 | Train Loss: 0.0004, Train MAE: 0.0147 | Val Loss: 0.0003, Val MAE: 0.0130
Epoch 29/100 | Train Loss: 0.0004, Train MAE: 0.0151 | Val Loss: 0.0003, Val MAE: 0.0133
Epoch 30/100 | Train Loss: 0.0004, Train MAE: 0.0145 | Val Loss: 0.0003, Val MAE: 0.0125
Epoch 31/100 | Train Loss: 0.0004, Train MAE: 0.0143 | Val Loss: 0.0003, Val MAE: 0.0126
Epoch 32/100 | Train Loss: 0.0004, Train MAE: 0.0142 | Val Loss: 0.0003, Val MAE: 0.0125
Epoch 33/100 | Train Loss: 0.0003, Train MAE: 0.0141 | Val Loss: 0.0003, Val MAE: 0.0122
Epoch 34/100 | Train Loss: 0.0003, Train MAE: 0.0142 | Val Loss: 0.0003, Val MAE: 0.0123
Epoch 35/100 | Train Loss: 0.0003, Train MAE: 0.0140 | Val Loss: 0.0002, Val MAE: 0.0119
Epoch 36/100 | Train Loss: 0.0003, Train MAE: 0.0142 | Val Loss: 0.0003, Val MAE: 0.0125
Epoch 37/100 | Train Loss: 0.0003, Train MAE: 0.0142 | Val Loss: 0.0003, Val MAE: 0.0127
Epoch 38/100 | Train Loss: 0.0003, Train MAE: 0.0139 | Val Loss: 0.0002, Val MAE: 0.0119
Epoch 39/100 | Train Loss: 0.0003, Train MAE: 0.0138 | Val Loss: 0.0003, Val MAE: 0.0122
Epoch 40/100 | Train Loss: 0.0003, Train MAE: 0.0135 | Val Loss: 0.0002, Val MAE: 0.0118
Epoch 41/100 | Train Loss: 0.0003, Train MAE: 0.0135 | Val Loss:

0.0002, Val MAE: 0.0121
Epoch 42/100 | Train Loss: 0.0003, Train MAE: 0.0139 | Val Loss:
0.0002, Val MAE: 0.0120
Epoch 43/100 | Train Loss: 0.0003, Train MAE: 0.0137 | Val Loss:
0.0002, Val MAE: 0.0117
Epoch 44/100 | Train Loss: 0.0003, Train MAE: 0.0134 | Val Loss:
0.0002, Val MAE: 0.0121
Epoch 45/100 | Train Loss: 0.0003, Train MAE: 0.0134 | Val Loss:
0.0002, Val MAE: 0.0121
Epoch 46/100 | Train Loss: 0.0003, Train MAE: 0.0137 | Val Loss:
0.0002, Val MAE: 0.0118
Epoch 47/100 | Train Loss: 0.0003, Train MAE: 0.0134 | Val Loss:
0.0002, Val MAE: 0.0120
Epoch 48/100 | Train Loss: 0.0003, Train MAE: 0.0132 | Val Loss:
0.0002, Val MAE: 0.0120
Epoch 49/100 | Train Loss: 0.0003, Train MAE: 0.0134 | Val Loss:
0.0003, Val MAE: 0.0132
Epoch 50/100 | Train Loss: 0.0003, Train MAE: 0.0137 | Val Loss:
0.0003, Val MAE: 0.0127
Epoch 51/100 | Train Loss: 0.0003, Train MAE: 0.0135 | Val Loss:
0.0002, Val MAE: 0.0118
Epoch 52/100 | Train Loss: 0.0003, Train MAE: 0.0131 | Val Loss:
0.0002, Val MAE: 0.0118
Epoch 53/100 | Train Loss: 0.0003, Train MAE: 0.0130 | Val Loss:
0.0003, Val MAE: 0.0126
Epoch 54/100 | Train Loss: 0.0003, Train MAE: 0.0134 | Val Loss:
0.0002, Val MAE: 0.0118
Epoch 55/100 | Train Loss: 0.0003, Train MAE: 0.0138 | Val Loss:
0.0002, Val MAE: 0.0119
Epoch 56/100 | Train Loss: 0.0003, Train MAE: 0.0138 | Val Loss:
0.0003, Val MAE: 0.0126
Epoch 57/100 | Train Loss: 0.0003, Train MAE: 0.0131 | Val Loss:
0.0002, Val MAE: 0.0123
Epoch 58/100 | Train Loss: 0.0003, Train MAE: 0.0136 | Val Loss:
0.0002, Val MAE: 0.0120
Epoch 59/100 | Train Loss: 0.0003, Train MAE: 0.0129 | Val Loss:
0.0002, Val MAE: 0.0116
Epoch 60/100 | Train Loss: 0.0003, Train MAE: 0.0128 | Val Loss:
0.0002, Val MAE: 0.0120
Epoch 61/100 | Train Loss: 0.0003, Train MAE: 0.0127 | Val Loss:
0.0003, Val MAE: 0.0129
Epoch 62/100 | Train Loss: 0.0003, Train MAE: 0.0128 | Val Loss:
0.0002, Val MAE: 0.0120
Epoch 63/100 | Train Loss: 0.0003, Train MAE: 0.0132 | Val Loss:
0.0002, Val MAE: 0.0120
Epoch 64/100 | Train Loss: 0.0003, Train MAE: 0.0134 | Val Loss:
0.0002, Val MAE: 0.0122
Epoch 65/100 | Train Loss: 0.0003, Train MAE: 0.0130 | Val Loss:
0.0002, Val MAE: 0.0122

Epoch 66/100 | Train Loss: 0.0003, Train MAE: 0.0128 | Val Loss: 0.0002, Val MAE: 0.0117
Epoch 67/100 | Train Loss: 0.0003, Train MAE: 0.0126 | Val Loss: 0.0002, Val MAE: 0.0117
Epoch 68/100 | Train Loss: 0.0003, Train MAE: 0.0129 | Val Loss: 0.0002, Val MAE: 0.0124
Epoch 69/100 | Train Loss: 0.0003, Train MAE: 0.0127 | Val Loss: 0.0002, Val MAE: 0.0117
Epoch 70/100 | Train Loss: 0.0003, Train MAE: 0.0132 | Val Loss: 0.0002, Val MAE: 0.0116
Epoch 71/100 | Train Loss: 0.0003, Train MAE: 0.0127 | Val Loss: 0.0002, Val MAE: 0.0120
Epoch 72/100 | Train Loss: 0.0003, Train MAE: 0.0126 | Val Loss: 0.0003, Val MAE: 0.0128
Epoch 73/100 | Train Loss: 0.0003, Train MAE: 0.0126 | Val Loss: 0.0002, Val MAE: 0.0119
Epoch 74/100 | Train Loss: 0.0003, Train MAE: 0.0129 | Val Loss: 0.0002, Val MAE: 0.0114
Epoch 75/100 | Train Loss: 0.0003, Train MAE: 0.0126 | Val Loss: 0.0002, Val MAE: 0.0121
Epoch 76/100 | Train Loss: 0.0003, Train MAE: 0.0124 | Val Loss: 0.0002, Val MAE: 0.0116
Epoch 77/100 | Train Loss: 0.0003, Train MAE: 0.0124 | Val Loss: 0.0002, Val MAE: 0.0117
Epoch 78/100 | Train Loss: 0.0003, Train MAE: 0.0125 | Val Loss: 0.0002, Val MAE: 0.0117
Epoch 79/100 | Train Loss: 0.0003, Train MAE: 0.0124 | Val Loss: 0.0002, Val MAE: 0.0118
Epoch 80/100 | Train Loss: 0.0003, Train MAE: 0.0128 | Val Loss: 0.0002, Val MAE: 0.0117
Epoch 81/100 | Train Loss: 0.0003, Train MAE: 0.0124 | Val Loss: 0.0002, Val MAE: 0.0117
Epoch 82/100 | Train Loss: 0.0002, Train MAE: 0.0122 | Val Loss: 0.0002, Val MAE: 0.0118
Epoch 83/100 | Train Loss: 0.0003, Train MAE: 0.0124 | Val Loss: 0.0002, Val MAE: 0.0119
Epoch 84/100 | Train Loss: 0.0003, Train MAE: 0.0124 | Val Loss: 0.0003, Val MAE: 0.0128
Epoch 85/100 | Train Loss: 0.0003, Train MAE: 0.0123 | Val Loss: 0.0002, Val MAE: 0.0116
Epoch 86/100 | Train Loss: 0.0003, Train MAE: 0.0123 | Val Loss: 0.0002, Val MAE: 0.0119
Epoch 87/100 | Train Loss: 0.0003, Train MAE: 0.0124 | Val Loss: 0.0002, Val MAE: 0.0116
Epoch 88/100 | Train Loss: 0.0002, Train MAE: 0.0122 | Val Loss: 0.0002, Val MAE: 0.0115
Epoch 89/100 | Train Loss: 0.0003, Train MAE: 0.0124 | Val Loss: 0.0002, Val MAE: 0.0118
Epoch 90/100 | Train Loss: 0.0003, Train MAE: 0.0124 | Val Loss:

```

0.0002, Val MAE: 0.0116
Epoch 91/100 | Train Loss: 0.0003, Train MAE: 0.0123 | Val Loss:
0.0002, Val MAE: 0.0118
Epoch 92/100 | Train Loss: 0.0003, Train MAE: 0.0125 | Val Loss:
0.0003, Val MAE: 0.0127
Epoch 93/100 | Train Loss: 0.0003, Train MAE: 0.0123 | Val Loss:
0.0002, Val MAE: 0.0116
Epoch 94/100 | Train Loss: 0.0002, Train MAE: 0.0121 | Val Loss:
0.0002, Val MAE: 0.0115
Epoch 95/100 | Train Loss: 0.0002, Train MAE: 0.0120 | Val Loss:
0.0002, Val MAE: 0.0117
Epoch 96/100 | Train Loss: 0.0002, Train MAE: 0.0121 | Val Loss:
0.0002, Val MAE: 0.0117
Epoch 97/100 | Train Loss: 0.0002, Train MAE: 0.0120 | Val Loss:
0.0002, Val MAE: 0.0116
Epoch 98/100 | Train Loss: 0.0002, Train MAE: 0.0121 | Val Loss:
0.0002, Val MAE: 0.0114
Epoch 99/100 | Train Loss: 0.0003, Train MAE: 0.0124 | Val Loss:
0.0002, Val MAE: 0.0116
Epoch 100/100 | Train Loss: 0.0002, Train MAE: 0.0120 | Val Loss:
0.0002, Val MAE: 0.0116
MSE: 0.000226, RMSE: 0.015038, MAE: 0.011633, R2 score: 0.679580

```

```

{"summary": "{\n  \"name\": \"summary_1dConv_df\",\n  \"rows\": 9,\n  \"fields\": [\n    {\n      \"column\": \"Config\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 9,\n        \"samples\": [\n          \"w_12_do\",\n          \"w_12\",\n          \"w_12_ks5_128_64\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      {\n        \"column\": \"Window Size\",\n        \"properties\": {\n          \"dtype\": \"number\",\n          \"std\": 6,\n          \"min\": 6,\n          \"max\": 24,\n          \"num_unique_values\": 3,\n          \"samples\": [\n            6,\n            12,\n            24\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n        },\n        {\n          \"column\": \"Out Channels 1\",\n          \"properties\": {\n            \"dtype\": \"number\",\n            \"std\": 25,\n            \"min\": 32,\n            \"max\": 128,\n            \"num_unique_values\": 3,\n            \"samples\": [\n              64,\n              32,\n              128\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\"\n          },\n          {\n            \"column\": \"Out Channels 2\",\n            \"properties\": {\n              \"dtype\": \"number\",\n              \"std\": 12,\n              \"min\": 16,\n              \"max\": 64,\n              \"num_unique_values\": 3,\n              \"samples\": [\n                32,\n                16,\n                64\n              ],\n              \"semantic_type\": \"\",\n              \"description\": \"\"\n            },\n            {\n              \"column\": \"Kernel Size 1\",\n              \"properties\": {\n                \"dtype\": \"number\",\n                \"std\": 0,\n                \"min\": 3,\n                \"max\": 5,\n                \"num_unique_values\": 2,\n                \"samples\": [\n                  5,\n                  3\n                ],\n                \"semantic_type\": \"\"

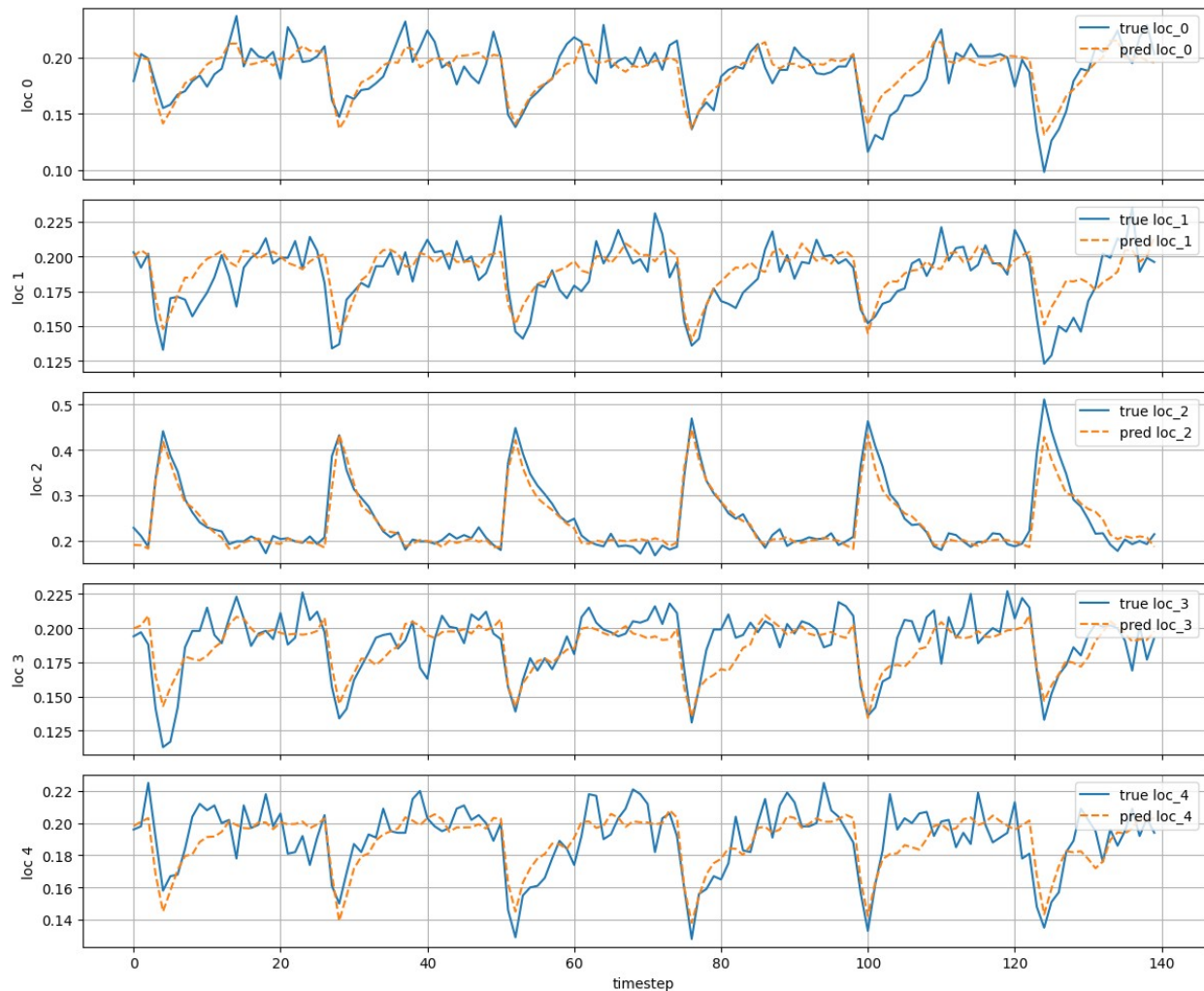
```



```

axes[-1].set_xlabel("timestep")
plt.tight_layout()
plt.show()

```



```

from sklearn.metrics import r2_score, mean_squared_error

```

```

r2 = r2_score(y_test, y_pred)
print(f'MSE: {mean_squared_error(y_test, y_pred)} RMSE: {np.sqrt(mean_squared_error(y_test,y_pred))} R2 score: {r2_score(y_test, y_pred)}')

```

```

MSE: 0.0002519343727519398 RMSE: 0.015872440667771916 R2 score: 0.6701296678752013

```

2.5 Tasks

Write a short report containing the following items. Use plots and tables. Make comments. Deliver as a PDF file.

- Section 2.1 (15%)
 - Select number of epochs being a tradeoff between processing time and accuracy.
 - epochs = 75, batch_size = 512
 - Analyze the confusion matrices. Indicate which articles were misclassified and why.
 - Compare results for TensorFlow and PyTorch for identical hiperparameters. Are they the same?
- Section 2.2 (15%)
 - Compute numbers of parameters CNN depeneding on input size, filter size and depth. Present results as a table
- Section 2.3 (30%)
 - Define configurations and perform test. Use various models and hyperparameters. Gather results in a table.
- Section 2.4 (Use Tensorflow or PyTorch) (30%)
 - Experiment with various window sizes, e.g. 6,12, 24
 - Use different numbers of filters and filter width (kernel sizes: k s)
 - You may also try to add a Dropout layer for regularization
 - Present results in a table. Report parameters and obtained scores (MAE, RMSE, R2). Display selected plots.
- Answer the following questions using 1-2 concise sentences (10%):
 1. In the case of binary classification, how many output neurons and activation functions can be used?
 2. What are the padding and stride parameters used in convolution responsible for?
 3. What parameters of the convolution network are modified during training?
 4. Why for image processing we used 2D convolution and for time series 1D